

UZ++

DASTURLASH TILI

To'liq o'quv qo'llanma

Shaxriyor Malikov

2026-yil, may

v2.2.1 versiyasi asosida tayyorlangan

MIT litsenziyasi

Mundarija

1	01 · Kirish — Dasturlash dunyosiga xush kelibsiz	1
1.1	Dasturlash nima?	1
1.2	uz++ — bu qanday til?	2
1.3	uz++ ichida nima sodir boʻladi?	3
1.4	Oʻrnatish	6
1.5	Birinchi dastur: “Salom, dunyo!”	7
1.6	Izohlar — kelajakdagi oʻzingizga xat	10
1.7	Qatorlar va koʻrsatmalar	11
1.8	Birinchi interaktiv dastur	12
1.9	Tilning asosiy elementlari	14
1.10	Eng koʻp uchraydigan birinchi xatolar	16
1.11	Asboblari — dasturchi sherigi	17
1.12	Bob boʻyicha mashqlar	18
1.13	Xulosa — biz bu bobda nimani oʻrgandik	19
2	02 · Oʻzgaruvchilar va maʼlumot turlari	21
2.1	Oʻzgaruvchi nima?	21
2.2	Asosiy maʼlumot turlari	22
2.3	Oʻzgaruvchi eʼlon qilish usullari	24
2.4	Oʻzgarmaslar (Konstantalar)	25
2.5	Tur oʻzgartirish (Type casting)	26
2.6	Oʻzgaruvchining koʻrinish sohasi (Scope)	27
2.7	Qiymat berish va operatorlar	29
2.8	Amaliy misol: Talabaning maʼlumotnomasi	29
2.9	Eng koʻp uchraydigan xatolar	31
2.10	Bob boʻyicha mashqlar	32
2.11	Xulosa	33
3	03 · Operatorlar va ifodalar	35
3.1	Ifoda nima?	35
3.2	Arifmetik operatorlar	35
3.3	Taqqoslash operatorlari	37
3.4	Mantiqiy operatorlar	38
3.5	Bit operatorlari	40
3.6	Qiymat berish operatorlari	42
3.7	Boshqa operatorlar	42
3.8	Operatorlar ustunligi (Precedence)	44
3.9	Amaliy misol: Kvadrat tenglama yechish	44
3.10	Eng koʻp uchraydigan xatolar	46
3.11	Bob boʻyicha mashqlar	47
3.12	Xulosa	48
4	04 · Boshqaruv tuzilmalari	49

4.1	Dastur oqimi nima?	49
4.2	Shart — agar operatori	49
4.3	tanlash — bir nechta yo'naltirishlardan tanlash	52
4.4	Sikl — davr (while)	53
4.5	Sikl — uchun (for)	55
4.6	Sikl ichida sikl (Nested loops)	56
4.7	Sikldan chiqish va o'tkazib yuborish	57
4.8	Cheksiz sikllar va ulardan qutilish	58
4.9	Amaliy misollar	59
4.10	Eng ko'p uchraydigan xatolar	60
4.11	Bob bo'yicha mashqlar	62
4.12	Xulosa	63
5	05 · Funksiyalar	65
5.1	Funksiya nima va nima uchun kerak?	65
5.2	Funksiyaning anatomiyasi	66
5.3	Parametrlar	66
5.4	Qaytarish (return)	67
5.5	Parametr berish usullari	69
5.6	Standart parametr qiymatlari	70
5.7	Funksiya overloading (qayta yuklash)	70
5.8	Rekursiya — funksiya o'zini chaqiradi	71
5.9	Lambda ifodalar	72
5.10	Inline funksiyalar	73
5.11	Funksiya shablonlari	73
5.12	Amaliy misol: Matematik funksiyalar to'plami	74
5.13	Eng ko'p uchraydigan xatolar	75
5.14	Bob bo'yicha mashqlar	76
5.15	Xulosa	77
6	06 · Ko'rsatkichlar va havolalar	79
6.1	Bu bobning maqsadi	79
6.2	Xotira tashkil etilishi	79
6.3	Havola (Reference) — sodda boshlash	80
6.4	Ko'rsatkich (Pointer)	81
6.5	nullptr — bo'sh ko'rsatkich	82
6.6	Dinamik xotira: yangi va o'chirish	82
6.7	Aqlli ko'rsatkichlar (Smart Pointers)	83
6.8	Ko'rsatkich arifmetikasi	84
6.9	Ko'rsatkich va const	85
6.10	Amaliy misol: Bog'langan ro'yxat (Linked List)	85
6.11	Eng ko'p uchraydigan xatolar	86
6.12	Bob bo'yicha mashqlar	87
6.13	Xulosa	88
7	07 · Massivlar va to'plamlar	89
7.1	Nima uchun to'plamlar kerak?	89
7.2	Massivlar (Statik)	89
7.3	Vektor (vektor) — dinamik massiv	91
7.4	std::map — tartiblangan xarita	93
7.5	std::set — takrorlanmas to'plam	94
7.6	Boshqa ketma-ketlik to'plamlari	95
7.7	std::pair — juftlik	96

7.8	<code>std::tuple</code> — uchlik va undan ko'p	96
7.9	<code>std::optional</code> — bo'lishi mumkin	96
7.10	Amaliy misol: Talabalar jurnali	97
7.11	Eng ko'p uchraydigan xatolar	98
7.12	Bob bo'yicha mashqlar	99
7.13	Xulosa	100
8	08 · Sinfilar — Obyektga Yo'naltirilgan Dasturlash	101
8.1	OOP nima va nima uchun kerak?	101
8.2	Sinf e'lon qilish	102
8.3	Inkapsulyatsiya — yopiq va ochiq	103
8.4	Konstruktor va destruktor	104
8.5	Meros (Inheritance)	106
8.6	Virtual metodlar va polimorfizm	108
8.7	Abstrakt sinf	109
8.8	Operatorlarni qayta yuklash (Operator Overloading)	109
8.9	<code>static</code> a'zolar	110
8.10	<code>dost</code> (friend) deklaratsiyasi	111
8.11	Amaliy misol: Bank hisobi tizimi	111
8.12	Eng ko'p uchraydigan xatolar	112
8.13	Bob bo'yicha mashqlar	113
8.14	Xulosa	114
9	09 · Shablonlar — Umumlashgan dasturlash	115
9.1	Muammo: bir xil kod, har xil tur	115
9.2	Funksiya shablonlari	116
9.3	Sinf shablonlari	117
9.4	Standart shablon kutubxonasi (STL)	118
9.5	Concepts (C++20)	119
9.6	Shablonlar va turlash	119
9.7	Amaliy misol: Generic stack	120
9.8	Eng ko'p uchraydigan xatolar	121
9.9	Bob bo'yicha mashqlar	121
9.10	Xulosa	122
10	10 · Xatolik bilan ishlash	123
10.1	Xatolik — bu hayotning bir qismi	123
10.2	Usul 1: Qaytarish qiymati	123
10.3	Usul 2: Istisnolar (Exceptions)	124
10.4	Usul 3: <code>Natija<T> / std::expected</code> (C++23)	126
10.5	Usul 4: <code>Tanlov<T> / std::optional</code>	126
10.6	Usul 5: <code>tasdiqlash</code> (assert)	127
10.7	Xatolik strategiyasini tanlash	128
10.8	Amaliy misol: Konfiguratsiya faylini o'qish	128
10.9	Eng ko'p uchraydigan xatolar	129
10.10	Bob bo'yicha mashqlar	130
10.11	Xulosa	130
11	11 · Matn va satrlar bilan ishlash	131
11.1	Matn — eng ko'p ishlatiladigan tur	131
11.2	Asosiy matn amallari	131
11.3	Matnni boshqa turlarga aylantirish	133
11.4	Matnni solishtirish	134

11.5	Formatli matn (C++20/23 <code>std::format</code>)	134
11.6	Son va matn orasidagi konvertatsiya	135
11.7	UTF-8 va o'zbek alifbosi	136
11.8	<code>std::string_view</code> — matn ko'rinishi	136
11.9	Amaliy misol: Matn tahlilchisi	136
11.10	Eng ko'p uchraydigan xatolar	137
11.11	Bob bo'yicha mashqlar	138
11.12	Xulosa	138
12	12 · Fayl tizimi va JSON	141
12.1	Nima uchun fayllar?	141
12.2	Faylga yozish	141
12.3	Fayldan o'qish	142
12.4	CSV bilan ishlash	143
12.5	JSON bilan ishlash	144
12.6	Fayl yo'llari (C++17 <code>std::filesystem</code>)	145
12.7	Amaliy misol: Talabalar ma'lumotlar bazasi (JSON)	146
12.8	Eng ko'p uchraydigan xatolar	148
12.9	Bob bo'yicha mashqlar	149
12.10	Xulosa	150
13	13 · Ko'p oqimlilik — Parallel dasturlash	151
13.1	Nega ko'p oqimlilik kerak?	151
13.2	<code>std::thread</code> — birinchi oqim	152
13.3	Sinxronizatsiya — <code>std::mutex</code>	153
13.4	<code>std::atomic</code> — engil sinxronizatsiya	155
13.5	<code>std::async</code> — asinxron bajarish	155
13.6	<code>std::promise</code> va <code>std::future</code>	156
13.7	Oqimlar havzasi (Thread pool)	156
13.8	Sinxronizatsiya vositalari	157
13.9	Amaliy misol: Parallel massiv qayta ishlash	157
13.10	Eng ko'p uchraydigan xatolar	158
13.11	Bob bo'yicha mashqlar	159
13.12	Xulosa	160
14	14 · Standart kutubxona	161
14.1	uz++ standart kutubxonasi haqida	161
14.2	matematika moduli	161
14.3	vaqt moduli	162
14.4	sinov moduli — Test framework	163
14.5	jurnal moduli — Logging	163
14.6	kripto moduli — Kriptografiya	164
14.7	tarmoq moduli — Tarmoq aloqalari	164
14.8	suniy_intellekt moduli — AI integratsiyasi	165
14.9	C++ standart kutubxonasidan foydalanish	165
14.10	Amaliy misol: Vaqt hisoblagichi bilan testlash	167
14.11	Bob bo'yicha mashqlar	167
14.12	Xulosa	168
15	15 · GUI dasturlash (CopperSpice)	169
15.1	GUI nima va nima uchun kerak?	169
15.2	Birinchi GUI dastur	169
15.3	Maketlar (Layouts)	170

15.4	Signallar va slotlar	171
15.5	Muhim vidjetlar	172
15.6	Menyu va status qatori	172
15.7	To'liq misol: Mini matn muharriri	173
15.8	--bare rejimi	174
15.9	Bob bo'yicha mashqlar	174
15.10	Xulosa	175
16	16 · Ilovalar	177
16.1	A · Kalit so'zlar lug'ati	177
16.2	B · O'rnatish bo'yicha batafsil qo'llanma	179
16.3	C · Fayl kengaytmalari	180
16.4	D · CLI buyruqlari	180
16.5	E · C++23 mosligi	181
16.6	F · Foydali resurslar	181
16.7	G · Eng ko'p uchraydigan xatoliklar	182
16.8	H · Kitob bo'yicha xulosa	182

01 · Kirish — Dasturlash dunyosiga xush kelibsiz

1.1 Dasturlash nima?

Tasavvur qiling, siz oshpazsiz va do'stingizga osh tayyorlashni o'rgatmoqchisiz. Siz unga “osh qil” desangiz, hech narsa kelib chiqmaydi. Lekin agar siz aniq ko'rsatmalar bersangiz — “guruchni yuvib qo'y, sabzini chiziqcha qilib to'g'ra, go'shtni qovur, suv qo'shib qaynat, guruchni sol, 40 daqiqa damla” — do'stingiz ovqatni tayyorlay oladi.

Dasturlash — bu aynan shunday narsa, faqat kompyuter uchun. Kompyuter nimanidir o'zicha qilolmaydi. Unga aniq, ketma-ket, hech qanday talqin qilinmaydigan ko'rsatmalar berishingiz kerak. Bu ko'rsatmalar majmuasi — *dastur*, ularni yozish jarayoni esa — *dasturlash* deyiladi.

Bir muammo bor: kompyuter o'zbek tilini ham, ingliz tilini ham bevosita tushunmaydi. U faqat ikkita raqamni biladi — 0 va 1. Bizning vazifamiz — odam tushunadigan tilda yozilgan dasturni mashina tushunadigan ikkilik (*binary*) kodiga aylantirish. Buni **dasturlash tillari** va **kompilyatorlar** bajaradi.

***Hayotiy misol:** Bir teatrdagi rejissor aktyorlarga gapirayotgan paytida “siz bu yerda yig'lab yuborasiz” deydi. Aktyor buni o'z miyasida shunday talqin qiladi: yuz mushaklarini bo'shashtirish, ko'z yoshlarini hosil qilish, ovozni titratish. Rejissor “neyron impulslari” yuborolmaydi — u odam tilida so'zlaydi. Dasturlash tili xuddi shu vositachi: siz niyatingizni gapirasiz, til esa uni mashinaga tushunarli signallarga aylantiradi.*

1.1.1 Past va yuqori darajadagi tillar

Dasturlash tillarini ikki katta guruhga bo'lish mumkin:

- **Past darajadagi tillar** (Assembler, machine code) — mashina arxitekturasiga juda yaqin. Har bir buyruq aniq bir elektron amal (registr yuklash, qiymatlar qo'shish, xotira manziliga yozish). Odam uchun yozish ham, o'qish ham juda qiyin. Bitta dastur — minglab satrlar.
- **Yuqori darajadagi tillar** (C, C++, Python, Java, **uz++**) — abstraktsiya beradi. Bitta qator kod 10-100 ta mashina buyrug'iga aylanishi mumkin. Odam uchun yozish va o'qish oson, lekin orqada murakkab tarjima jarayoni yashiringan.

Yuqori darajadagi tillar yana ikki guruhga bo'linadi:

- **Kompilyatsiya qilinadigan tillar** (C, C++, Rust, Go) — kod *oldindan to'liq* mashina kodiga aylantiriladi, so'ng ishga tushiriladi. Natijada dastur juda tez.
- **Interpretatsiya qilinadigan tillar** (Python, JavaScript, Ruby) — kod *ishga tushirish vaqtida* qator-qator o'qiladi va bajariladi. Yozish oson, lekin ishlash sekinroq.

uz++ — kompilyatsiya qilinadigan tillar oilasidan. Lekin uning yo'li biroz boshqacha: u oldin *transpilyatsiya* qiladi. Bu nima ekanini keyinroq batafsil ko'ramiz.

1.2 uz++ — bu qanday til?

uz++ — bu o'zbek kalit so'zlari asosida ishlaydigan, **C++23 standarti** ustida qurilgan zamonaviy dasturlash tili. U C++ ning butun quvvatini saqlab qoladi, lekin sintaksisni o'zbek tiliga yaqinlashtiradi.

1.2.1 Nima uchun aynan uz++ ni o'rganish kerak?

Dunyoda yuzlab dasturlash tillari bor. Nega aynan uz++? Sababi to'rt asosiy:

1. Ona tilingizda

Tibbiyot, huquq, fizika kabi sohalarda kitoblar o'zbek tiliga tarjima qilinadi. Nima uchun dasturlash bunday qilinmasin? uz++ shu savolga amaliy javob. Barcha kalit so'zlar — *agar, funksiya, qaytarish, sinf, aks_holda* — o'zbek tilida. Sizning miyangiz endi parallel ravishda ikkita ishni qilishi shart emas: “if degani nima edi?” va “bu yerda mantiq nima?” — siz to'g'ridan-to'g'ri mantiqni o'ylashingiz mumkin.

2. Haqiqiy quvvat

uz++ — o'yinchoq emas. U C++23 ning barcha imkoniyatlarini taqdim etadi: - **Tezlik:** Python'dan 10-100 marta tezroq, JavaScript'dan 20 marta tezroq. - **To'g'ridan-to'g'ri xotira boshqaruvi:** har bir baytni siz nazorat qilasiz. - **Shablonlar (templates):** bitta funksiya istalgan tur bilan ishlay oladi. - **C++23 yangiliklari:** `std::expected` (xatoliklarni qaytarish), `std::format` (formatlash), `ranges` (zamonaviy iteratsiya), `coroutines` (asinxron amallar).

uz++ da yozilgan dastur *xuddi shu C++ kodi kabi* ishlaydi — chunki u oxir-oqibat C++ ga aylantirilib, so'ng kompilyatsiya qilinadi.

3. Sodda sintaksis

uz++ Python tilidan ilhomlangan: kod oson o'qiladi, blocklar chekinish (indentation) orqali ko'rinadi, nuqtali vergullar ko'p hollarda ixtiyoriy:


```
butun asosiy() {
    o'zgaruvchan ism = "Aziz"
    yozish << "Salom, " << ism << qator_oxiri
    qaytarish 0
}
```

Solishtiring, bir xil dastur C++ da:

```
#include <iostream>
#include <string>

int main() {
    std::string ism = "Aziz";
    std::cout << "Salom, " << ism << std::endl;
    return 0;
}
```

uz++ versiyasi qisqaroq, soddaroq va — eng muhimi — o'zbekcha.

4. Zamonaviy asboblari

uz++ bilan birga keladi: - **VS Code kengaytmasi** — kod yozayotganda real vaqtda xatolarni ko'rsatadi, avtoto'ldirish beradi, har bir o'zgaruvchining turini hover'da ko'rsatadi. - **LSP server** — barcha zamonaviy IDE'lar bilan ishlaydi. - **DAP server** — disk to'xtatish (debugging) breakpoint'lar bilan. - **Formatter** — kodingizni avtomatik tartibga keltiradi.

***Yangi boshlovchilar uchun:** Agar siz birinchi marta dasturlash bilan tanishayotgan bo'lsangiz, uz++ siz uchun mukammal til. Sintaksis soddadir, lekin orqada haqiqiy til turadi. Bu yerda o'rgangan ko'nikmalar to'g'ridan-to'g'ri C++ ga o'tadi — kelajakda agar "katta dunyo"ga chiqmoqchi bo'lsangiz, hech qanday qayta o'rganish kerak emas.*

1.3 uz++ ichida nima sodir bo'ladi?

Bu bo'lim ehtimol kitobning eng muhim bo'limi. Agar siz dasturlash qanday ishlashini tushungangiz, hamma narsa keyin ravshan bo'ladi.

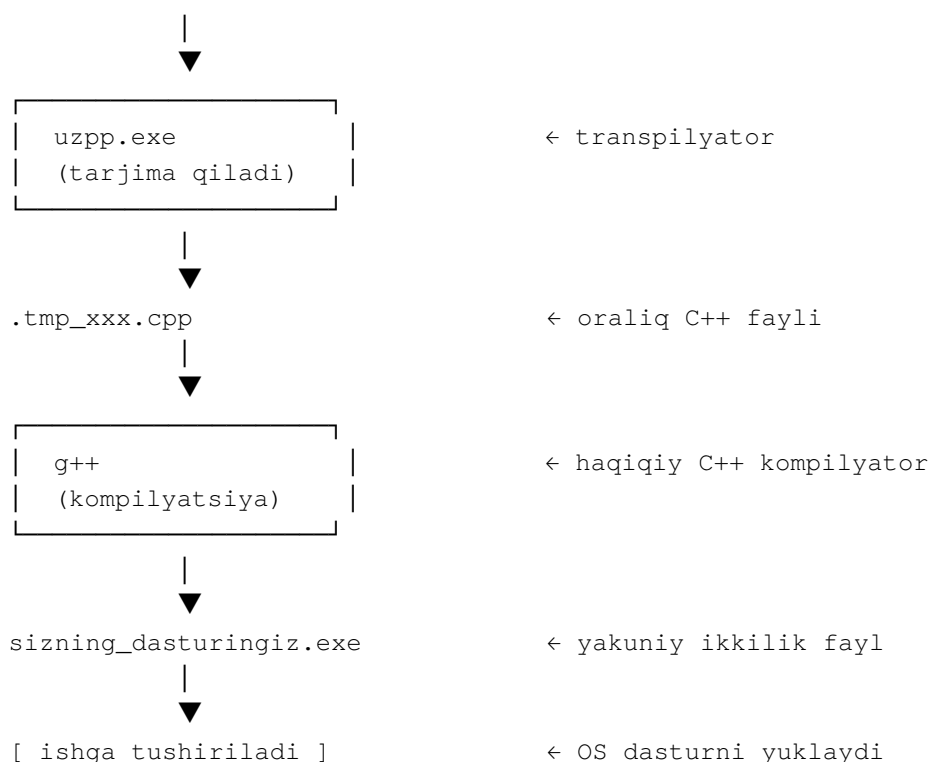
1.3.1 Transpilyator nima?

Oddiy kompilyator (masalan, g++) sizning kodingizni to'g'ridan-to'g'ri mashina kodiga aylantiradi. Lekin uz++ avval bir narsa qiladi: u sizning .uzpp faylingizni C++23 kodiga aylantiradi. Keyin C++ kompilyatori (g++) bu kodni mashina kodiga aylantiradi.

Quyidagi diagrammaga e'tibor bering — bu sizning dasturingizning yo'li:

sizning_dasturingiz.uzpp

← siz yozgan fayl



Har bir bosqichni batafsil koʻrib chiqamiz.

1.3.2 1-bosqich: Leksik tahlil (Lexer)

Transpilyator avval sizning kodingizni oʻqiydi va uni alohida “soʻzlar”ga ajratadi. Bu soʻzlar **token** deb ataladi.

Misol uchun, agar siz quyidagi kodni yozsangiz:

```
o'zgaruvchan x = 10
```

Lexer uni quyidagi tokenlarga ajratadi:

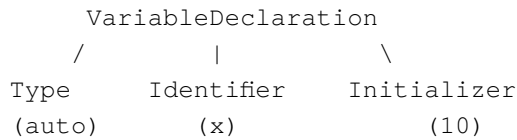
Token	Turi
o'zgaruvchan	kalit soʻz (KEYWORD)
x	identifikator (IDENT)
=	operator (ASSIGN)
10	son literal (NUMBER)

Bu — xuddi siz gapni alohida soʻzlarga ajratganingizdek.

1.3.3 2-bosqich: Sintaksik tahlil (Parser)

Endi tokenlardan *maʼno* olish kerak. Parser tokenlarni oʻqiydi va ulardan **AST** (Abstract Syntax Tree — abstrakt sintaksis daraxti) quradi.

Yuqoridagi misol uchun AST shunday koʻrinadi:



Bu daraxt aytadi: “bu yerda o’zgaruvchi e’lon qilinmoqda, nomi *x*, boshlang’ich qiymat 10”.

1.3.4 3-bosqich: Turini tekshirish (Type Checker)

Parser AST’ni qurganidan keyin, **type checker** uni tekshiradi. U quyidagi savollarga javob beradi:

- O’zgaruvchining turi to’g’ri ishlatilganmi?
- Funksiya chaqirilganda to’g’ri turdagi argumentlar berilganmi?
- Sinflar va metodlarni to’g’ri qo’llanilganmi?

Misol uchun:

```
butun yosh = "yigirma" // ← xato! butun (int) ga matn berilmoqda
```

Type checker buni ushlaydi va sizga xato xabarini chiqaradi — *dastur ishga tushirilmasidan oldin*. Bu qadar foydali xususiyat: ko’p xatolarni siz tezda ushlay olasiz.

1.3.5 4-bosqich: Kod generatsiyasi (CodeGen)

Endi tekshirilgan AST C++23 kodiga aylantiriladi. Bu erda “sehr” sodir bo’ladi:

uz++:

```
o'zgaruvchan x = 10
yozish << x << qator_oxiri
```

Generated C++ (sizga ko’rinmaydi, lekin shunday yaratiladi):

```
auto x = 10;
std::cout << x << std::endl;
```

E’tibor bering — har bir uz++ tushunchasi tegishli C++ tushunchasiga aylangan: *o'zgaruvchan* → *auto*, *yozish* → *std::cout*, *qator_oxiri* → *std::endl*.

1.3.6 5-bosqich: Kompilyatsiya (g++)

Hosil bo’lgan C++ kodi g++ kompilyatoriga uzatiladi. g++ uni mashina kodiga aylantiradi va sizga *.exe* (Linux’da kengaytmasiz) fayl beradi.

1.3.7 6-bosqich: Bajarish (OS)

Siz *.exe* faylni ishga tushirsangiz, operatsion tizim uni xotiraga yuklaydi va protsessor unga buyruqlarni yetkazib bera boshlaydi. Dastur ishlamoqda!

Ishonsangiz: Bu 6 bosqichli jarayon millisoniyada o'tadi. Siz "uzpp ishga-tushirish dastur.uzpp" deganingizdan keyin 1-2 soniya ichida natija ko'rinadi. Sahnaning orqasida bo'layotgan barcha ishlar — sizning ko'zingiz uchun ko'rinmas.

1.4 O'rnatish

Dasturlashga boshlash uchun ikkita narsa kerak: 1. **uz++ transpilyatori** (uzpp.exe) 2. **C++ kompilyatori** (g++, MinGW yoki Clang)

Yaxshi xabar: Windows'da har ikkalasini bir o'rnatuvchi orqali olishingiz mumkin.

1.4.1 Windows

1. **uz++ o'rnatuvchisini yuklab oling:** [GitHub Releases](#) sahifasidan eng so'nggi `uzpp-setup.exe` faylini olib oling.
2. **O'rnatuvchini ishga tushiring.** U avtomatik ravishda quyidagilarni o'rnatadi:
 - `uzpp.exe` — transpilyatorning o'zi
 - **MinGW-w64** — Windows uchun g++ kompilyatori
 - **VS Code kengaytmasi** (agar VS Code o'rnatilgan bo'lsa)
3. **Tekshiring.** Terminalni qayta oching va quyidagi buyruqni kiriting:

```
uzpp --yordam
```

Agar yordam matni chiqsa — hammasi joyida! Agar "uzpp: command not found" deb yozilsa, terminalni yopib qayta oching (yo'l o'zgaradi).

1.4.2 Linux / macOS

Terminalda quyidagi buyruqni ishga tushiring:

```
curl -fsSL https://raw.githubusercontent.com/timetolivechk-spec/uzpp/main/install.sh  
↵ | bash
```

Bu skript g++ ning o'rnatilganligini tekshiradi (agar yo'q bo'lsa, sistema paket menejeri orqali o'rnatadi) va uz++ ni yuklab oladi.

1.4.3 VS Code kengaytmasi

Agar siz VS Code'dan foydalansangiz (tavsiya qilamiz), Marketplace'dan **uzpp.uzpp** kengaytmasini o'rnatish. Kengaytma sizga quyidagilarni beradi:

- **Sintaksis yoritilishi:** kalit so'zlar rangli ko'rinadi

- **Avtoto’ldirish (IntelliSense):** sizning kodingizni tushunadi
- **Real vaqtli xatoliklar:** yozayotganingizda xatolarni darhol ko’rsatadi
- **F5 orqali ishga tushirish:** kodni bir tugmacha bilan yuritish

***Birinchi marta o’rnatish bilan muammo bormi?** Eng ko’p uchraydigan muammolar: - **PATH’ga qo’shilmadi:** terminalni yopib qayta oching, yoki kompyuterni qayta yuklang. - **MinGW topilmadi:** o’rnatuvchini administrator nomidan ishga tushiring. - **VS Code kengaytmasi ko’rinmaydi:** VS Code’ni qayta ishga tushiring va Extensions panelida “uzpp” deb qidiring.*

1.5 Birinchi dastur: “Salom, dunyo!”

Har qanday dasturlash tilini o’rganish an’anaviy ravishda **“Salom, dunyo!”** dasturi bilan boshlanadi. Bu — 1972 yilda Brian Kernighan tomonidan C tilining birinchi darsligida ishlatilgan dastur. O’sha paytdan beri har bir dasturchi shu dastur bilan boshlaydi — bu kasbiy an’ana.

1.5.1 Kod

Quyidagi kodni `salom.uzpp` nomli faylga saqlang:

```
// salom.uzpp - birinchi uz++ dasturi

butun asosiy() {
    yozish << "Salom, dunyo!" << qator_oxiri
    qaytarish 0
}
```

1.5.2 Ishga tushirish

Terminalni faylingiz joylashgan papkada oching va kiriting:

```
uzpp ishga-tushirish salom.uzpp
```

Yoki agar VS Code’da bo’lsangiz, oddiy F5 tugmasini bosing.

Natija:

Salom, dunyo!

Tabriklayman! Siz hozirgina birinchi uz++ dasturingizni yozdingiz va ishga tushirdingiz.

1.5.3 Endi nimani ko’rdik — qator-qator tahlil

Hozir dasturimizning har bir qatorini batafsil ko’rib chiqamiz. Agar ba’zi tushunchalar sizga noma’lum bo’lsa — xavotir olmang, biz ularning har biriga keyingi boblar bag’ishlaymiz.

1-qator: Izoh

```
// salom.uzpp - birinchi uz++ dasturi
```

// belgisi izoh (kommentariy) boshlanadi. Izohlar dastur uchun ahamiyatsiz — kompyuter ularni o'qimaydi. Lekin **odamlar** uchun muhim: izohlar sizga keyingi safar kelganingizda kodingiz nima qilayotganini eslatadi.

Yaxshi tavsiya: Izohlar — bu o'tmishdagi siz kelajakdagi sizga qoldirgan xat. Yarim yildan keyin o'z kodingizga qaytsangiz, "Bu nima ekan?" demaslik uchun, izohlar yozing.

2-qator: Funksiya e'loni

```
butun asosiy() {
```

Bu qator juda muhim. Uni qismlarga ajrataylik:

Qism	Ma'nosi
butun	Funksiya qaytaradigan qiymat turi — <i>butun son</i>
asosiy	Funksiya nomi — <code>asosiy</code> (bu maxsus nom!)
()	Funksiya parametrlari — bu yerda parametr yo'q (bo'sh qavslar)
{	Funksiya tanasining boshlanishi

asosiy — bu maxsus nom. Har bir uz++ dasturida bitta `asosiy` funksiya bo'lishi shart. Bu — *dasturning kirish nuqtasi*. Operatsion tizim sizning dasturingizni ishga tushirganida, u *aynan shu* funktsiyani chaqiradi.

Real hayotdagi misol: Tasavvur qiling, sizning uyingizda ko'p xona bor — yotoqxona, oshxona, hammom. Lekin sizga mehmon kelganda, u har doim **eshik orqali kiradi**, to'g'ridan-to'g'ri yotoqxona-ga emas. *asosiy funksiyasi* — bu sizning dasturingizning eshigi.

3-qator: Konsolga yozish

```
yozish << "Salom, dunyo!" << qator_oxiri
```

E'tibor bering — qator boshida 4 ta bo'sh joy bor. Bu **chekinish** (*indentation*). uz++ da chekinish bloklarni ko'rsatadi: bu qator `asosiy` funksiyaning *ichida* ekanligini aytadi.

Endi qatorning o'zini tahlil qilaylik:

Qism	Ma'nosi
yozish	Konsolga (terminalga) yozish vositasi
<<	"Bu ma'lumotni yozishga yubor" operatori
"Salom, dunyo!"	Matn satri (<i>string</i>)
<<	Yana yuborish operatori
qator_oxiri	Yangi qator belgisi (<i>newline</i>)

Bu qator shuni aytadi: "konsolga Salom, dunyo! matnini chiqaring, keyin yangi qatorga o'ting".
 << operatorini "kanal" deb tasavvur qiling: ma'lumotlar oqim bo'lib bir tomonga oqadi.

```
"Salom, dunyo!"  ┌
                  └─→ [yozish (konsol)]
qator_oxiri      ┌
```

4-qator: Qiymat qaytarish

```
qaytarish 0
```

qaytarish 0 — funksiyadan 0 raqamini qaytarish degani.

Nima uchun 0? Chunki **asosiy** funksiyaning qaytarish qiymati — bu dasturning *tugash holati*. Konventsia bo'yicha: - 0 — "hammasi yaxshi tugadi" degan ma'noda - 0 dan boshqa raqam — "xatolik yuz berdi" degan ma'noda

Operations tizim bu qiymatni qabul qiladi va u orqali sizning dasturingizning natijasi haqida xulosa qiladi.

Real hayotdagi misol: Siz pochta xizmatiga buyurtma berdingiz. Buyurtma yetkazilganidan keyin sizdan ovoz beriladi: "qaytib keldim, yetkazib berdim" (0), yoki "muvaffaqiyatsizlik bo'ldi, qaytib kelmoqdaman" (1, 2, va h.k.). qaytarish 0 shu birinchi holat.

5-qator: Funksiya tanasining yopilishi

```
}
```

{ ochilgan edi, } uni yopadi. `asosiy` funksiya tugadi.

1.5.4 Endi, butun rasm

Qachon biz `uzpp` ishga-tushirish `salom.uzpp` desak:

1. **Transpilyator** sizning kodingizni o'qiydi va C++ ga aylantiradi:

```
#include "uzpp_runtime.hpp"
int main() {
    std::cout << "Salom, dunyo!" << std::endl;
    return 0;
}
```

2. **g++** bu C++ kodini mashina kodiga aylantiradi va `salom.exe` yaratadi.
3. **Operatsion tizim** `salom.exe` ni ishga tushiradi.
4. Dastur `main` (yoki `uz++` da asosiy) funksiyasidan boshlanadi.
5. `std::cout` orqali "Salom, dunyo!" konsolga chiqariladi.
6. `return 0` orqali dastur tugaydi va OS ga "men muvaffaqiyatli tugadim" deydi.

Hech qachon bu kabi oddiy 4 qator kod orqasida bunchalik ko'p ish borligini o'ylab ko'rganmidingiz?

1.6 Izohlar — kelajakdagi o'zingizga xat

Izohlar — dastur uchun ko'rinmas, lekin *odamlar* uchun ko'rinadigan matn. `uz++` da izohlarning ikki turi bor:

1.6.1 Bir qatorli izoh

```
// bu - bir qatorli izoh
butun yosh = 25 // o'zgaruvchidan keyin ham yozish mumkin
```

// belgisidan keyin satr oxirigacha hamma narsa — izoh.

1.6.2 Ko'p qatorli izoh

```
/*
    Bu - ko'p qatorli izoh.
    U bir necha qatorni egallashi mumkin.
    Foydali bo'limni tushuntirish uchun kerak bo'lganda.
*/
```

`/*` bilan boshlanib `*/` bilan tugaydi.

1.6.3 Yaxshi izoh va yomon izoh

Yomon izoh — kodda yozilgan narsani takrorlaydigan izoh:


```
butun yosh = 25 // yoshni 25 ga teng qil ← bu izoh keraksiz!
```

Yaxshi izoh — kod *nima qilayotgani* emas, **nega** shunday qilayotganini tushuntiradi:

```
// Yoshi 18 dan kichik foydalanuvchilarga ovoz berish ruxsat etilmaydi.
agar (yosh < 18) {
    yozish << "Kechirasiz, ovoz bera olmaysiz" << qator_oxiri
}
```

Mashhur dasturchi Brian Kernighan'ning maslahati: “Eng yaxshi izoh — bu kerakmas izoh. Agar kodingiz tushunarli bo'lsa, izoh kerak emas. Agar tushunarli emas bo'lsa — kodni qayta yozing, izoh qo'shmang.”

1.7 Qatorlar va ko'rsatmalar

uz++ ning bir o'ziga xosligi — **nuqtali vergullar ixtiyoriy**. C++ da har bir ko'rsatma ; bilan tugashi shart, lekin uz++ siz ularsiz yozishingiz mumkin:

```
butun a = 5
butun b = 10
butun c = a + b
yozish << c << qator_oxiri
```

Lekin agar xohlasangiz, vergullarni qo'yishingiz mumkin — xato bo'lmaydi:

```
butun a = 5;
butun b = 10;
butun c = a + b;
yozish << c << qator_oxiri;
```

Tavsiya: Boshlanishida vergullarsiz yozing — kod tozaroq ko'rinadi.

1.7.1 Bir qator ichida bir nechta ko'rsatma

Agar siz bir qatorda bir nechta ko'rsatma yozmoqchi bo'lsangiz, ularni ; bilan ajrating:

```
butun x = 5; butun y = 10; butun z = x + y
```

Lekin bu uslubdan qochish kerak — kod o'qish qiyin bo'ladi.

1.7.2 Bloklar — chekinish orqali

uz++ da kod bloklari **chekinish** (indentation) orqali ifodalanadi:

```
butun asosiy() {
    agar (rost) {
        yozish << "ichkari blok" << qator_oxiri
    }
    yozish << "tashqari blok" << qator_oxiri
}
```

E'tibor bering: ichkari yozish 8 ta bo'sh joy bilan chekinmoqda (agar ichida + asosiy ichida). Tashqarisi — 4 ta bo'sh joy.

Tavsiya: har bir daraja uchun **4 ta bo'sh joy** ishlating. Tab tugmasidan foydalanmang — turli muharrirlarda u turli kenglikda ko'rinishi mumkin.

1.8 Birinchi interaktiv dastur

Endi biz dasturni biroz "tirik" qilaylik — foydalanuvchidan ma'lumot so'raylik va unga javob beraylik.

1.8.1 Kod

```
// muloqot.uzpp - foydalanuvchi bilan dastlabki muloqot

butun asosiy() {
    matn ism
    butun yosh

    yozish << "Ismingiz nima? "
    kiritish >> ism

    yozish << "Yoshingiz necha? "
    kiritish >> yosh

    yozish << qator_oxiri
    yozish << "Salom, " << ism << "!" << qator_oxiri
    yozish << "Siz " << yosh << " yoshdasiz." << qator_oxiri

    agar (yosh < 18) {
        yozish << "Siz hali yoshsiz - kelajak sizniki!" << qator_oxiri
    }
    aks_holda {
        yozish << "Tajribangizdan foydalanish vaqti keldi." << qator_oxiri
    }

    qaytarish 0
}
```

1.8.2 Ishga tushirish

```
uzpp ishga-tushirish muloqot.uzpp
```

Natija (misol):

```
Ismingiz nima? Aziza
Yoshingiz necha? 22
```

```
Salom, Aziza!
Siz 22 yoshdasiz.
Tajribangizdan foydalanish vaqti keldi.
```

1.8.3 Yangi tushunchalar

Bu dasturda biz bir necha yangi narsalarni ko'rdik:

1. O'zgaruvchi e'loni

```
matn ism
butun yosh
```

O'zgaruvchi — bu xotirada saqlash uchun nomlangan joy. Bu yerda: - `ism` — `matn` turidagi (`string`) o'zgaruvchi - `yosh` — `butun` turidagi (`int`) o'zgaruvchi

Tasavvur qiling, sizda ikkita quti bor: birida “`ism`” yorlig'i yopishtirilgan (faqat `matn` saqlash uchun), ikkinchisida “`yosh`” yorlig'i (faqat `butun` sonlar uchun).

2. Kiritish (Input)

```
kiritish >> ism
```

`kiritish` — bu klaviaturadan ma'lumot olish vositasi. `>>` operatori “klaviaturadan o'qib `ism` o'zgaruvchisiga sol” degan ma'nodi.

E'tibor bering: chiqarish uchun `<<`, kiritish uchun `>>`. Strelkalarning yo'nalishi yordam beradi: - `<<` — “bu yerga oqsin” (chiqarish) - `>>` — “u yerdan keladi” (kiritish)

3. Shart operator

```
agar (yosh < 18) {
    yozish << "Siz hali yoshsiz - kelajak sizniki!" << qator_oxiri
```

```

}
aks_holda {
    yozish << "Tajribangizdan foydalanish vaqti keldi." << qator_oxiri
}

```

agar (if) shart bajarilsa, birinchi blok ishlaydi. aks_holda (else) — aks holda, ikkinchi blok ishlaydi.

< belgisi — “kichik” (less than) ma’nosida. Boshqa taqqoslash operatorlari:

Operator	Ma’nosi
<	kichik
>	katta
<=	kichik yoki teng
>=	katta yoki teng
==	teng (e’tibor bering, ikkita belgi!)
!=	teng emas

Eng ko’p uchraydigan xatolik: = va == ni adashtirib qo’yish. = — qiymat berish ($a = 5$ — a ga 5 ni saqla). == — taqqoslash ($a == 5$ — a 5 ga tengmi?). Bu xato birinchi haftalarda ko’p sodir bo’ladi.

1.9 Tilning asosiy elementlari

Kelinglar, hozir `uz++` ning eng asosiy 3 ta narsasini umumiy ko’rib chiqaylik. Keyingi boblarda har birini batafsil o’rganamiz.

1.9.1 Kalit so’zlar (Keywords)

Quyida eng muhim kalit so’zlar — siz ularni keyingi boblarda barchasini o’rganasiz:

uz++	C++ ekvivalenti	Ma’nosi
agar	if	Agar shart bajarilsa
aks_holda	else	Aks holda
uchun	for	Takrorlash sikli
davr	while	Shartli takrorlash
tanlash	switch	Tanlash operatori
sinf	class	Klass e’lon qilish
tuzilma	struct	Tuzilma e’lon qilish
qaytarish	return	Qiymat qaytarish
rost	true	Mantiqiy rost qiymat
yolg’on	false	Mantiqiy yolg’on qiymat
to’xtatish	break	Sikldan chiqish
davom_ettirish	continue	Keyingi takrorlashga o’tish

Eslatma: Kalit so'zlar — bu **bron qilingan** so'zlar. Siz ularni o'zgaruvchi nomi sifatida ishlatolmaysiz. Masalan, `butun agar = 5` deb yozolmasiz — chunki `agar` allaqachon kalit so'z.

1.9.2 Ma'lumot turlari

Kompyuter har xil turdagi ma'lumotlarni boshqacha saqlaydi. Asosiy turlar:

Tur	C++ ekvivalenti	Ma'nosi	Misol
butun	<code>int</code>	Butun son (32 bit)	5, -100, 42
kasr	<code>float</code>	Haqiqiy son (32 bit)	3.14, 0.5
ikkilangan	<code>double</code>	Aniq haqiqiy son (64 bit)	3.14159265
mantiq	<code>bool</code>	Rost/yolg'on	<code>rost</code> , <code>yolg'on</code>
belgi	<code>char</code>	Bitta belgi	'A', 'b', '!'
matn	<code>std::string</code>	Matn satri	"Salom", "Aziza"
bosh	<code>void</code>	Bo'sh (qiymatsiz)	—

Nima uchun turlar kerak? Kompyuter *butun* va *kasr*ni boshqa joyda saqlaydi. 42 — bu 4 baytlik xotirada saqlanadi (`int`). 3.14 — bu 4 baytlik, lekin boshqacha tashkil etilgan xotirada (`float`). Agar siz turini noto'g'ri ko'rsatsangiz, kompyuter ma'lumotni noto'g'ri talqin qiladi va dasturingiz xato beradi.

1.9.3 Operatorlar

Operatorlar — bu qiymatlar ustida amallar bajaradigan belgilar:

```
// Arifmetik amallar
butun yigindi = 5 + 3      // 8
butun farq = 10 - 4       // 6
butun kopaytma = 6 * 7    // 42
butun bolinma = 15 / 3    // 5
butun qoldiq = 17 % 5     // 2 (17 / 5 dan qolgani)

// Taqqoslash - natija mantiq turida (rost/yolg'on)
mantiq teng = (5 == 5)    // rost
mantiq kichik = (3 < 7)   // rost
mantiq farqli = (5 != 5)  // yolg'on

// Mantiqiy amallar
mantiq va = rost && yolg'on // yolg'on (ikkalasi ham rost bo'lishi kerak)
mantiq yo = rost || yolg'on // rost (kamida bittasi rost)
mantiq inkor = !rost       // yolg'on
```

Operatorlarni keyingi bobda batafsil ko'ramiz.

1.10 Eng ko'p uchraydigan birinchi xatolar

Yangi boshlovchilar tez-tez bir xil xatolarga duch keladi. Mana ulardan eng ko'p tarqalganlari:

1.10.1 1. Qo'shtirnoqlarni unutish

```
yozish << Salom, dunyo! << qator_oxiri // ← XATO
yozish << "Salom, dunyo!" << qator_oxiri // ← TO'G'RI
```

Matn satrlari har doim qo'shtirnoq ("...") ichida bo'lishi kerak.

1.10.2 2. Operatorni noto'g'ri yo'naltirish

```
yozish >> "Salom" // ← XATO (yozish chiqarish uchun, >> kiritish uchun)
yozish << "Salom" // ← TO'G'RI
```

1.10.3 3. Funksiya tanasini yopishni unutish

```
butun asosiy() {
    yozish << "Salom" << qator_oxiri
    qaytarish 0
    // ← XATO: } yo'q!
```

Har bir { uchun mos } bo'lishi kerak.

1.10.4 4. Tip mosligini buzish

```
butun yosh = "yigirma" // ← XATO: matn `butun` ga sig'maydi
butun yosh = 20 // ← TO'G'RI
```

1.10.5 5. = va == ni adashtirish

```
agar (yosh = 18) { ... } // ← XATO: `yosh` ga 18 ni saqlamoqda!
agar (yosh == 18) { ... } // ← TO'G'RI: 18 ga tengmi?
```

Bu juda nozik xato. Birinchi qator dasturingizda buzilmaydi, lekin u har doim "rost" deb hisoblanadi. Buni topish qiyin bo'ladi!

Yaxshi xabar: `uz++` ning `type checker`'i bu xatolarning ko'pini kod yozayotgan paytda darhol ushlaydi. VS Code'da xato joylari qizil chiziq bilan ostiga chiziladi. Foydalaning!

1.11 Asboblar — dasturchi sherigi

Yaxshi dasturchi — bu nafaqat tilni biluvchi odam, balki o'z asboblarini yaxshi bilgan odam. Quyidagi 4 ta narsa dasturchining asosiy “asboblarini” tashkil qiladi:

1.11.1 1. Matn muharriri (Editor)

Kod yozish uchun. Tavsiya qilamiz: **VS Code** — bepul, kuchli, va `uz++` kengaytmasi mavjud.

1.11.2 2. Terminal

Buyruqlarni ishga tushirish uchun. Windows'da: PowerShell yoki Windows Terminal. Linux/macOS'da: tabiiy terminal.

1.11.3 3. Kompilyator / Transpilyator

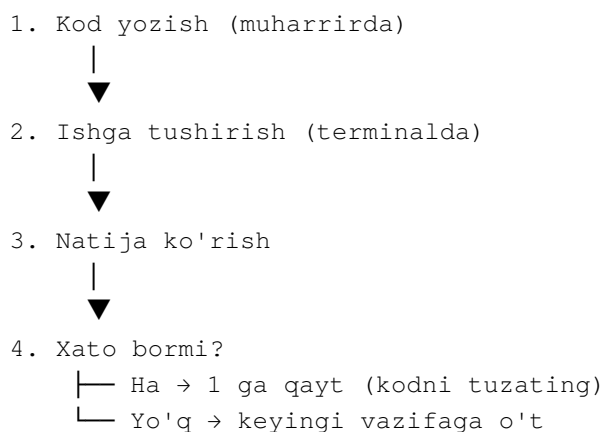
Sizning kodingizni mashina kodiga aylantiradi. Bizning holatda — `uzpp` va u o'z navbatida `g++` ni chaqiradi.

1.11.4 4. Debugger

Dasturingiz qanday ishlayotganini ko'rish uchun. `uz++` DAP server orqali breakpoint'larni qo'llab-quvvatlaydi.

1.11.5 Rivojlanish sikli

Dasturchining odatdagi ish kuni shunday ko'rinadi:



Bu **REPL** sikli (Read-Eval-Print Loop) — har bir dasturchining kundalik mehnati. U qancha tezroq aylanib o'tsa, siz shuncha samaraliroq ishlaysiz.

1.12 Bob bo'yicha mashqlar

Quyidagi mashqlarni o'zingiz hal qiling. Har birining yechimi sizning keyingi kuningizgacha bo'lgan asosiy ko'nikmangizdir.

1.12.1 Mashq 1: Salomlashish (oson)

“Salom, dunyo!” dasturini yozing va ishga tushiring. Keyin matnni o'zgartiring — masalan, “Salom, do'stlarim!” deb yozing.

Maslahat: Hech qanday maslahat kerak emas — bu darsda ko'rgan dastur.

1.12.2 Mashq 2: Ism va familiya (oson)

Foydalanuvchidan **ikkita** narsani so'rang — ism va familiya. So'ng “Salom, [ism] [familiya]!” deb javob bering.

Maslahat: Sizga ikkita `matn` o'zgaruvchi kerak bo'ladi va ikkita `kiritish` qatori.

1.12.3 Mashq 3: Sodda kalkulyator (o'rtacha)

Foydalanuvchidan ikkita butun son so'rang. Keyin ularning yig'indisi, ayirmasi, ko'paytmasi va bo'linmasini chiqaring.

Maslahat:

```
butun a, b
kiritish >> a >> b      // ikkita sonni ketma-ket o'qish mumkin
```

Hushyor bo'ling! Agar `b` 0 bo'lsa, `a / b` bo'lganda dastur xato beradi. Buni keyingi boblarda ko'ramiz.

1.12.4 Mashq 4: Tug'ilgan yili (o'rtacha)

Foydalanuvchidan tug'ilgan yilini so'rang. Joriy yil deb 2026 ni oling. Foydalanuvchining yoshini hisoblang va chiqaring.

Maslahat: `Yosh = joriy_yil - tug'ilgan_yil`.

1.12.5 Mashq 5: O'rtacha baho (o'rtacha)

Foydalanuvchidan 3 ta fan bahosini so'rang (har biri 0 dan 100 gacha). O'rtacha bahoni hisoblab chiqaring.

Maslahat: $O'rtacha = (a + b + c) / 3$. Lekin natija ikkilangan turida bo'lishi kerak — agar siz butun ishlatsangiz, kasr qismi yo'qoladi! o'zgaruvchan $ortacha = (a + b + c) / 3.0$ deb yozsangiz, 3.0 dueto avtomatik kasrli hisoblanadi.

1.12.6 Mashq 6: Quyon va toshbaqa (qiyinroq)

Quyon soatiga 50 km, toshbaqa soatiga 5 km yuradi. Foydalanuvchidan ikki hayvon orasidagi masofani (km) so'rang. Toshbaqa boshlang'ich masofani oladi (favorit). Necha soatdan keyin quyon toshbaqani quvib yetadi?

Maslahat: Tenglama: $50 * t = 5 * t + masofa$. Yechimi: $t = masofa / 45$.

1.12.7 Mashq 7: O'z dasturingiz (cheksiz)

Endi sizning o'z dasturingiz — biror narsa o'ylab toping va yozing. Bu turing hamfikrlarga ko'rsating!

1.13 Xulosa — biz bu bobda nimani o'rgandik

Bu bobda biz quyidagi narsalarni ko'rdik:

- **Dasturlash nima** — kompyuterga aniq ko'rsatmalar yozish jarayoni.
- **Tillar ierarxiyasi** — past darajadagi va yuqori darajadagi tillar, kompilyatsiya va interpretatsiya farqi.
- **uz++ ning falsafasi** — ona tilida dasturlash, lekin C++23 ning to'liq quvvati bilan.
- **Transpilyatsiya jarayoni** — kodingizdan binary fayligacha bo'lgan 6 bosqichli yo'l.
- **Birinchi dastur** — “Salom, dunyo!” ni yozdik va har bir qatorini tushuntirdik.
- **Asosiy sintaksis** — izohlar, chekinish, blokklar, ko'rsatmalar.
- **Birinchi interaktiv dastur** — foydalanuvchi bilan muloqot qilishni o'rgandik.
- **Asosiy tushunchalar** — kalit so'zlar, ma'lumot turlari, operatorlar.
- **Eng ko'p uchraydigan xatolar** — qo'shtirnoqlar, operatorlar, tiplar mosligi.
- **Asboblari** — muharrir, terminal, kompilyator, debugger.

Keyingi bobda biz o'zgaruvchilar va ma'lumot turlari haqida batafsil gaplashamiz. Sizning xotirangizda nima va qanday saqlanishini, har bir tur qancha joy egallashini, va kompyuter ularni qanday ishlatishini o'rganasiz.

Eslatma — dasturlash yo'lida birinchi qadam: Eng muhimi — yozib boring. Kitobni o'qish kifoya emas. Har bir mashqni o'zingiz qo'lingiz bilan terib chiqing, ishga tushiring, xatolarni tuzatib. Dasturlash — bu fizika emas, falsafa emas. Bu — amaliy mahorat. Faqat amaliyot orqali siz dasturchi bo'lasiz.

02 · O'zgaruvchilar va ma'lumot turlari

2.1 O'zgaruvchi nima?

Tasavvur qiling, siz omborxonada ishlaysiz. Omborxona — bu katta zal, unda minglab javon va qutilar bor. Har bir qutida bir narsa saqlanadi: biriga “Olma”, boshqasiga “Sabzi”, uchinchisiga “Un” deb yorliq yopishtirilgan. Siz “Olma” kerak bo'lsa, “olma” yozilgan qutiga borasiz, undan olasiz. “Olma”ni qaytarish kerak bo'lsa — yana o'sha qutiga qo'yasiz.

Kompyuter xotirasi — xuddi shu omborxona. **O'zgaruvchi** — bu xotirada nomlangan quti. Siz unga qiymat saqlaysiz, keyin nom orqali murojaat qilasiz va qiymatni o'qiysiz yoki o'zgartirasiz.

```
butun yosh = 25           // "yosh" deb nomlangan qutiga 25 ni saqla
yozish << yosh           // "yosh" qutisidan qiymatni o'qib chiqar
yosh = 26                // qiymatni o'zgartir
```

E'tibor bering: `yosh` — bu nom (etiketka). `25` — qiymat (mazmun). Quti o'zi — xotira manzili. Bu uchta narsa bir-biriga bog'liq, lekin alohida tushunchalar.

Nima uchun “o'zgaruvchi” deyiladi? Chunki uning qiymati o'zgarishi mumkin. Yuqoridagi misolda biz `yosh` ni 25 dan 26 ga o'zgartirdik. Bunga taqqoslash uchun, doimiy (o'zgarmas) qiymat ham bor — uni bir marta belgilab, keyin o'zgartirib bo'lmaydi. Buni keyinroq ko'ramiz.

2.1.1 Xotiradagi haqiqat

Yuqoridagi kod ishga tushganida, xotirada nima sodir bo'ladi?

Tasavvur qiling, xotira — bu juda uzun lenta, har bir katakda bitta **bayt** (8 bit, ya'ni 0-255 oraliqdagi son) saqlanadi:

Manzil:	...	0x7FF8	0x7FF9	0x7FFA	0x7FFB	...
Mazmun:	...	25	0	0	0	...

^

└── “yosh” nomi shu yerga ko'rsatadi

`butun` (32-bitlik son) 4 ta baytni egallaydi. `25` raqami xotirada quyidagicha saqlanadi: `25` (qiymatning eng past bayti), keyin `0`, `0`, `0`. Kompyuter “yosh” nomini hisobga olib, 4 baytni o'qib

bittadek ko'radi: 25.

Bu darajada chuqur tushunish hozir shart emas — lekin **muhim narsa shu**: o'zgaruvchi xotirada aniq bir joyni egallaydi va aniq miqdordagi baytni iste'mol qiladi.

2.2 Asosiy ma'lumot turlari

uz++ da ma'lumotning ko'p turlari bor. Avval eng asosiylarini tanitamiz, keyin har biriga batafsil to'xtalamiz.

2.2.1 Tezkor jadval

Tur	C++ ekvivalenti	Hajm	Diapazon	Misol
butun	int	4 bayt	-2,147,483,648 dan 2,147,483,647 gacha	25, -7, 1000000
uzun	long long	8 bayt	-9×10^{18} dan 9×10^{18} gacha	9999999999
kasr	float	4 bayt	$\pm 3.4 \times 10^{38}$ (taxminan 7 ta aniq raqam)	3.14, -0.5
ikkilangandouble		8 bayt	$\pm 1.8 \times 10^{308}$ (taxminan 15 ta aniq raqam)	3.141592653589
mantiq	bool	1 bayt	rost yoki yolg'on	rost, yolg'on
belgi	char	1 bayt	0 dan 255 gacha (ASCII)	'A', '?', '5'
matn	std::string	o'zgaruvchan	har qanday matn	"Salom", "Aziza"

2.2.2 butun — butun sonlar

Bu eng ko'p ishlatiladigan tur. butun 4 baytni egallaydi va manfiy hamda musbat butun sonlarni saqlay oladi.

```
butun yosh = 25
butun temperatura = -15
butun aholisi = 35000000 // Toshkent aholisi
```

Real hayotdagi misol: Talabani yoshi, mahsulot soni, do'kondagi narx (so'mlarda), kompyuter xotirasidagi piksel koordinatasi — bularning hammasi butun.

Diqqat — diapazon! butun 4 baytlik bo'lgani uchun, u taxminan ± 2.1 milliard oralig'idagi sonlarni saqlay oladi. Agar siz bundan katta son saqlamoqchi bo'lsangiz (masalan, dunyo aholisi 8 milliard), uzun ishlatishingiz kerak.

```
butun dunyo_aholisi = 8000000000 // ← XATO! butun chegaradan tashqari
uzun dunyo_aholisi = 8000000000 // ← TO'G'RI
```

2.2.3 uzun — katta butun sonlar

8 bayt, ± 9 kvintillion (kvintillion = 10^{18}). Bu son shu darajada katta — u koinotdagi yulduzlar sonidan ko'p!

```
uzun galaktika_yulduzlari = 100000000000 // 1011
uzun fayl_hajmi_baytda = 5368709120 // 5 GB
```

2.2.4 kasr va ikkilangan — haqiqiy sonlar

Agar son kasrli bo'lsa (3.14, 0.5, -2.7), unda kasr yoki ikkilangan ishlatiladi.

Tur	Hajm	Aniqligi
kasr	4 bayt	taxminan 7 ta raqam
ikkilangan	8 bayt	taxminan 15 ta raqam

```
kasr balandlik = 1.75 // metrlarda
ikkilangan pi = 3.14159265358979
```

Qaysi birini ishlatish kerak? Ko'p hollarda — ikkilangan. U aniqroq, va zamonaviy protses-sorlar ikkilangan bilan kasr kabi tez ishlaydi. kasr ni faqat juda ko'p sonlar saqlash kerak bo'lsa ishlatish (grafika, mashina o'rganishi).

Diqqat! Kasr aniqligi. Kompyuter 0.1 ni aniq saqlay olmaydi! U binar (ikkilik) sistemada ishlaydi, va 0.1 o'nlik kasr ikkilik kasrda **cheksiz takrorlanuvchi** son. Shuning uchun:

```
ikkilangan a = 0.1 + 0.2
yozish << a << qator_oxiri // 0.30000000000000004
```

Aniqlik talab qilinadigan hisoblashlarda (pul, narx) — siz butun ishlatib so'mlarda hisoblashingiz yaxshiroq, yoki maxsus kutubxonalardan foydalanishingiz mumkin.

2.2.5 mantiq — rost yoki yolg'on

mantiq faqat ikkita qiymatni saqlay oladi: rost (true) va yolg'on (false). Bu shartlarni va bayroqlarni ifodalash uchun ishlatiladi.

```
mantiq foydalanuvchi_kirgan = rost
mantiq parol_togri = yolg'on
mantiq tugma_bosilgan = (yosh > 18)
```

Real hayotdagi misol: Lift tugmasi bosilganmi yoki yo'qmi, foydalanuvchi tizimga kirganmi, hujjat saqlandimi — barchasi mantiq.

2.2.6 belgi — bitta belgi

Bitta belgini saqlash uchun. Qo'shtirnoqlar yagona apostrofdan:

```
belgi birinchi = 'A'
belgi savol = '?'
belgi raqam = '5'           // diqqat - bu son emas, BELGI!
```

belgi aslida 1 baytlik son sifatida saqlanadi. 'A' ning ichki qiymati 65 (ASCII jadvalida). Shu sababli siz unga arifmetik amallar ham qila olasiz:

```
belgi A = 'A'
belgi B = A + 1           // 'B' bo'ladi (65 + 1 = 66)
```

Real hayotda: Eski Caesar shifri — har bir harfni 3 ta oldinga siljitish. belgi arifmetikasi bilan bunga 2 qator kod yetadi!

2.2.7 matn — matn satrlari

Matn (so'z, jumla, paragraf) matn turi orqali saqlanadi:

```
matn ism = "Aziza"
matn salom = "Salom, dunyo!"
matn paragraf = "Bu uzun matn ham bo'lishi mumkin."
```

matn — bu maxsus tur, u xotirada o'zgaruvchan miqdorda joy egallaydi (matn uzunligiga qarab). 11-bobda biz matnlarning ichini batafsil o'rganamiz.

2.3 O'zgaruvchi e'lon qilish usullari

2.3.1 1-usul: aniq tur belgisi bilan

```
butun yosh = 25
matn ism = "Aziza"
ikkilangan PI = 3.14159
```

Bu eng aniq usul. Siz kompyuterga: "men butun son saqlayman" deb aytasiz.

2.3.2 2-usul: tip xulosalash (o'zgaruvchan)

```
o'zgaruvchan yosh = 25           // butun deb xulosa qilinadi
o'zgaruvchan ism = "Aziza"       // matn deb xulosa qilinadi
o'zgaruvchan PI = 3.14159        // ikkilangan deb xulosa qilinadi
```

o'zgaruvchan (C++ da auto) — bu “qiymatdan kelib chiqib tipni o'zing aniqla” demakdir. Bu uz++ ning juda qulay xususiyati — siz qisqaroq kod yozasiz.

Qachon o'zgaruvchan ishlatish kerak? - Tip ko'rinib turibdi va aniq (o'zgaruvchan x = 5 — albatta butun) - Tip uzoq va batafsil yozish noqulay (std::map<matn, butun> kabi)

Qachon aniq tip yozish kerak? - Boshqalar uchun kod o'qish oson bo'lsin - Tip muhim va aniq belgilash kerak

2.3.3 3-usul: qiymatsiz e'lon

```
butun yosh           // butun e'lon qilindi, lekin qiymat yo'q
matn ism
```

Diqqat! Qiymatsiz e'lon qilingan butun son **noma'lum qiymat** oladi (xotirada qaysi bayt bo'lsa, o'sha). Buni “garbage value” deyiladi va bu xato manbai. Doim qiymat bering:

```
butun yosh = 0           // yaxshi
matn ism = ""           // yaxshi (bo'sh matn)
```

2.3.4 Bir nechta o'zgaruvchini bir satrda

```
butun a = 5, b = 10, c = 15
```

Yoki:

```
butun a, b, c
a = 5
b = 10
c = 15
```

Bu uslub kichik o'zgaruvchilar uchun qulay, lekin har bir o'zgaruvchi muhim bo'lsa, alohida qator bering — kod o'qish oson bo'ladi.

2.4 O'zgarmlar (Konstantalar)

Ba'zi qiymatlar **hech qachon o'zgarmasligi kerak**. Masalan: - π (pi) son — 3.14159... - Bir yildagi kunlar soni — 365 - Soatdagi soniyalar — 3600

Bunday qiymatlar uchun o'zgarmas (const) kalit so'zi ishlatiladi:

```
o'zgarmas ikkilangan PI = 3.14159
o'zgarmas butun YIL_KUNLARI = 365
o'zgarmas matn DAVLAT = "O'zbekiston"
```

Endi agar dasturda siz PI ni o'zgartirmoqchi bo'lsangiz, kompilyator xato beradi:

```
PI = 3.14 // ← XATO! o'zgarmasni o'zgartirib bo'lmaydi
```

Nima uchun o'zgarmaslar muhim? Birinchidan, ular kodingizni himoyalaydi — siz tasodifan muhim qiymatni o'zgartirib qo'yolmaysiz. Ikkinchidan, ular kodingizni o'qishni osonlashtiradi — YIL_KUNLARI ko'rsa, hamma tushunadi nima nazarda tutilayotganini. 365 ni ko'rsa esa o'ylab qolish kerak. Uchinchidan, kompilyator o'zgarmas qiymatlar bilan tezroq ishlay oladi.

Nomlash konvensiyasi: O'zgarmaslar nomlari odatda **BARCHA HARFLAR KATTA** yozilab, so'zlar _ (pastki chiziq) bilan ajratiladi. Bu o'zgaruvchilardan ko'rinish jihatidan ajratish uchun.

```
o'zgarmas butun MAKS_FOYDALANUVCHILAR = 1000 // konstanta
butun joriy_foydalanuvchilar = 25 // o'zgaruvchi
```

2.5 Tur o'zgartirish (Type casting)

Ba'zan bir turdagi qiymatni boshqa turga aylantirish kerak bo'ladi. Bu **tur o'zgartirish** (yoki **casting**) deyiladi.

2.5.1 Yashirin (implicit) o'zgartirish

uz++ ba'zan turlarni avtomatik o'zgartiradi:

```
butun a = 5
ikkilangan b = a // butun → ikkilangan, ma'lumot yo'qolmaydi
yozish << b // 5
```

Lekin ba'zan ma'lumot yo'qoladi:

```
ikkilangan x = 3.7
butun y = x // ikkilangan → butun, KASR YO'QOLADI!
yozish << y // 3 (3.7 emas!)
```

2.5.2 Aniq (explicit) o'zgartirish

Yaxshiroq — siz aniq aytasiz nimani qilmoqchisiz:


```
ikkilangan x = 3.7
butun y = butun(x)           // aniq aytamiz: butunga aylantir
```

Yoki C++ uslubida:

```
butun y = static_cast<butun>(x)
```

Real hayotdagi misol: Sizda 3.7 metr arqon bor. Lekin sizga faqat butun metr o'lchaydigan asbob berildi. Asbob "3 metr" deydi — qolgan 0.7 metr "yo'qoladi".

```
ikkilangan arqon_uzunligi = 3.7
butun ulchov = butun(arqon_uzunligi)    // 3
yozish << "Asbob ko'rsatdi: " << ulchov << " metr"
```

2.5.3 Bo'lish va integer division

uz++ da (C++ kabi) **butun ÷ butun = butun**. Kasr qismi yo'qoladi:

```
butun a = 7
butun b = 2
butun natija = a / b           // 3 (3.5 emas!)
```

Agar kasrli natija kerak bo'lsa, kamida bittasi ikkilangan bo'lsin:

```
butun a = 7
butun b = 2
ikkilangan natija = ikkilangan(a) / b    // 3.5
```

Yoki:

```
ikkilangan natija = 7.0 / 2             // 3.5 (7.0 - kasr)
```

Eng ko'p uchraydigan xato: Yangi boshlovchilar $5 / 2$ ni 2.5 deb o'ylashadi, lekin natija 2 chiqadi. Doim eslatib qo'ying: agar har ikki operand ham butun bo'lsa, natija ham butun!

2.6 O'zgaruvchining ko'rinish sohasi (Scope)

Har bir o'zgaruvchining **yashash davri** va **ko'rinish sohasi** bor. Bu nimani bildiradi?

Tasavvur qiling, omborxonada uchta zal bor: katta, o'rta, kichik. Katta zaldan siz hamma zallarni ko'rasiz. O'rta zaldan — faqat o'rta va kichik. Kichik zaldan — faqat kichik.

Xuddi shunday, o'zgaruvchi qaysi blokda e'lon qilingan bo'lsa, faqat o'sha blokda va uning ichidagi bloklarda ko'rinadi.

```
butun asosiy() {
    butun x = 10                                // x - asosiy ichida ko'rinadi

    agar (x > 0) {
        butun y = 20                            // y - faqat shu blokda
        yozish << x << " " << y              // ikkalasi ham ko'rinadi
    }

    yozish << x                                  // x ko'rinadi
    yozish << y                                  // ← XATO! y ko'rinmaydi bu yerda
}
```

2.6.1 Mahalliy (Local) o'zgaruvchilar

Funksiya yoki blok ichida e'lon qilingan o'zgaruvchilar **mahalliy** hisoblanadi. Ular blok tugashi bilan *yo'qoladi*.

2.6.2 Global o'zgaruvchilar

Funksiyalardan *tashqarida* e'lon qilingan o'zgaruvchilar **global** hisoblanadi. Ular butun dastur davomida yashaydi va hamma joydan ko'rinadi:

```
butun global_x = 100                          // global, hamma joydan ko'rinadi

butun asosiy() {
    yozish << global_x                          // 100
}
```

Diqqat — global o'zgaruvchilardan ehtiyot bo'ling! Ularni har joyda o'zgartirish mumkin, va bu dasturni o'qish hamda tuzatishni qiyinlashtiradi. Iloji bo'lsa, global o'zgaruvchilar o'rniga mahalliy ishlatib, ularni funksiyalarga parametr sifatida bering.

2.6.3 Shadowing (Soyalash)

Agar ichki blokda tashqi bilan bir xil nomli o'zgaruvchi e'lon qilinsa — ichkari *tashqarisini yopadi*:

```
butun asosiy() {
    butun x = 10
    {
        butun x = 20                            // tashqi x ni yopadi
        yozish << x                              // 20
    }
    yozish << x                                  // 10 (tashqi x qaytdi)
}
```

Bu odatda **yomon amaliyot**. Ikki xil o'zgaruvchini bir xil nomlash xatoga olib keladi.

2.7 Qiymat berish va operatorlar

2.7.1 Asosiy qiymat berish

```
butun x = 5           // x ga 5 qiymatini ber
x = 10                // x ni 10 ga o'zgartir
x = x + 1             // x ni o'zining qiymati + 1 ga o'zgartir
```

2.7.2 Qisqacha qiymat berish

$x = x + 1$ ni qisqacha yozish mumkin:

```
x += 1                // x = x + 1
x -= 2                // x = x - 2
x *= 3                // x = x * 3
x /= 4                // x = x / 4
x %= 5                // x = x % 5
```

Va eng qisqasi — **inkrement** va **dekrement**:

```
x++                  // x = x + 1 (orqa)
++x                  // x = x + 1 (oldi)
x--                  // x = x - 1
--x                  // x = x - 1
```

++x va **x++** o'rtasidagi farq: ifoda ichida qiymat *qachon* yangilanadi:

```
butun a = 5
butun b = a++        // b = 5, keyin a = 6
                     // (oldin qiymat berdi, keyin oshirdi)

butun c = 5
butun d = ++c        // c = 6, keyin d = 6
                     // (oldin oshirdi, keyin qiymat berdi)
```

Yangi boshlovchilarga: agar ifoda ichida qiymat olmasangiz (faqat `x++` yoki `++x` deb yozsangiz) — farqi yo'q.

2.8 Amaliy misol: Talabanning ma'lumotnomasi

Endi biz o'rgangan barcha narsalarni bitta amaliy dasturda ko'rib chiqamiz. Talabanning ma'lumotnomasini yarataylik.

```
// talaba.uzpp - talaba ma'lumotnomasi

o'zgarmas butun JORIY_YIL = 2026

butun asosiy() {
    // Talaba ma'lumotlari
    matn ism = "Aziza"
    matn familiya = "Karimova"
    butun tugilgan_yili = 2003
    ikkilangan ortacha_bahosi = 4.7
    mantiq stipendiyada = rost

    // Hisoblash
    butun yoshi = JORIY_YIL - tugilgan_yili

    // Chiqarish
    yozish << "=== TALABA MA'LUMOTNOMASI ===" << qator_oxiri
    yozish << "Ism: " << ism << " " << familiya << qator_oxiri
    yozish << "Yoshi: " << yoshi << qator_oxiri
    yozish << "O'rtacha baho: " << ortacha_bahosi << qator_oxiri

    agar (stipendiyada) {
        yozish << "Stipendiyada: ha" << qator_oxiri
    }
    aks_holda {
        yozish << "Stipendiyada: yo'q" << qator_oxiri
    }

    qaytarish 0
}
```

Natija:

```
=== TALABA MA'LUMOTNOMASI ===
Ism: Aziza Karimova
Yoshi: 23
O'rtacha baho: 4.7
Stipendiyada: ha
```

2.8.1 Bu dasturda nima bo'ldi?

1. **JORIY_YIL** — o'zgarmas konstanta. U hech qachon o'zgarmaydi (yoki yangi yilga ko'tarish kerak bo'lsa, **bitta joyda** — bu yerda).
2. **Turli turdagi o'zgaruvchilar** — matn (ism), butun (yosh), kasr (baho), mantiq (stipendiya). Har bir tur o'zining maqsadi uchun.
3. **Hisoblash** — `yoshi = JORIY_YIL - tugilgan_yili`. Ikki butun sonning ayirmasi yana butun.
4. **Shart operatori** — `agar` orqali boolean qiymatga qarab boshqacha xabar chiqarish.

2.9 Eng ko'p uchraydigan xatolar

2.9.1 1. Tip mosligini buzish

```
butun yosh = 3.7           // ← XATO yoki ogohlantirish: 3 bo'lib qoladi
butun yosh = 3             // TO'G'RI
ikkilangan yosh = 3.7     // TO'G'RI
```

2.9.2 2. Butun bo'lish

```
ikkilangan ortacha = 7 / 2    // ← XATO: 3.0 (3.5 emas)
ikkilangan ortacha = 7.0 / 2  // TO'G'RI: 3.5
```

2.9.3 3. Inicializatsiya qilmaslik

```
butun a                      // a - noma'lum qiymat
yozish << a                  // ← XATO: kim biladi nima chiqadi!

butun a = 0                  // TO'G'RI
```

2.9.4 4. O'zgarmasni o'zgartirishga urinish

```
o'zgarmas butun PI_TAXMINAN = 3
PI_TAXMINAN = 4              // ← XATO: o'zgarmas
```

2.9.5 5. Scope tashqarisidan o'zgaruvchini ishlatish

```
butun asosiy() {
    {
        butun temp = 100
    }
    yozish << temp           // ← XATO: temp ko'rinmaydi
}
```

2.10 Bob bo'yicha mashqlar

2.10.1 Mashq 1: Barcha turlar (oson)

Har bir asosiy tur (`butun`, `kasr`, `mantiq`, `belgi`, `matn`) uchun bittadan o'zgaruvchi yarating va konsolga chiqaring.

2.10.2 Mashq 2: Tur o'zgartirish (o'rta)

Foydalanuvchidan ikkilangan turdagi son so'rang. Uni `butun` ga aylantiring va ikkala qiymatni chiqaring. Kasr qismi qancha yo'qolganini ham ko'rsating.

Maslahat: `kasr_qismi = asl - butun(asl)`

2.10.3 Mashq 3: Yosh kalkulyatori (o'rta)

Foydalanuvchidan tug'ilgan yilini va joriy yilni so'rang. Yoshini hisoblang. Lekin bu safar: shuningdek, foydalanuvchi qancha kun, soat va daqiqa yashaganini ham hisoblang.

Maslahat: - Kunlar: `yosh × 365` (yoki aniqroq — `365.25`) - Soatlar: `kunlar × 24` - Daqiqalar: `soatlar × 60`

2.10.4 Mashq 4: O'zgarmaslarni qo'llash (o'rtacha)

Doira yuzasini hisoblovchi dastur yozing. Foydalanuvchidan radiusni so'rang. $Yuzasi = \pi \times r^2$. PI ni o'zgarmas qilib e'lon qiling.

2.10.5 Mashq 5: Pul kalkulyatori (o'rtacha)

Foydalanuvchidan miqdorni (so'mlarda) va valyuta kursini (1 dollar necha so'm) so'rang. Miqdorni dollarga aylantiring.

2.10.6 Mashq 6: Caesar shifri (qiyinroq)

Foydalanuvchidan bitta belgi so'rang. Uni 3 ta oldinga siljitib chiqaring (`A → D`, `B → E`, va h.k.).

2.10.7 Mashq 7: O'zgaruvchini almashtirish (qiyin)

Ikkita o'zgaruvchining qiymatlarini almashtirib chiqing. Sizga uchinchi o'zgaruvchi kerak — vaqtinchalik saqlash uchun.

```
butun temp = a
a = b
b = temp
```

2.11 Xulosa

Bu bobda biz:

- **O'zgaruvchi nimaligini** o'rgandik — xotirada nomlangan quti.
- **Xotira haqida** umumiy tushunchaga ega bo'ldik — har bir o'zgaruvchi baytlar miqdorini egallaydi.
- **Asosiy turlarni** ko'rib chiqdik: *butun, uzun, kasr, ikkilangan, mantiq, belgi, matn*.
- **O'zgaruvchi e'lon qilish** uch usulini bilib oldik — aniq tur, o'zgaruvchan, qiymatsiz.
- **O'zgarmaslar** (**o'zgarmas**) bilan tanishdik.
- **Tur o'zgartirishni** va undagi tuzoqlarni o'rgandik.
- **Ko'rinish sohasi (scope)** — mahalliy va global o'zgaruvchilar o'rtasidagi farq.
- **Qiymat berish operatorlari** — `=`, `+=`, `++`.
- Eng ko'p uchraydigan xatolar va ularni qanday oldini olish.

Keyingi bobda biz **operatorlar va ifodalar** haqida batafsil gaplashamiz.

03 · Operatorlar va ifodalar

3.1 Ifoda nima?

Ifoda (expression) — bu bir yoki bir nechta qiymat va ular ustida amallar bajaradigan belgilar (operatorlar) majmuasi, qaysi natijada **bitta qiymat** beradi.

Tasavvur qiling, siz oshxonada turli ingredientlardan yaxshi taom tayyorlaysiz. Kartoshka + sabzi + suv + olov = osh. Bu — *ifoda*: bir nechta narsa (operandlar) va amal (qo'shish/qaynatish — operator) bir yangi narsa (taom — qiymat) beradi.

Kodda:

```
5 + 3           // ifoda, qiymati 8
yosh > 18       // ifoda, qiymati rost yoki yolg'on
(a + b) * c     // murakkab ifoda
yozish << "Salom" // bu ham ifoda!
```

Har bir ifoda **bitta qiymat** beradi. Bu qiymatni siz o'zgaruvchiga saqlashingiz, taqqoslashingiz yoki boshqa ifodaga qo'shishingiz mumkin.

```
butun a = 5 + 3           // 8 qiymati a ga saqlandi
 mantiq katta = (a > 5)   // rost qiymati katta ga saqlandi
butun b = (a + 3) * 2     // 22 qiymati b ga saqlandi
```

Operator va operand: Bu yerda kichik atamalar bor. - **Operator** — bu amal belgisi. Masalan, +, -, *, ==. - **Operand** — bu operator ishlaydigan qiymat. 5 + 3 da 5 va 3 — operandlar, + — operator.

Endi keling, har bir operator turini batafsil ko'rib chiqamiz.

3.2 Arifmetik operatorlar

Bular eng asosiy operatorlar — siz maktabda matematikadan o'rgangan amallar. Beshtasi bor:

Operator	Ma'nosi	Misol	Natija
+	Qo'shish	$5 + 3$	8
-	Ayirish	$10 - 4$	6
*	Ko'paytirish	$6 * 7$	42
/	Bo'lish	$15 / 3$	5
%	Qoldiq (modulo)	$17 \% 5$	2

+, -, * belgilarini siz biladingiz. / (bo'lish) ham aniq. Lekin % (modulo) sizga yangi bo'lishi mumkin.

3.2.1 Modulo (%) — bo'lishdan qolgan qism

$a \% b$ — a ni b ga bo'lganda qolgan qism.

Misollar: $-17 \% 5 = 2$ ($17 = 5 \times 3 + 2$) - $10 \% 3 = 1$ ($10 = 3 \times 3 + 1$) - $20 \% 4 = 0$ (aniq bo'linadi)
 $-7 \% 2 = 1$ (7 toq) - $8 \% 2 = 0$ (8 juft)

Modulo nima uchun foydali?

1. Juft yoki toqligini aniqlash

```
agar (son % 2 == 0) {
    yozish << "Juft son" << qator_oxiri
}
aks_holda {
    yozish << "Toq son" << qator_oxiri
}
```

2. Aylanish (circular indexing)

```
butun kun = 10
matn hafta_kunlari[7] = {"Dush", "Sesh", "Chor", "Pay", "Jum", "Shan", "Yak"}
matn bugun = hafta_kunlari[kun % 7] // har 7 kunda aylanadi
```

3. Bir necha raqamga ajratish

```
butun son = 12345
butun birlar = son % 10 // 5
butun onliklar = (son / 10) % 10 // 4
butun yuzliklar = (son / 100) % 10 // 3
```

Hayotiy misol: Soat 24 soatlik ko'rsatkichdan 12 soatlikga o'tkazish kerak: $12_soatlik = 24_soatlik \% 12$. Soat 14 → 2, soat 23 → 11.

3.2.2 Bo'lish va butun bo'lish — eslang!

Avvalgi bobda ko'rganimizdek, butun / butun = butun. Kasr qismi yo'qoladi:

```
butun a = 7 / 2           // 3 (3.5 emas!)
ikkilangan b = 7.0 / 2   // 3.5
ikkilangan c = 7 / 2.0   // 3.5
ikkilangan d = 7.0 / 2.0 // 3.5
```

Eng muhim qoida: Agar siz kasr natija olishni xohlasangiz, kamida bittasi operand ikkilangan (yoki **kasr**) bo'lishi kerak.

3.2.3 Bir operandli minus (-)

– ikki holatda ishlatiladi:

```
butun a = 5 - 3           // ikki operandli ayirish
butun b = -7              // bir operandli - manfiy belgi
butun c = -a              // -5 ga teng
```

3.3 Taqqoslash operatorlari

Bular ikkita qiymatni solishtiradi. Natija har doim **mantiq** turida (rost yoki yolg'on).

Operator	Ma'nosi	Misol	Natija
==	Teng	5 == 5	rost
!=	Teng emas	5 != 3	rost
<	Kichik	3 < 5	rost
>	Katta	7 > 2	rost
<=	Kichik yoki teng	5 <= 5	rost
>=	Katta yoki teng	5 >= 3	rost

3.3.1 = va == — eng yomon sintaktik xato

Bu ikkisini farqlash juda muhim:

- = — **qiymat berish** (saqlash). `a = 5` — `a` ga 5 ni saqla.
- == — **tenglikni tekshirish**. `a == 5` — `a` 5 ga tengmi?

```
butun yosh = 18

agar (yosh = 21) {           // ← XATO: yosh ga 21 ni saqlamoqda
    // Bu blok HAR DOIM bajariladi!
```

```
}

agar (yosh == 21) {           // TO'G'RI: solishtirmoqda
    yozish << "Bu yoshda" << qator_oxiri
}
```

uz++ kompilyatori bunday xatoni ushlashga harakat qiladi (ogohlantirish beradi), lekin siz e'tiborli bo'ling.

3.3.2 Matnlarni taqqoslash

matn turi bilan ham == va != ishlaydi:

```
matn ism1 = "Aziza"
matn ism2 = "Aziza"
matn ism3 = "Bobur"

agar (ism1 == ism2) {
    yozish << "Bir xil" << qator_oxiri    // ← chiqadi
}

agar (ism1 != ism3) {
    yozish << "Farqli" << qator_oxiri    // ← chiqadi
}
```

< va > ham ishlaydi — alfavit bo'yicha tartiblaydi:

```
agar ("Aziza" < "Bobur") {
    yozish << "A B dan oldin" << qator_oxiri
}
```

3.4 Mantiqiy operatorlar

Bularning yordamida bir nechta shartni birlashtirish mumkin. Faqat **mantiq** turidagi qiymatlar bilan ishlaydi.

Operator	Ma'nosi	Misol
&&	VA (AND)	rost && rost = rost
	YOKI (OR)	rost yolg'on = rost
!	EMAS (NOT)	!rost = yolg'on

3.4.1 Haqiqat jadvali

a	b	a && b	a b
rost	rost	rost	rost
rost	yolg'on	yolg'on	rost
yolg'on	rost	yolg'on	rost
yolg'on	yolg'on	yolg'on	yolg'on

3.4.2 Misollar

```
butun yosh = 25
mantiq talaba = rost
mantiq stipendiyada = yolg'on

agar (yosh >= 18 && yosh <= 25) {
    yozish << "Yosh - talaba" << qator_oxiri
}

agar (talaba || stipendiyada) {
    yozish << "Imtiyozli" << qator_oxiri
}

agar (!stipendiyada) {
    yozish << "Stipendiya yo'q" << qator_oxiri
}
```

3.4.3 Real hayotdagi misol

Ovoz berishga ruxsat berish sharti: - Yoshi 18 dan katta YA'NI - O'zbekiston fuqarosi YA'NI - Royhatdan o'tgan

```
mantiq ruxsat = (yosh >= 18) && (fuqaro == "O'zbekiston") && royxatda
```

3.4.4 Qisqa baholash (short-circuit evaluation)

Bu — juda muhim xususiyat. uz++ (C++ kabi) mantiqiy ifodani **chapdan o'ngga** baholaydi va to'xtaydi natija ma'lum bo'lishi bilanoq.

```
// && bilan: agar birinchi yolg'on - ikkinchini umuman tekshirmaydi
agar (yolg'on && murakkab_amal()) {           // murakkab_amal() chaqirilmaydi!
    // ...
}

// || bilan: agar birinchi rost - ikkinchini umuman tekshirmaydi
agar (rost || murakkab_amal()) {               // murakkab_amal() chaqirilmaydi!
    // ...
}
```

Bu qachon foydali? Xavfsiz tekshirish uchun:

```
// Avval tekshiramiz x noldan farqli, keyin bo'lamiz
agar (x != 0 && y / x > 5) {
    // Agar x = 0 bo'lsa, y / x amalga oshmaydi
    // chunki birinchi shart yolg'on, qisqa baholash to'xtaydi
}
```

Buni “qisqa baholash” yoki “lazy evaluation” deyiladi. C++ ning klassik xususiyatlaridan biri.

3.5 Bit operatorlari

Bu operatorlar **bitlar darajasida** ishlaydi. Yangi boshlovchilar uchun bu ko'pincha murakkab tuyuladi, lekin amaliyotda juda foydali.

Eslating: har bir butun son xotirada bitlarda saqlanadi. Masalan, 5 = 00000101 (8 bit), 3 = 00000011.

Operator	Ma'nosi
&	Bit AND
	Bit OR
^	Bit XOR
~	Bit NOT (inkor)
<<	Chapga siljitish
>>	O'ngga siljitish

3.5.1 Bit AND (&)

Har bir bit pozitsiyasini alohida AND amal:

```
00000101 (5)
& 00000011 (3)
-----
00000001 (1)
```

```
butun a = 5 & 3 // 1
```

3.5.2 Bit OR (|)

```
00000101 (5)
| 00000011 (3)
-----
00000111 (7)
```

```
butun a = 5 | 3           // 7
```

3.5.3 Bit XOR (^)

XOR (eksklyuziv yoki) — agar bitlar **farqli** bo'lsa 1, aks holda 0:

```

00000101  (5)
^ 00000011  (3)
-----
00000110  (6)
```

3.5.4 Siljitish (<< va >>)

$a \ll n$ — a ning bitlari n ta chap tomonga siljiydi. Bu 2 ga ko'paytirish bilan teng:

```

butun a = 5 << 1           // 10 (5 × 2)
butun b = 5 << 2           // 20 (5 × 4)
butun c = 5 << 3           // 40 (5 × 8)
```

$a \gg n$ — a ning bitlari n ta o'ng tomonga siljiydi. Bu 2 ga bo'lish bilan teng:

```

butun a = 20 >> 1          // 10 (20 / 2)
butun b = 20 >> 2          // 5
```

Diqqat! << va >> bit operatorlari, lekin biz ularni yozish << matn da ham ishlatamiz. Bu boshqacha — yozish da bu **chiqarish operatori**, son emas. Kompilyator kontekstga qarab to'g'ri tushunadi.

3.5.5 Bit operatorlari amaliyotda

1. Sifatlar bayrog'i (flags)

Tasavvur qiling, foydalanuvchi sifatleri mavjud: o'qiy oladi, yoza oladi, administrator. Har birini bitta bit bilan ifodalaymiz:

```

o'zgarmas butun OQISH = 1           // 001
o'zgarmas butun YOZISH = 2          // 010
o'zgarmas butun ADMIN = 4           // 100

// Foydalanuvchi o'qiy oladi VA yoza oladi
butun sifatlar = OQISH | YOZISH      // 011 = 3

// O'qiy oladimi?
agar (sifatlar & OQISH) {
    yozish << "O'qiy oladi" << qator_oxiri
}
```

```
// Yoza oladimi?
agar (sifatlar & YOZISH) {
    yozish << "Yoza oladi" << qator_oxiri
}
```

Bu juda foydali — siz **bitta butun son**da 32 ta turli xil bayroqni saqlashingiz mumkin (32-bit butun uchun)!

2. Tezkor 2 ga ko'paytirish/bo'lish

Eski mashinalarda << va >> * va / dan ancha tez edi. Zamonaviy protsessorlarda farq sezilarli emas, lekin bit operatorlari hali ham kodni qisqaroq qiladi.

3.6 Qiymat berish operatorlari

Operator	Ma'nosi	Ekvivalenti
=	Sodda qiymat berish	—
+=	Qo'shib berish	$a = a + b$
-=	Ayirib berish	$a = a - b$
*=	Ko'paytirib berish	$a = a * b$
/=	Bo'lib berish	$a = a / b$
%=	Qoldiqni berish	$a = a \% b$
&=	Bit AND berish	$a = a \& b$
\ =	Bit OR berish	$a = a \ b$
<<=	Chap siljitib berish	$a = a << b$

```
butun a = 10
a += 5           // a = 15
a *= 2           // a = 30
a %= 7           // a = 2 (30 % 7)
a <<= 3          // a = 16 (2 x 8)
```

3.7 Boshqa operatorlar

3.7.1 Inkrement (++) va dekrement (--)


```
butun a = 5
a++           // a = 6 (post-increment)
++a          // a = 7 (pre-increment)
a--          // a = 6
--a          // a = 5
```

`++a` va `a++` ifoda ichida farqli ishlaydi:

```
butun a = 5
butun x = a++           // x = 5 (oldin oldi, keyin oshirdi)
yozish << a             // 6

butun b = 5
butun y = ++b           // y = 6 (oldin oshirdi, keyin oldi)
yozish << b             // 6
```

3.7.2 Uchlik operator (?:)

Bu — qisqartirilgan `agar/aks_holda`:

```
butun yosh = 20
matn natija = (yosh >= 18) ? "Voyaga yetgan" : "Voyaga yetmagan"
```

Ya'ni:

```
matn natija
agar (yosh >= 18) {
    natija = "Voyaga yetgan"
}
aks_holda {
    natija = "Voyaga yetmagan"
}
```

Sintaksis: `shart ? agar_rost_qiymat : agar_yolg'on_qiymat`

Qachon ishlatish kerak? - Oddiy ikki tomonlama tanlov uchun (kod qisqaroq) - Lekin ko'p shartlar bo'lsa — `agar/aks_holda` o'qish osonroq

3.7.3 sizeof operatori

Bu turning xotirada qancha bayt egallashini qaytaradi:

```
yozish << sizeof(butun) << qator_oxiri // 4
yozish << sizeof(ikkilangan) << qator_oxiri // 8
yozish << sizeof(belgi) << qator_oxiri // 1
```

3.8 Operatorlar ustunligi (Precedence)

Agar ifoda ichida bir nechta operator bo'lsa, ular qaysi tartibda bajariladi?

Matematikadagi kabi: birinchi $*$ va $/$, keyin $+$ va $-$.

```
butun a = 2 + 3 * 4      // 14, not 20 (3*4 oldin)
butun b = (2 + 3) * 4    // 20
```

3.8.1 To'liq jadval

Daraja	Operatorlar	Yo'nalish
1 (eng yuqori)	$()$, $[]$, $.$, $->$	chapdan o'ngga
2	$!$, $\sim$, $++$, $--$, $-$ (unar)	o'ngdan chapga
3	$*$, $/$, $\%$	chapdan o'ngga
4	$+$, $-$	chapdan o'ngga
5	$<<$, $>>$	chapdan o'ngga
6	$<$, $<=$, $>$, $>=$	chapdan o'ngga
7	$=$, $!=$	chapdan o'ngga
8	$\&$ (bit)	chapdan o'ngga
9	$\wedge$	chapdan o'ngga
10	$\backslash $ (bit)	chapdan o'ngga
11	$\&\&$	chapdan o'ngga
12	$\backslash \backslash $	chapdan o'ngga
13	$?:$	o'ngdan chapga
14	$=$, $+=$, $-=$, $\dots$	o'ngdan chapga

Eslab qolish kerak emas! Ikkita maslahat:

1. **Murakkab ifodalarda doimo qavslar qo'ying.** Hatto kerak bo'lmasa ham.

```
// Yomon - ustunlikka tayanish
butun natija = a + b * c << 2

// Yaxshi - aniq ifoda
butun natija = (a + (b * c)) << 2
```

2. **Kod o'qish kerakliroq ekanligini eslang.** Sizning kodingizni keyin boshqalar (yoki kelajakdagi siz) o'qiydi. Ortiqcha qavslar yomon emas, ular yordam beradi.

3.9 Amaliy misol: Kvadrat tenglama yechish

$ax^2 + bx + c = 0$ ko'rinishidagi tenglamani yechish. Diskriminant: $D = b^2 - 4ac$.

```
// kvadrat_tenglama.uzpp

butun asosiy() {
    // Koeffitsiyentlarni kiritish
    ikkilangan a, b, c
    yozish << "a koeffitsiyent: "
    kiritish >> a
    yozish << "b koeffitsiyent: "
    kiritish >> b
    yozish << "c koeffitsiyent: "
    kiritish >> c

    // Diskriminant
    ikkilangan D = b * b - 4 * a * c
    yozish << "Diskriminant D = " << D << qator_oxiri

    // Yechimlarni hisoblash
    agar (D > 0) {
        ikkilangan x1 = (-b + std::sqrt(D)) / (2 * a)
        ikkilangan x2 = (-b - std::sqrt(D)) / (2 * a)
        yozish << "Ikkita haqiqiy ildiz:" << qator_oxiri
        yozish << "  x_1 = " << x1 << qator_oxiri
        yozish << "  x_2 = " << x2 << qator_oxiri
    }
    aks_holda agar (D == 0) {
        ikkilangan x = -b / (2 * a)
        yozish << "Bitta haqiqiy ildiz: x = " << x << qator_oxiri
    }
    aks_holda {
        yozish << "Haqiqiy ildizlar mavjud emas." << qator_oxiri
    }

    qaytarish 0
}
```

Natija (kiritilgan: a=1, b=-5, c=6):

```
Diskriminant D = 1
Ikkita haqiqiy ildiz:
x_1 = 3
x_2 = 2
```

3.9.1 Bu dasturda nima kechdi?

1. **ikkilangan turi** — koeffitsiyentlar va natijalar kasr bo'lishi mumkin.
2. **std::sqrt(D)** — kvadrat ildiz funksiyasi. 8-bobda funksiyalar haqida o'rganganimizda batafsil ko'ramiz.
3. **Operatorlar ustunligi** — $b * b - 4 * a * c$ da $*$ — dan oldin ishlaydi: aslida $(b*b) - (4*a*c)$. Lekin tushunmoqchi bo'lsangiz, qavs qo'yish yaxshi.
4. **aks_holda agar** — bir necha shartlarni ketma-ket tekshirish.

3.10 Eng ko'p uchraydigan xatolar

3.10.1 1. = va == ni adashtirish

```
agar (yosh = 18) { ... }    // ← XATO: 18 ni saqlamoqda
agar (yosh == 18) { ... }  // TO'G'RI
```

3.10.2 2. Butun bo'lish kutilmagan natija beradi

```
ikkilangan ortacha = (1 + 2 + 3) / 3    // 2 (2.0 emas!)
ikkilangan ortacha = (1 + 2 + 3) / 3.0  // 2.0
```

3.10.3 3. Operatorlar ustunligini noto'g'ri tushunish

```
agar (a == 5 || b == 5 && c == 5)
// Bu aslida: a == 5 || (b == 5 && c == 5)
// Sizning niyatingiz boshqa edimi? Qavs qo'ying!
agar ((a == 5 || b == 5) && c == 5)
```

3.10.4 4. Sonni 0 ga bo'lish

```
butun a = 10
butun b = 0
butun c = a / b    // ← XATO: dastur to'xtaydi (crash)
```

Tekshiring:

```
agar (b != 0) {
    butun c = a / b
}
aks_holda {
    yozish << "0 ga bo'lib bo'lmaydi" << qator_oxiri
}
```

3.10.5 5. && va & ni adashtirish

```
agar (a > 5 & b < 10)      // bit AND - yomon, lekin ishlaydi
agar (a > 5 && b < 10)     // mantiqiy AND - TO'G'RI
```

Mantiqiy operatorlar uchun har doim **ikki belgi** (&&, ||).

3.11 Bob bo'yicha mashqlar

3.11.1 Mashq 1: Sodda kalkulyator (oson)

Foydalanuvchidan ikkita son va amal (+, -, *, /) so'rang. Natijani chiqaring.

3.11.2 Mashq 2: Juft yoki toq (oson)

Foydalanuvchidan son so'rang. % operatori orqali u juft yoki toq ekanligini aniqlang.

3.11.3 Mashq 3: Yil — kabisa yili (o'rta)

Kabisa yili — bu: - 4 ga bo'linadi, lekin - 100 ga bo'linmaydi yoki - 400 ga bo'linadi

Foydalanuvchidan yil so'rang va kabisami yoki yo'qligini aniqlang.

Maslahat:

```
mantiq kabisa = (yil % 4 == 0 && yil % 100 != 0) || (yil % 400 == 0)
```

3.11.4 Mashq 4: Sonning raqamlari yig'indisi (o'rta)

3 xonali son uchun raqamlarini ajrating va yig'indisini hisoblang. Masalan, 234 → 2 + 3 + 4 = 9.

Maslahat: % va / ishlatib ajratish mumkin.

3.11.5 Mashq 5: Soat hisoblovchi (o'rtacha)

Sekundlar kiritilsa, ularni soat:daqiqa:sekund ko'rinishida chiqaring. Masalan, 3725 sekund → 1:02:05.

Maslahat:

```
butun soat = sekundlar / 3600
butun daqiqa = (sekundlar % 3600) / 60
butun sekund = sekundlar % 60
```

3.11.6 Mashq 6: Bayroqlar (qiyinroq)

3 ta bayroq belgilang: O'QISH, YOZISH, BAJARISH (har biri bitta bit). Foydalanuvchining sifatlarini kiriting (*butun* qiymat sifatida) va har bir sifat uchun “ha/yo’q” deb chiqaring.

3.11.7 Mashq 7: Eng katta uchidan (qiyin)

Foydalanuvchidan 3 ta son so’rang. Eng kattasini toping. Faqat `agar/aks_holda` va taqqoslash operatorlaridan foydalaning.

3.12 Xulosa

Bu bobda biz:

- Ifoda nimaligini tushunib oldik — operandlar va operatorlardan iborat, qiymat beruvchi kod bo’lagi.
- Arifmetik operatorlar (+, -, *, /, %) bilan ishladik va **modulo** ning amaliy ishlatilishini ko’rdik.
- Taqqoslash operatorlari bilan tanishdik va = va == ni farqlashni o’rgandik.
- Mantiqiy operatorlar (&&, ||, !) va ularning qisqa baholash xususiyatini bilib oldik.
- Bit operatorlari (&, |, ^, <<, >>) bilan tanishdik va ularning bayroqlar ko’rinishidagi amaliy qo’llanishini ko’rdik.
- Qiymat berish operatorlari (=, +=, -=, va h.k.) ni o’rgandik.
- Uchlik operator (?:) va sizeof bilan tanishdik.
- Operatorlar ustunligi ni ko’rdik va qavslar muhimligini tushundik.
- Amaliyotda kvadrat tenglamani yechib chiqdik.

Keyingi bobda biz **boshqaruv tuzilmalari** ni o’rganamiz — `agar`, `uchun`, `davr` kabi konstruktsiyalar yordamida dasturning oqimini boshqarishni o’rganasiz. Bularsiz dasturlar — bu shunchaki bir necha qator ko’rsatmalar; ular bilan — bu mantiqiy mexanizmlar.

04 · Boshqaruv tuzilmalari

4.1 Dastur oqimi nima?

Hozirgacha bizning dasturlarimiz oddiy edi — har bir qator yuqoridan pastga, ketma-ket bajarilar edi. Lekin haqiqiy dasturlar shunday ishlamaydi. Kompyuter sharoitlarga qarab boshqacha yo'l tutadi, takroriy ishlarni avtomatik qiladi, va kerakli paytda to'xtab biror ish qiladi.

Tasavvur qiling, siz mehmonni kutib olyapsiz. Sizning xatti-harakatingiz ham sodda emas:

- **Agar** eshik qo'ng'irog'i jiringlasa — eshikni oching.
- **Agar** mehmon yaqin do'st bo'lsa — quchoqlang. **Aks holda** — qo'l berib salomlashing.
- Choy quying. **Bu jarayonni** har bir mehmon uchun **takrorlang**.
- Mehmonlar ketgancha — ketishni kutib turing. **To'xtang** faqat hammasi ketganidan keyin.

Bu — **boshqaruv mantiqi**. Dasturda ham xuddi shunday: biz kompyuterga ayrim shartlarda boshqacha amal qilishni, ba'zi ishlarni takrorlashni, kerak bo'lganda to'xtashni buyurishimiz kerak.

Bu boshqaruvga uchta asosiy mexanizm xizmat qiladi:

1. **Tanlov (shart)** — agar shart bajarilsa, biror kodni bajar (agar)
2. **Takrorlash (sikl)** — kodni bir necha marta bajar (uchun, davr)
3. **Tartibni o'zgartirish** — sikldan chiqish, keyingi takrorlashga o'tish (to'xtatish, davom ettirish)

4.2 Shart — agar operatori

4.2.1 Eng oddiy shakl

```
agar (shart) {  
    // shart rost bo'lsa, bu blok bajariladi  
}
```

Misol:

```
butun yosh = 20

agar (yosh >= 18) {
    yozish << "Siz voyaga yetgansiz" << qator_oxiri
}
```

Bu yerda: - (yosh >= 18) — **shart**. Bu mantiqiy ifoda — uning natijasi rost yoki yolg'on. - { ... } — **blok**. Shart rost bo'lsa, ichidagi kod bajariladi.

4.2.2 agar/aks_holda

Ko'pincha biz “agar bu — buni qil, aks holda — buni” demoqchimiz:

```
agar (yosh >= 18) {
    yozish << "Voyaga yetgan" << qator_oxiri
}
aks_holda {
    yozish << "Voyaga yetmagan" << qator_oxiri
}
```

aks_holda (C++ da else) ni keng tilda “boshqa”, “agar bu shartni qondirmasa” deb tarjima qilish mumkin.

4.2.3 aks_holda agar — bir necha shart

Ko'p marta biz uchta yoki undan ko'p tanlovga duch kelamiz:

```
butun baho = 87

agar (baho >= 90) {
    yozish << "A'lo" << qator_oxiri
}
aks_holda agar (baho >= 80) {
    yozish << "Yaxshi" << qator_oxiri
}
aks_holda agar (baho >= 60) {
    yozish << "Qoniqarli" << qator_oxiri
}
aks_holda {
    yozish << "Qoniqarsiz" << qator_oxiri
}
```

Muhim: Shartlar **yuqoridan pastga** tekshiriladi. Birinchi rost shart topilsa, qolganlari tekshirilmaydi.

Real hayotdagi misol: Avtomobil yo'lidagi yo'l harakati boshqaruvi. Avval qizil chiroqni tekshiramiz (to'xtang), keyin sariqni (sekinlashing), keyin yashilni (yuring). Sariq chiroqni qizil borligida ham yondiramiz. Har bir shart oldingisini istisno qiladi.

4.2.4 Ichma-ich shartlar

Shart ichida yana shart bo'lishi mumkin:

```
agar (yosh >= 18) {
  agar (haydovchilik_guvohnomasi) {
    yozish << "Avtomobil haydashga ruxsat" << qator_oxiri
  }
  aks_holda {
    yozish << "Avval guvohnoma olishingiz kerak" << qator_oxiri
  }
}
aks_holda {
  yozish << "Voyaga yetmaganlar avtomobil haydolmaydi" << qator_oxiri
}
```

Buni && bilan ham yozish mumkin:

```
agar (yosh >= 18 && haydovchilik_guvohnomasi) {
  yozish << "Avtomobil haydashga ruxsat" << qator_oxiri
}
aks_holda agar (yosh >= 18) {
  yozish << "Avval guvohnoma olishingiz kerak" << qator_oxiri
}
aks_holda {
  yozish << "Voyaga yetmaganlar avtomobil haydolmaydi" << qator_oxiri
}
```

Ikkalasi ham to'g'ri, lekin kontekstga qarab biri yoki ikkinchisi o'qish osonroq bo'ladi.

4.2.5 Bir qatorli shart

Agar shart blokida bitta ko'rsatma bo'lsa, qavslarsiz yozish mumkin:

```
agar (yosh >= 18) yozish << "Voyaga yetgan" << qator_oxiri
```

Lekin **biz buni tavsiya qilmaymiz**. Qavslar bo'lsa, kod tartibga keladi va kelajakda yangi qator qo'shish oson bo'ladi.

4.3 tanlash — bir nechta yo'naltirishlardan tanlash

Agar siz bir nechta aniq qiymatlarni tekshirayotgan bo'lsangiz, tanlash (C++ da switch) qulayroq:

```
butun kun = 3

tanlash (kun) {
    holat 1: yozish << "Dushanba" << qator_oxiri; to'xtatish
    holat 2: yozish << "Seshanba" << qator_oxiri; to'xtatish
    holat 3: yozish << "Chorshanba" << qator_oxiri; to'xtatish
    holat 4: yozish << "Payshanba" << qator_oxiri; to'xtatish
    holat 5: yozish << "Juma" << qator_oxiri; to'xtatish
    holat 6: yozish << "Shanba" << qator_oxiri; to'xtatish
    holat 7: yozish << "Yakshanba" << qator_oxiri; to'xtatish
    boshqa: yozish << "Noto'g'ri kun" << qator_oxiri
}
```

4.3.1 Sintaksis va qoidalar

- tanlash (ifoda) — qaysi qiymatni tekshirayapmiz
- holat qiymat: — agar ifoda == qiymat bo'lsa, bu bo'limga o'tamiz
- to'xtatish — tanlash dan chiqamiz (boshqasini ham tekshirmaymiz)
- boshqa: — agar hech bir holat mos kelmasa, bu bajariladi (C++ da default)

Diqqat — to'xtatish ni unutmang! Agar to'xtatish yo'q bo'lsa, dastur keyingi holatga ham o'tib ketadi. Bu ba'zan ataylab qilinadi, lekin ko'pincha xato.

4.3.2 to'xtatish siz “fallthrough”

```
butun oy = 2

tanlash (oy) {
    holat 12:
    holat 1:
    holat 2:
        yozish << "Qish" << qator_oxiri
        to'xtatish
    holat 3:
    holat 4:
    holat 5:
        yozish << "Bahor" << qator_oxiri
        to'xtatish
    holat 6:
    holat 7:
    holat 8:
        yozish << "Yoz" << qator_oxiri
}
```

```

        to'xtatish
holat 9:
holat 10:
holat 11:
    yozish << "Kuz" << qator_oxiri
    to'xtatish
}

```

12, 1, va 2-oylar uchun bitta natija — “Qish”. `to'xtatish` qo'yilmagani sababli kompilyator pastki holatlarga oqib boradi.

4.3.3 tanlash ning chegaralari

`tanlash` faqat **butun va belgi** turi bilan ishlay oladi. Matn, kasr yoki mantiq bilan ishlamaydi. Shu bilan bir qatorda — har bir holat **aniq qiymat** bo'lishi kerak, oraliq emas.

```

tanlash (baho) {
    holat (baho >= 90):      // ← XATO! Bunday yozib bo'lmaydi
    ...
}

```

Bu hollarda `agar/aks_holda` ishlatilg.

4.4 Sikl — davr (while)

Sikl — bu kod blokini bir necha marta takrorlovchi konstruktsiya. **davr** (C++ da `while`) eng oddiy sikl:

```

davr (shart) {
    // shart rost bo'lguncha takrorlanadi
}

```

Misol — 1 dan 5 gacha sonlarni chiqarish:

```

butun i = 1
davr (i <= 5) {
    yozish << i << qator_oxiri
    i++
}

```

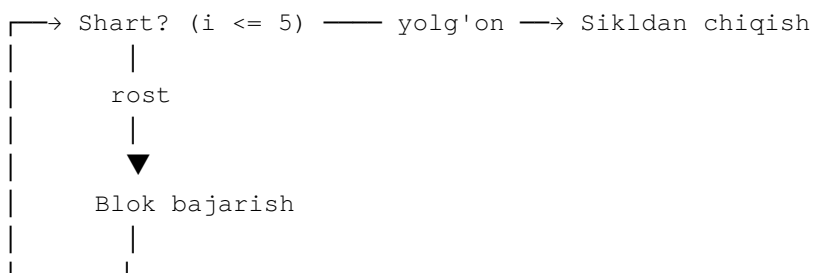
Natija:

1
2
3
4
5

4.4.1 Qanday ishlaydi?

Har bir iteratsiyada:

1. Shart tekshiriladi ($i \leq 5$)
2. Agar rost — blok bajariladi
3. Blok tugagandan keyin yana 1-bosqichga qaytamiz
4. Agar yolg'on — sikldan chiqamiz



4.4.2 Sikl o'zgaruvchisi (counter)

Yuqoridagi i o'zgaruvchisi — **sikl o'zgaruvchisi** yoki **counter**. U sikl davomida o'zgaradi va sikl qachon to'xtashini belgilaydi.

Diqqat! Agar siz $i++$ ni unutsangiz, sikl **cheksiz** bo'ladi — dastur hech qachon to'xtamaydi:

```

butun i = 1
davr (i <= 5) {
    yozish << i << qator_oxiri
    // i++ unutilgan - cheksiz sikl!
}
  
```

Bunday holatlarda dasturni Ctrl+C bilan to'xtatishingiz kerak.

4.4.3 bajar/davr — kamida bir marta bajariladi

```

butun javob

bajar {
    yozish << "0 dan katta son kiriting: "
    kiritish >> javob
} davr (javob <= 0)

yozish << "Rahmat! " << javob << " qabul qilindi" << qator_oxiri
  
```

Bu yerda farq: blok **birinchi marta har holda** bajariladi, keyin shart tekshiriladi.

Qachon bajar/davr ishlatish kerak? Foydalanuvchidan ma'lumot so'raganda — siz birinchi marta savol berishingiz **shart**, keyin javobiga qarab davom etish yoki to'xtash.

4.5 Sikl — uchun (for)

`uchun` — eng ko'p ishlatiladigan sikl. U `davr` ga qaraganda qisqaroq va aniqroq.

4.5.1 Sintaksis

```
uchun (ishga tushirish; shart; o'zgartirish) {
    // takrorlanadigan blok
}
```

Ucha qism — ; bilan ajratiladi:

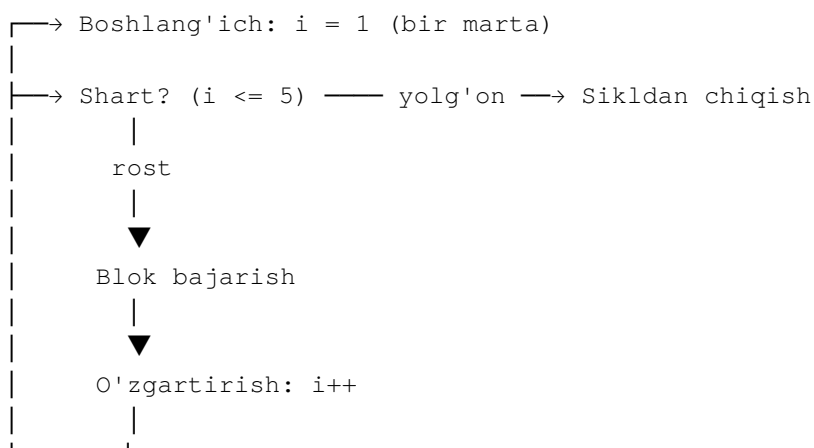
1. **Ishga tushirish** — sikl boshlanishidan oldin bir marta bajariladi
2. **Shart** — har iteratsiyada tekshiriladi
3. **O'zgartirish** — har iteratsiya oxirida bajariladi

Misol — 1 dan 5 gacha sonlarni chiqarish (yuqoridagi `davr` kabi, lekin qisqaroq):

```
uchun (butun i = 1; i <= 5; i++) {
    yozish << i << qator_oxiri
}
```

Bu xuddi `davr` kabi ishlaydi, lekin 3 qator o'rniga 1 qator.

4.5.2 uchun qanday ishlaydi?



4.5.3 Murakkab uchun sikllari

```
// 10 dan 1 gacha teskari hisoblash
uchun (butun i = 10; i >= 1; i--) {
    yozish << i << " "
}
// Natija: 10 9 8 7 6 5 4 3 2 1

// 2 lik qadam bilan
uchun (butun i = 0; i <= 20; i += 2) {
    yozish << i << " "
}
// Natija: 0 2 4 6 8 10 12 14 16 18 20

// Quvvatlash - 1, 2, 4, 8, 16, ...
uchun (butun i = 1; i <= 1000; i *= 2) {
    yozish << i << " "
}
// Natija: 1 2 4 8 16 32 64 128 256 512
```

4.5.4 Range-based uchun (uchun har)

Agar siz to'plam (massiv, vector) bo'yicha takrorlamoqchi bo'lsangiz, oddiy sintaksis:

```
vektor<butun> sonlar = {10, 20, 30, 40, 50}

uchun (butun son : sonlar) {
    yozish << son << qator_oxiri
}
}
```

Bu — *range-based for* deb ataladi. Buni to'plamlar haqida 7-bobda ko'ramiz.

4.6 Sikl ichida sikl (Nested loops)

Bir sikl ichida boshqa sikl bo'lishi mumkin. Bu ichma-ich sikllar.

4.6.1 Misol — ko'paytirish jadvali

```
uchun (butun i = 1; i <= 5; i++) {
    uchun (butun j = 1; j <= 5; j++) {
        yozish << (i * j) << "\t"
    }
    yozish << qator_oxiri
}
```

Natija:

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20
5	10	15	20	25

4.6.2 Qanday ishlaydi?

- Tashqi sikl (*i*) 1 dan 5 gacha boradi.
- Har bir *i* qiymati uchun, ichki sikl (*j*) 1 dan 5 gacha boradi.
- Jami $5 \times 5 = 25$ marta (*i* * *j*) chop etiladi.

4.6.3 Ehtiyot bo'ling — murakkablik

Ichma-ich sikllar **tez sekinlashadi**:

- 1 sikl $\times$ 1000 iteratsiya = 1000 amal
- 2 ichma-ich sikl $\times$ 1000 iteratsiya = 1,000,000 amal
- 3 ichma-ich sikl $\times$ 1000 iteratsiya = 1,000,000,000 amal (1 milliard)

Agar siz katta ma'lumotlar bilan ishlasangiz, ehtiyot bo'ling: 3 ichma-ich sikl bilan dastur soatlab ishlashi mumkin.

4.7 Sikldan chiqish va o'tkazib yuborish**4.7.1 to'xtatish (break)**

Sikldan **darhol chiqish**. Sikldagi qolgan iteratsiyalar bekor qilinadi.

```

uchun (butun i = 1; i <= 100; i++) {
    agar (i == 5) {
        to'xtatish
    }
    yozish << i << " "
}
// Natija: 1 2 3 4

```

Real qo'llanish: Foydalanuvchidan parol so'rash — agar to'g'ri bo'lsa, sikldan chiqish:

```

davr (rost) {
    matn parol
    yozish << "Parolni kiriting: "

```

```
kiritish >> parol

agar (parol == "salom123") {
  yozish << "Tabriklayman! Tizimga kirdingiz" << qator_oxiri
  to'xtatish
}
aks_holda {
  yozish << "Noto'g'ri parol. Qayta urinib ko'ring" << qator_oxiri
}
}
```

davr (rost) — bu cheksiz sikl. U faqat to'xtatish orqali to'xtaydi.

4.7.2 davom_ettirish (continue)

Joriy iteratsiyani o'tkazib yuborish, lekin sikldan chiqmaslik — keyingi iteratsiyaga o'tish.

```
uchun (butun i = 1; i <= 10; i++) {
  agar (i % 2 == 0) {
    davom_ettirish      // juft sonlarni o'tkazib yuboramiz
  }
  yozish << i << " "
}
// Natija: 1 3 5 7 9 (faqat toq sonlar)
```

Eng ko'p uchraydigan xato: to'xtatish va davom_ettirish ni chalkashtirish. to'xtatish — sikldan butunlay chiqish. davom_ettirish — faqat joriy iteratsiyani o'tkazib, keyingisiga o'tish.

4.8 Cheksiz sikllar va ulardan qutilish

4.8.1 davr (rost)

Bu eng tipik cheksiz sikl. Faqat to'xtatish orqali chiqish mumkin.

4.8.2 Cheksiz uchun

```
uchun (;;) {
  // ...
}
```

Bo'sh shart rost deb hisoblanadi.

4.8.3 Qachon cheksiz sikl foydali?

- Foydalanuvchidan kirim olganda (xato bo'lsa qayta so'rash uchun)
- Server dasturlarida — har doim mijozlardan ulanishlarni kutish
- O'yin "loop"larida — har bir kadrni qayta-qayta chizish

4.9 Amaliy misollar

4.9.1 1-misol: Mukammal son tekshiruvchi

Mukammal son — bu uning barcha bo'luvchilari (o'zi tashqari) yig'indisi o'ziga teng bo'lgan son. Masalan: $6 = 1 + 2 + 3$.

```
butun asosiy() {
    butun son
    yozish << "Sonni kiriting: "
    kiritish >> son

    butun yigindi = 0
    uchun (butun i = 1; i < son; i++) {
        agar (son % i == 0) {
            yigindi += i
        }
    }

    agar (yigindi == son) {
        yozish << son << " - mukammal son!" << qator_oxiri
    }
    aks_holda {
        yozish << son << " - mukammal son emas" << qator_oxiri
    }

    qaytarish 0
}
```

4.9.2 2-misol: Fibonachchi sonlari

Fibonachchi sonlari: 0, 1, 1, 2, 3, 5, 8, 13, 21, ... — har bir keyingi son oldingi ikkitasining yig'indisi.

```
butun asosiy() {
    butun n
    yozish << "Necha ta Fibonachchi soni? "
    kiritish >> n

    butun a = 0
    butun b = 1
```

```
    uchun (butun i = 0; i < n; i++) {  
        yozish << a << " "  
        butun temp = a + b  
        a = b  
        b = temp  
    }  
    yozish << qator_oxiri  
  
    qaytarish 0  
}
```

4.9.3 3-misol: Tub sonlarni topish

Tub son — bu faqat 1 va o'ziga bo'linadigan son. 2, 3, 5, 7, 11, 13...

```
butun asosiy() {  
    butun n  
    yozish << "N gacha tub sonlar (N): "  
    kiritish >> n  
  
    uchun (butun son = 2; son <= n; son++) {  
        mantiq tub = rost  
        uchun (butun i = 2; i * i <= son; i++) {  
            agar (son % i == 0) {  
                tub = yolg'on  
                to'xtatish  
            }  
        }  
        agar (tub) {  
            yozish << son << " "  
        }  
    }  
    yozish << qator_oxiri  
  
    qaytarish 0  
}
```

Kichik tushuntirish: nima uchun $i * i \leq \text{son}$? Chunki agar son bo'luvchisi $\text{sqrt}(\text{son})$ dan kichik bo'lsa, biz uni topamiz. Aks holda — u tub son. Bu siklning ishlash tezligini sezilarli oshiradi.

4.10 Eng ko'p uchraydigan xatolar

4.10.1 1. Cheksiz sikl

```
butun i = 1
davr (i <= 10) {
    yozish << i << qator_oxiri
    // i++ ni unutdi! Cheksiz sikl.
}
```

4.10.2 2. Off-by-one xato

```
// "1 dan 10 gacha" deb yozmoqchimiz
uchun (butun i = 1; i < 10; i++) // 1 dan 9 gacha (faqat 9!)
uchun (butun i = 1; i <= 10; i++) // 1 dan 10 gacha (TO'G'RI)
uchun (butun i = 0; i < 10; i++) // 0 dan 9 gacha (10 ta son)
```

Off-by-one — bu eng ko'p uchraydigan xato. Doim aniq tekshiring: “Birinci qiymat nima? Oxirgi qiymat nima?”

4.10.3 3. to'xtatish faqat bitta sikldan chiqaradi

```
uchun (butun i = 0; i < 10; i++) {
    uchun (butun j = 0; j < 10; j++) {
        agar (j == 5) {
            to'xtatish // Faqat ichki sikldan chiqadi!
        }
    }
    // Tashqi sikl davom etadi
}
```

4.10.4 4. Switch ichida to'xtatish ni unutish

```
tanlash (kun) {
    holat 1: yozish << "Dush"
    holat 2: yozish << "Sesh"
}
// "Dush" ham, "Sesh" ham chiqadi (fallthrough)!
```

4.10.5 5. = ni shart ichida ishlatish

```
agar (a = 5) { ... } // ← XATO: 5 ni saqladi, har doim rost
agar (a == 5) { ... } // TO'G'RI
```

4.11 Bob bo'yicha mashqlar

4.11.1 Mashq 1: Eng katta son (oson)

Foydalanuvchidan 3 ta son so'rang. `agar/aks_holda` orqali eng kattasini toping.

4.11.2 Mashq 2: Baho hisoblovchi (oson)

Talabadan ball so'rang (0-100). Bahoni chiqaring: - 90+: A'lo - 70-89: Yaxshi - 60-69: Qoniqarli - <60: Qoniqarsiz

4.11.3 Mashq 3: Faktoriyal (o'rta)

$n!$ ni hisoblang ($5! = 1*2*3*4*5 = 120$). Foydalanuvchidan n so'rang.

```
butun faktorial = 1
uchun (butun i = 1; i <= n; i++) {
    faktorial *= i
}
```

4.11.4 Mashq 4: Sonni teskari aylantirish (o'rta)

Foydalanuvchidan butun son so'rang. Uni teskari aylantiring ($123 \rightarrow 321$).

Maslahat:

```
butun teskari = 0
davr (son > 0) {
    teskari = teskari * 10 + son % 10
    son /= 10
}
```

4.11.5 Mashq 5: Yulduzlar piramidasi (o'rtacha)

Quyidagicha piramida chiqaring ($n=5$ uchun):

```
  *
 ***
*****
*****
*****
```

Maslahat: Ichma-ich sikllar. Tashqi sikl — qatorlar uchun. Ichki — bo'sh joylar va yulduzlar uchun.

4.11.6 Mashq 6: Tasodifiy sonlar o'yini (qiyinroq)

Kompyuter 1 dan 100 gacha tasodifiy son tanlasin. Foydalanuvchi taxmin qiladi. Har taxminga “kattaroq” yoki “kichikroq” deb yozsin. Foydalanuvchi topgancha davom eting va necha urinish kerakligini chiqaring.

Maslahat: Tasodifiy son uchun:

```
#include <random>
butun sirli_son = std::rand() % 100 + 1
```

4.11.7 Mashq 7: Eng katta umumiy bo'luvchi (qiyin)

Ikkita son uchun GCD (greatest common divisor) ni Euclid algoritmi bilan toping.

Maslahat:

```
davr (b != 0) {
    butun temp = b
    b = a % b
    a = temp
}
// a - javob
```

4.12 Xulosa

Bu bobda biz dastur oqimini boshqarishni o'rgandik:

- **agar/aks_holda** — shartga qarab boshqacha kod bajarish.
- **tanlash** — bir nechta aniq qiymatdan tanlash, tanlash ning holat va to'xtatish qoidalari.
- **davr** — shart rost bo'lguncha takrorlash.
- **bajar/davr** — kamida bir marta bajaradigan sikl.
- **uchun** — ishga tushirish, shart va o'zgartirish bilan eng ko'p ishlatiladigan sikl.
- **Sikl ichida sikl** — ichma-ich sikllar va ularning murakkabligi.
- **to'xtatish** va **davom_ettirish** — sikldan chiqish va iteratsiyani o'tkazib yuborish.
- **Amaliyotda** — Fibonachchi, mukammal son, tub sonlar dasturlarini yozdik.

Keyingi bobda biz **funksiyalar** ni o'rganamiz — kodni qayta ishlatish va dasturni mantiqiy bo'laklarga ajratish eng muhim usul. Hozirgi katta dasturlaringizni o'nlab funksiyalarga bo'lib, har birini alohida tushunarli qila olasiz.

05 · Funksiyalar

5.1 Funksiya nima va nima uchun kerak?

Tasavvur qiling, siz oshpazsiz. Restoraningizda 50 ta xil taom bor. Lekin har bir taomda — chunin, palov, manti — *kartoshkani arrachilash* kerak. Bu — bir xil ish, lekin u 50 marta takrorlanadi. Yomon strategiya: har taom retseptida “kartoshkani arrachilash” bo’limini qayta-qayta yozish. Yaxshi strategiya: “kartoshkani arrachilash” usulini bitta joyda batafsil yozib qo’yish, keyin har taom retseptida “kartoshkani arrachilash” deb havola qilish.

Dasturlashda **funksiya** — aynan shu havola qilingan ish bo’lagi. Bir marta yozilgan, keyin *istalgan joyda* chaqirilishi mumkin.

Funksiyaning afzalliklari:

1. **Kodni qayta ishlatish** — bir marta yoz, ko’p marta foydalan.
2. **Tushunarli kod** — katta dastur kichik bo’laklarga ajraladi.
3. **Tuzatishni osonlashtirish** — bitta joyni o’zgartirsangiz, hamma chaqirish o’zgaradi.
4. **Mantiqiy izolyatsiya** — har bir funksiyaning bir aniq vazifasi bor.

5.1.1 Birinchi funksiya — oddiy misol

```
// salomlashish funksiyasi
bosh salom_ber(matn ism) {
    yozish << "Salom, " << ism << "!" << qator_oxiri
}

butun asosiy() {
    salom_ber("Aziza")           // funksiyani chaqirish
    salom_ber("Bobur")
    salom_ber("Karim")
    qaytarish 0
}
```

Natija:

Salom, Aziza!

Salom, Bobur!

Salom, Karim!

Bitta funksiya — uchta chaqirish. Tasavvur qiling, 100 ta foydalanuvchi bo'lsa, har biri uchun “yozish « ...” ni qaytaqayta yozish kerak emas.

5.2 Funksiyaning anatomiyasi

```
qaytarish_turi funksiya_nomi ( parametrlar ) {  
    // funksiya tanasi  
    qaytarish qiymat  
}
```

Misol uchun:

```
butun kvadrat(butun x) {  
    qaytarish x * x  
}
```

Tahlil qilamiz:

Qism	Misol	Ma'nosi
Qaytarish turi	butun	Funksiya qaytaradigan qiymat turi
Nom	kvadrat	Funksiyaning nomi
Parametrlar	butun x	Kirish qiymatlari (ko'p bo'lishi mumkin)
Tanasi	{ ... }	Bajariladigan kod
qaytarish	qaytarish x * x	Qaytadigan qiymat

5.2.1 Funksiya chaqirish

```
butun natija = kvadrat(5)           // 25  
butun ikkinchi = kvadrat(10)        // 100
```

Bu yerda biz kvadrat funksiyasini ikki marta chaqirdik, har safar boshqa qiymat berib.

5.3 Parametrlar

Parametr — bu funksiyaga kiritiluvchi qiymat. Funksiyaga *kirish nuqtasi*.

5.3.1 Parametrsiz funksiya

```
bosh salom_ber() {
  yozish << "Salom!" << qator_oxiri
}

salom_ber()           // chaqirish
salom_ber()           // yana
```

Hech qanday kirish kerak emas — har safar bir xil ish.

5.3.2 Bir nechta parametr

```
butun yigindi(butun a, butun b) {
  qaytarish a + b
}

butun n1 = yigindi(5, 3)           // 8
butun n2 = yigindi(10, 20)        // 30
butun n3 = yigindi(n1, n2)        // 38 (8 + 30)
```

Parametrlar , (vergul) bilan ajratiladi. Har bir parametrning turi alohida yozilishi kerak — yigindi(butun a, b) deb yozib bo'lmaydi.

5.3.3 Turli xil parametrlar

```
matn formatlash(matn ism, butun yosh, mantiq talaba) {
  matn natija = ism + " (" + std::to_string(yosh) + " yosh)"
  agar (talaba) {
    natija += ", talaba"
  }
  qaytarish natija
}

yozish << formatlash("Aziza", 22, rost)
// Natija: Aziza (22 yosh), talaba
```

5.4 Qaytarish (return)

qaytarish kalit so'zi orqali funksiya o'z natijasini chaqiruvchiga qaytaradi.

```
butun kub(butun x) {  
    qaytarish x * x * x      // qaytar va funksiyadan chiq  
}
```

5.4.1 qaytarish darhol funksiyadan chiqaradi

```
butun absolyut(butun x) {  
    agar (x >= 0) {  
        qaytarish x          // bu yerda chiqamiz  
    }  
    qaytarish -x             // bu satr faqat x < 0 bo'lsa bajariladi  
}
```

5.4.2 Bir nechta qaytarish

```
matn baho_nomi(butun ball) {  
    agar (ball >= 90) qaytarish "A'lo"  
    agar (ball >= 70) qaytarish "Yaxshi"  
    agar (ball >= 60) qaytarish "Qoniqarli"  
    qaytarish "Qoniqarsiz"  
}
```

Birinchi qaytarish bajarilishi bilanoq, qolganlari ishlamaydi.

5.4.3 Qaytarmaslik — bosh

Agar funksiya qiymat qaytarmasa, qaytarish turi **bosh** (C++ da `void`) bo'ladi:

```
bosh chiqarish(matn xabar) {  
    yozish << xabar << qator_oxiri  
    // qaytarish yo'q  
}
```

bosh funksiyalarda qaytarish ni yozish shart emas, lekin yozish mumkin (faqat qiymatsiz):

```
bosh tekshirish(butun x) {  
    agar (x < 0) {  
        yozish << "Xato!" << qator_oxiri  
        qaytarish          // funksiyadan darhol chiqamiz  
    }  
    yozish << "OK" << qator_oxiri  
}
```

5.5 Parametr berish usullari

Bu juda muhim mavzu. `uz++` (va `C++`) da parametrlarni funksiyaga **uch xil usulda** berish mumkin.

5.5.1 1. Qiymat orqali (by value)

Eng oddiy va xavfsiz usul. Funksiyaga *nusxa* uzatiladi:

```
bosh oshiring(butun x) {
    x = x + 100
    yozish << "Funksiyada: " << x << qator_oxiri
}

butun asosiy() {
    butun a = 5
    oshiring(a)
    yozish << "Asosiyda: " << a << qator_oxiri
}
```

Natija:

Funksiyada: 105

Asosiyda: 5

`a` o'zgarmadi! Chunki `oshiring` funksiyasi `a` ning *nusxasini* oldi. Aslida ikkita alohida o'zgaruvchi bo'ldi: `asosiy` dagi `a` va `oshiring` dagi `x`.

Real hayotdagi misol: Siz qog'ozga raqam yozdingiz. Do'stingizga bu qog'ozning *photocopy* ini berdingiz. Do'stingiz photocopy ustida nimani o'zgartirsa, sizning asl qog'ozingiz buzilmaydi.

5.5.2 2. Havola orqali (by reference)

Agar siz funksiyada **asl o'zgaruvchini** o'zgartirmoqchi bo'lsangiz, parametr nomidan oldin `&` qo'ying:

```
bosh oshiring(butun& x) {           // & belgisiga e'tibor!
    x = x + 100
}

butun asosiy() {
    butun a = 5
    oshiring(a)
    yozish << a << qator_oxiri    // 105
}
```

`&` — “asl narsani ber” demakdir. Endi `oshiring` ichidagi `x` — bu xuddi `a`. Ularni o'zgartirsangiz, `a` ham o'zgaradi.

5.5.3 3. Const havola orqali

Agar funksiya **katta** o'zgaruvchini olsa (matn, vektor), nusxa olish sekin. Lekin siz uni o'zgartirmoqchi emassiz. Bu holatda o'zgarmas ishlatilg:

```
butun matn_uzunligi(o'zgarmas matn& s) {      // havola, lekin o'zgartirib bo'lmaydi
    qaytarish s.length()
}

matn katta_matn = "Juda uzun matn..."
butun uzunlik = matn_uzunligi(katta_matn)
```

Bu eng tez va xavfsiz usul.

5.5.4 Qaysi usulni qachon ishlatish kerak?

Holat	Tavsiya
Kichik tur (butun, kasr, mantiq)	Qiymat orqali (butun x)
Funksiya parametrni o'zgartirishi kerak	Havola orqali (butun& x)
Katta tur, o'zgartirmaslik kerak	Const havola (o'zgarmas matn& s)

5.6 Standart parametr qiymatlari

Parametrga **standart qiymat** belgilash mumkin:

```
bosh salom_ber(matn ism = "Mehmon") {
    yozish << "Salom, " << ism << "!" << qator_oxiri
}

salom_ber()                // "Salom, Mehmon!"
salom_ber("Aziza")         // "Salom, Aziza!"
```

Standart qiymatli parametrlar **oxirida** bo'lishi kerak:

```
bosh f(butun a, butun b = 0, butun c = 0)    // TO'G'RI
bosh f(butun a = 0, butun b, butun c)       // XATO
```

5.7 Funksiya overloading (qayta yuklash)

Bir xil nomli funksiyaning **bir nechta versiyasi** bo'lishi mumkin — agar ularning parametrlari farqli bo'lsa:

```

butun yigindi(butun a, butun b) {
    qaytarish a + b
}

ikkilangan yigindi(ikkilangan a, ikkilangan b) {
    qaytarish a + b
}

butun yigindi(butun a, butun b, butun c) {
    qaytarish a + b + c
}

yigindi(5, 3)           // 1-versiya: butun
yigindi(2.5, 3.7)       // 2-versiya: ikkilangan
yigindi(1, 2, 3)        // 3-versiya: 3 ta parametr

```

Kompilyator parametrlarga qarab qaysi versiyani chaqirishni aniqlaydi.

5.8 Rekursiya — funksiya o'zini chaqiradi

Rekursiya — bu funksiyaning o'zini chaqirishi. Ko'pincha matematik muammolarni yechish uchun foydali.

5.8.1 Klassik misol: Faktoriyal

$n! = 1 \times 2 \times 3 \times \dots \times n$. Rekursiv ta'rif: $n! = n \times (n-1)!$.

```

butun faktoriyal(butun n) {
    agar (n <= 1) {
        qaytarish 1           // bazaviy holat
    }
    qaytarish n * faktoriyal(n - 1) // rekursiya
}

```

Qanday ishlaydi? - faktoriyal(5) → 5 * faktoriyal(4) - faktoriyal(4) → 4 * faktoriyal(3) - faktoriyal(3) → 3 * faktoriyal(2) - faktoriyal(2) → 2 * faktoriyal(1) - faktoriyal(1) → 1 (bazaviy) - Qaytib: 2*1=2, 3*2=6, 4*6=24, 5*24=120

5.8.2 Rekursiyaning ikki shart

Har bir rekursiv funksiya kerak:

1. **Bazaviy holat** — to'xtash sharti. Agar bu bo'lmasa — cheksiz rekursiya, dastur "stack overflow" bilan ishdan chiqadi.
2. **Rekursiv chaqiruv** — o'zini chaqirish, lekin boshqacha parametr bilan.

5.8.3 Yana bir misol: Fibonachchi

```
butun fibonachchi (butun n) {  
    agar (n <= 1) {  
        qaytarish n          // F(0) = 0, F(1) = 1  
    }  
    qaytarish fibonachchi(n-1) + fibonachchi(n-2)  
}
```

***Diqqat** — rekursiya sekin bo'lishi mumkin. Yuqoridagi fibonachchi (40) ni hisoblash juda uzoq vaqt oladi, chunki bir xil qiymatlar qayta-qayta hisoblanadi. Bunday hollarda **memoizatsiya** yoki **iterativ** yechim ishlatiladi.*

5.9 Lambda ifodalar

Lambda — bu **nomsiz funksiya**. Qachon kerak? Faqat bir joyda ishlatib, keyin tashlamoqchi bo'lganda.

5.9.1 Sintaksis

```
[ qo'lg'a olish ] ( parametrlar ) -> qaytarish_turi { tanasi }
```

5.9.2 Misol

```
o'zgaruvchan kvadrat = [] (butun x) { qaytarish x * x; }  
yozish << kvadrat(5)          // 25
```

5.9.3 Foydali misol — to'plamlarda tartiblash

```
vektor<butun> sonlar = {3, 1, 4, 1, 5, 9, 2, 6}  
  
// Pasayish bo'yicha tartiblash  
std::sort(sonlar.begin(), sonlar.end(), [] (butun a, butun b) {  
    qaytarish a > b  
})
```

Bu yerda lambda — taqqoslash funksiyasi sifatida `sort` ga uzatildi.

5.9.4 O'rab olish (capture)

Lambda tashqi o'zgaruvchilarni "qo'lga olishi" mumkin:

```
butun ko'paytma = 3
o'zgaruvchan ko'p = [ko'paytma](butun x) { qaytarish x * ko'paytma; }

yozish << ko'p(5)           // 15
```

[ko'paytma] — "tashqi ko'paytma ni qiymat orqali qo'lga ol" demakdir.

Lambda haqida 6-bobda batafsil ko'ramiz.

5.10 Inline funksiyalar

qator_ichi (C++ da inline) — funksiyani chaqirish o'rniga, uning tanasini chaqirish joyiga kiritib qo'yish kompilyatorga tavsiya:

```
qator_ichi butun maks(butun a, butun b) {
    qaytarish (a > b) ? a : b
}
```

Bu nima beradi? Tez, chunki funksiya chaqirish overhead'i yo'q. Lekin kompyuterning ishi ko'payadi va kompilatsiya sekinlashadi.

Qoida: Faqat juda kichik va tez funksiyalar uchun qator_ichi ishlatilg.

5.11 Funksiya shablonlari

Agar siz yigindi ni faqat butun uchun emas, har xil tur uchun yozmoqchi bo'lsangiz, **shablon** ishlatasiz:

```
shablon<turdash T>
T yigindi(T a, T b) {
    qaytarish a + b
}

yigindi(5, 3)           // butun
yigindi(2.5, 3.7)       // ikkilangan
yigindi("Salom ", "dunyo") // matn
```

Shablonlarni 9-bobda batafsil ko'ramiz.

5.12 Amaliy misol: Matematik funksiyalar to'plami

Mana, bir nechta matematik funksiyalarni birlashtirgan dastur:

```
// matematika.uzpp

butun yigindi(butun a, butun b) {
    qaytarish a + b
}

butun ko'paytma(butun a, butun b) {
    qaytarish a * b
}

butun maks(butun a, butun b) {
    qaytarish (a > b) ? a : b
}

butun min(butun a, butun b) {
    qaytarish (a < b) ? a : b
}

butun absolyut(butun x) {
    qaytarish (x >= 0) ? x : -x
}

butun faktoriyal(butun n) {
    agar (n <= 1) qaytarish 1
    qaytarish n * faktoriyal(n - 1)
}

 mantiq tubmi(butun n) {
    agar (n < 2) qaytarish yolg'on
    uchun (butun i = 2; i * i <= n; i++) {
        agar (n % i == 0) qaytarish yolg'on
    }
    qaytarish rost
}

butun asosiy() {
    yozish << "5 + 3 = " << yigindi(5, 3) << qator_oxiri
    yozish << "5 * 3 = " << ko'paytma(5, 3) << qator_oxiri
    yozish << "maks(5, 3) = " << maks(5, 3) << qator_oxiri
    yozish << "|-7| = " << absolyut(-7) << qator_oxiri
    yozish << "5! = " << faktoriyal(5) << qator_oxiri
    yozish << "7 tub? " << (tubmi(7) ? "ha" : "yo'q") << qator_oxiri

    qaytarish 0
}
```

Natija:

5 + 3 = 8


```
5 * 3 = 15
maks(5, 3) = 5
|-7| = 7
5! = 120
7 tub? ha
```

Har bir funksiya **bir aniq vazifa** bajaradi. asosiy faqat ularni chaqiradi. Bu — *yaxshi dasturlash* namunasi.

5.13 Eng ko'p uchraydigan xatolar

5.13.1 1. Qaytarmaslik

```
butun yigindi(butun a, butun b) {
    butun c = a + b
    // qaytarish yo'q!
}
// ← XATO: butun qaytarmadi
```

5.13.2 2. Noto'g'ri tur

```
butun yigindi(butun a, butun b) {
    qaytarish "natija" // ← XATO: matn qaytarmoqda
}
```

5.13.3 3. Parametr berish unutilgan

```
butun yigindi(butun a, butun b) {
    qaytarish a + b
}

yigindi(5) // ← XATO: ikkita parametr kerak
```

5.13.4 4. Cheksiz rekursiya

```
butun fibonachchi(butun n) {
    qaytarish fibonachchi(n-1) + fibonachchi(n-2)
    // bazaviy holat yo'q!
}
```

5.13.5 5. Funksiya nomini kalit so'z bilan bir xil qilish

```
butun agar(butun x) { ... } // ← XATO: agar - kalit so'z
```

5.14 Bob bo'yicha mashqlar

5.14.1 Mashq 1: Doira yuzasi (oson)

`doira_yuzasi(ikkilangan r)` funksiyasini yozing, u doira yuzasini qaytaradi.

5.14.2 Mashq 2: Eng katta (oson)

`maks_uch(butun a, butun b, butun c)` — uchta sondan eng kattasini qaytaring.

5.14.3 Mashq 3: Sonni teskari aylantirish (o'rta)

`teskari(butun n)` — sonni teskari aylantiring (123 → 321).

5.14.4 Mashq 4: Sonning raqamlari yig'indisi (o'rta)

`raqam_yigindisi(butun n)` — sonning barcha raqamlari yig'indisini qaytaring (rekursiv).

5.14.5 Mashq 5: Eng katta umumiy bo'luvchi (o'rtacha)

`gcd(butun a, butun b)` — Euclid algoritmi bilan. Rekursiv yozing.

```
gcd(a, b) = (b == 0) ? a : gcd(b, a % b)
```

5.14.6 Mashq 6: Quvvatga ko'tarish (o'rtacha)

`quvvat(ikkilangan asos, butun daraja)` — $asos^{daraja}$. Rekursiv versiya yozing.

5.14.7 Mashq 7: Stringni teskari aylantirish (qiyinroq)

`teskari_matn(matn s)` — matnni teskari aylantiring (“salom” → “molas”).

5.15 Xulosa

- **Funksiya** — bir necha marta ishlatish uchun nomlangan kod bloki.
- **Sintaksis:** `qaytarish_turi nom(parametrlar) { tanasi }`.
- **Parametr berish usullari:** qiymat orqali, havola orqali, `const` havola orqali.
- **qaytarish** — funksiyadan qiymat qaytarish va chiqish.
- **Standart parametrlar** — kerak bo'lmasa berilmasa, standart qiymat ishlatiladi.
- **Funksiya overloading** — bir xil nomli, har xil parametrli funksiyalar.
- **Rekursiya** — funksiya o'zini chaqiradi. Bazaviy holat shart.
- **Lambda** — nomsiz funksiya.
- **Shablonlar** — bitta funksiya, ko'p tur uchun (9-bobda).

Keyingi bobda biz **ko'rsatkichlar va havolalar** ni o'rganamiz — bu C++ ning eng kuchli va eng murakkab xususiyatlaridan biri. Ular orqali siz xotirani to'g'ridan-to'g'ri boshqarishni o'rganasiz.

06 · Ko'rsatkichlar va havolalar

6.1 Bu bobning maqsadi

Ko'rsatkichlar (pointers) — C++ (va shu sababli uz++) ning eng kuchli, shu bilan birga eng qiyin tushunarli xususiyatidan biri. Ko'p boshlovchilar bu yerda to'xtab qoladi. Bu bobni o'tib bo'lganingizdan keyin, siz quyidagilarga ega bo'lasiz:

- Xotira qanday tashkil etilganini chuqur tushunish
- Funktsiyalarni samarali yozish ko'nikmasi (havlolar bilan)
- Dinamik xotirani boshqarish — kerakli paytda olib, kerakli paytda qaytarish
- Klassik dasturlash strukturalarini yaratish (linked list, daraxt)

Boshlanglar — bu boshlanish, oxiri emas. Sabr-toqat bilan!

6.2 Xotira tashkil etilishi

Avval xotirani umumiy ko'rib chiqaylik. Sizning kompyuteringizdagi xotira (RAM) — bu juda uzun lenta. Har bir katakda 1 bayt (8 bit) saqlanadi. Har bir katakning **manzili** (address) bor — bu son.

Manzil:	0x1000	0x1001	0x1002	0x1003	0x1004	...
Mazmun:	72	0	0	0	'A'	...
	↑				↑	
	butun (4 bayt)			belgi (1 bayt)		

Yuqorida: - 0x1000 — 0x1003 manzillarida butun 72 saqlanadi (butun 4 bayt). - 0x1004 da 'A' belgisi (belgi 1 bayt).

Manzillar **o'n oltilik** (hexadecimal) sistemada yoziladi — shuning uchun 0x prefiks bor. Bu shunchaki katta sonlarni qisqaroq yozish usuli.

Endi savol: agar siz **o'zgaruvchining manzilini bilsangiz**, unga to'g'ridan-to'g'ri kira olasizmi? Javob — **ha**. Bu — **ko'rsatkich** deganda nazarda tutilgan narsa.

6.3 Havola (Reference) — sodda boshlash

Havolaga (&) biz funksiyalar bobida (5-bob) qisqacha teginan edik. Endi batafsil ko'rib chiqaylik.

Havola — bu o'zgaruvchining yangi nomi (psevdonimi). Tasavvur qiling, sizning ismingiz “Akmal”, lekin do'stlaringiz sizni “Aki” deyishadi. Aki = Akmal. Ikkala nom — *bir xil odam*.

```
butun yosh = 25
butun& yosh_psevdonim = yosh

yosh_psevdonim = 30           // bu yosh ni ham o'zgartiradi!
yozish << yosh                // 30
yozish << yosh_psevdonim      // 30 (bir xil narsa!)
```

6.3.1 Havola qoidalari

1. Havola e'lon paytida shart initsializatsiya qilinishi kerak:

```
butun& y           // ← XATO: nimaga havola?
butun& y = yosh    // TO'G'RI
```

2. Havola boshqa o'zgaruvchiga “ko'chirilmaydi”:

```
butun a = 5, b = 10
butun& r = a
r = b           // Bu - a ga b ning qiymatini saqlaydi (a = 10)
                // r endi b ga ko'rsatadi DEYIL
```

3. **null** havola yo'q — havola doim biror narsani ko'rsatadi.

6.3.2 Havolalar nima uchun foydali?

1. Funksiyalarda parametr o'zgartirish:

```
bosh almashtirish(butun& a, butun& b) {
    butun temp = a
    a = b
    b = temp
}

butun asosiy() {
    butun x = 5, y = 10
    almashtirish(x, y)
    yozish << x << " " << y    // 10 5
}
```

& siz: almashtirish(x, y) ichida a va b — x va y ning nusxalari. Ularni o'zgartirsangiz, asl o'zgaruvchilarga ta'sir qilmaydi. & bilan — bu xuddi bir xil o'zgaruvchilar.

2. Katta o'zgaruvchilarni samarali uzatish:

```
butun matn_uzunligi(o'zgarmas matn& s) {
    qaytarish s.length()
}
```

o'zgarmas matn& — havola (tez), lekin o'zgartirib bo'lmaydi (xavfsiz). Eng yaxshi parametr berish usuli.

6.4 Ko'rsatkich (Pointer)

Endi keling, ko'rsatkichga o'tamiz. **Ko'rsatkich** — bu boshqa o'zgaruvchining **manzilini** saqlovchi o'zgaruvchi.

6.4.1 Ko'rsatkichni e'lon qilish

```
butun a = 42
butun* p = &a           // p - a ning manzilini saqlaydi
```

Tahlil: - butun* — “butun turidagi qiymatga ko'rsatuvchi ko'rsatkich” - &a — “a ning manzili” (e'tibor: bu & boshqa kontekstda — havola emas, **manzilni olish** operatori) - p — endi a ning manziliga ko'rsatadi

6.4.2 * (dereference) — qiymatga kirish

Ko'rsatkichdan qiymatga kirish uchun * ishlatamiz:

```
butun a = 42
butun* p = &a

yozish << p           // a ning manzili (qandaydir son)
yozish << *p           // a ning qiymati (42)

*p = 100               // a ga 100 ni saqlash
yozish << a            // 100
```

*p — “p ko'rsatadigan joydagi qiymat” demakdir.

6.4.3 Visualization

Xotira:

```
|-----|
```

Manzil	Mazmun	
0x1000	42	← a
0x2000	0x1000	← p (a ning manzili)

```
p      = 0x1000  (a ning manzili)
*p     = 42      (manzildagi qiymat)
&a     = 0x1000  (a ning manzili)
&p     = 0x2000  (p ning manzili)
```

6.5 nullptr — bo'sh ko'rsatkich

`nullptr` — bu **hech narsaga ko'rsatmaslik** qiymati. Yangi ko'rsatkich yaratayotganda, qiymat berishni unutmang:

```
butun* p = nullptr           // bo'sh ko'rsatkich
```

`nullptr` ni * qilish — **xato!** Dastur ishdan chiqadi:

```
butun* p = nullptr
*p = 5           // ← XATO: dastur "crash" qiladi
```

Doim tekshiring:

```
agar (p != nullptr) {
    *p = 5
}
```

6.6 Dinamik xotira: yangi va o'chirish

Hozirgacha biz e'lon qilgan barcha o'zgaruvchilar **stack** xotirasida yashardi. Stack — bu funksiya tugashi bilan tozalanadigan vaqtinchalik xotira.

Lekin ba'zan bizga **dinamik xotira** kerak — funksiya tugagandan keyin ham yashaydigan. Bu **heap** xotirasi.

6.6.1 yangi (new) — heap'dan xotira olish


```
butun* p = yangi butun(42)    // heap'da yangi butun, qiymati 42
yozish << *p                // 42
```

yangi butun(42) — “heap'da yangi butun yarat, qiymati 42 bo'lsin, manzilini qaytar”.

6.6.2 o'chirish (delete) — xotirani qaytarish

```
butun* p = yangi butun(42)
*p = 100
o'chirish p                // xotirani qaytarish (free)
p = nullptr                // havsiz ko'rsatkich qoldirma
```

Muhim qoida: Har bir yangi ga bitta o'chirish mos kelishi kerak. Aks holda — xotira sizmasi (memory leak)!

6.6.3 Massivlar uchun

```
butun* sonlar = yangi butun[10]    // 10 ta butun
sonlar[0] = 1
sonlar[1] = 2
// ...

o'chirish[] sonlar                // diqqat - [] qo'shing!
```

Real hayotdagi misol: Stack — restoranning stoli (taom keldi, yedingiz, ketdingiz — stol tozalanadi). Heap — sizning oshxonangiz (siz nima qo'ysangiz, o'zingiz tozalashingiz kerak). Agar siz qo'ydingiz va unutsangiz — bardoshli iflos joy hosil bo'ladi.

6.6.4 Memory leak — eng yomon kelishuv

```
bosh funksiya() {
    butun* p = yangi butun(42)
    // p ni `o'chirish` qilmadik!
}

// Bu funksiya har chaqirilganda 4 bayt xotira "yo'qoladi"
// Agar 1 milliard marta chaqirsangiz - 4 GB yo'qolgan!
```

6.7 Aqlli ko'rsatkichlar (Smart Pointers)

yangi/o'chirish xato manbai. Zamonaviy C++ (C++11+) bu masalani hal qiladi: **aqlli ko'rsatkichlar**.

6.7.1 yagona_korsatkich (unique_ptr)

Avtomatik tozalanadigan ko'rsatkich. Faqat bitta egasi bor:

```
#include <memory>

butun asosiy() {
    yagona_korsatkich<butun> p = std::make_unique<butun>(42)
    yozish << *p << qator_oxiri    // 42

    // funksiya tugagach, p avtomatik tozalanadi
    qaytarish 0
}
```

o'chirish ni yozish kerak emas! Tozalash avtomatik.

6.7.2 umumiy_korsatkich (shared_ptr)

Bir necha egasi bo'lishi mumkin. Oxirgi egasi yo'qolganda — xotira tozalanadi:

```
umumiy_korsatkich<butun> p1 = std::make_shared<butun>(42)
umumiy_korsatkich<butun> p2 = p1    // ikkala egasi
yozish << *p1 << " " << *p2    // 42 42

// p1 va p2 ikkalasi yo'qolgach - xotira tozalanadi
```

Tavsiya: Yangi kodda har doim aqlli ko'rsatkichlar ishlating, oddiy yangi/o'chirish emas. Bu siz uchun katta foyda — xato kamayadi.

6.8 Ko'rsatkich arifmetikasi

Massiv elementlariga * orqali kirish mumkin:

```
butun massiv[5] = {10, 20, 30, 40, 50}
butun* p = massiv    // birinchi elementga ko'rsatadi

yozish << *p    // 10
p++    // keyingi elementga o'tdi
yozish << *p    // 20
p += 2    // 2 ta oldinga
yozish << *p    // 40
```

p++ faqat 1 oshmaydi — u **butun hajmiga** (4 bayt) oshadi! Bu nima uchun ishlaydi: kompilyator ko'rsatkich turidan kelib chiqib, qanchalik siljitish kerakligini biladi.

Ehtiyot bo'ling: Ko'rsatkich arifmetikasi xavfli — chegaradan tashqari joyga o'tib qolishingiz mumkin (out-of-bounds). Buni vektor kabi konteynerlar (7-bob) hal qiladi.

6.9 Ko'rsatkich va const

Uchta xil “const ko'rsatkich” bor:

```
// 1. O'zgarmas qiymatga ko'rsatadigan
o'zgarmas butun* p1 = &a
// *p1 = 10    // XATO: qiymatni o'zgartirib bo'lmaydi
// p1 = &b      // OK: p1 ni boshqa joyga qaratish mumkin

// 2. O'zgarmas ko'rsatkich
butun* o'zgarmas p2 = &a
// *p2 = 10    // OK
// p2 = &b      // XATO

// 3. Ikkala
o'zgarmas butun* o'zgarmas p3 = &a
// *p3 = 10    // XATO
// p3 = &b      // XATO
```

Bu narsa yangi boshlovchilar uchun chalkash, lekin amaliyotda foydali. Kompilyator sizning niyatingizni tushunadi va xatolardan saqlaydi.

6.10 Amaliy misol: Bog'langan ro'yxat (Linked List)

Ko'rsatkichlarning kuchli qo'llanish maydoni — **bog'langan ro'yxat**. Massivdan farqi: elementlar xotirada ketma-ket emas, har biri keyingisining manzilini bilib turadi.

```
tuzilma Tugun {
    butun qiymat
    Tugun* keyingi
}

butun asosiy() {
    // Uchta tugun yaratamiz
    Tugun* birinchi = yangi Tugun{10, nullptr}
    Tugun* ikkinchi = yangi Tugun{20, nullptr}
    Tugun* uchinchi = yangi Tugun{30, nullptr}

    // Bog'laymiz: 10 → 20 → 30
    birinchi->keyingi = ikkinchi
```

```

    ikkinchi->keyingi = uchinchi

    // Bo'ylab yuramiz
    Tugun* joriy = birinchi
    davr (joriy != nullptr) {
        yozish << joriy->qiymat << " "
        joriy = joriy->keyingi
    }
    yozish << qator_oxiri           // 10 20 30

    // Tozalash
    o'chirish birinchi
    o'chirish ikkinchi
    o'chirish uchinchi

    qaytarish 0
}

```

E'tibor bering: `joriy->qiymat = (*joriy).qiymat` ning qisqartmasi.

Real hayotdagi misol: Tuxum sotuvchining ro'yxati. Har bir mijozning kartochkasida quyidagi mijoz nomi yozilgan. Birinchi kartochkadan boshlab, har bir kartochkadan "keyingi" maydonini kuzatib borasiz va oxiri (`nullptr`) ga yetganingizda to'xtaysiz.

Bog'langan ro'yxat afzalliklari va kamchiliklari:

Afzallik	Kamchilik
Element qo'shish/o'chirish tez	Element qidirish sekin
Hajm o'zgaruvchan	Xotira sarflanishi ko'p (har bir tugun uchun)
Xotira bo'sh joylariga moslashadi	CPU cache uchun yomon

6.11 Eng ko'p uchraydigan xatolar

6.11.1 1. `nullptr` ni dereference qilish

```

butun* p = nullptr
*p = 5 // ← XATO: dastur crash

```

6.11.2 2. O'chirilgan ko'rsatkichni ishlatish

```

butun* p = yangi_butun(42)
o'chirish p
*p = 5 // ← XATO: dangling pointer

```

6.11.3 3. Bir xil joyni ikki marta o'chirish

```
butun* p = yangi_butun(42)
o'chirish p
o'chirish p // ← XATO: double-free
```

6.11.4 4. Massivni o'chirish (without [])

```
butun* arr = yangi_butun[10]
o'chirish arr // ← XATO: yo'qolgan
o'chirish[] arr // TO'G'RI
```

6.11.5 5. Memory leak

```
butun* p = yangi_butun(42)
// o'chirish ni unutdik!
```

Hammaning yechimi: Aqli ko'rsatkichlardan foydalaning.

6.12 Bob bo'yicha mashqlar

6.12.1 Mashq 1: Ikkita o'zgaruvchini almashtirish (oson)

`almashtirish_havola(butun& a, butun& b)` va `almashtirish_korsatkich(butun* a, butun* b)` — ikki versiyani yozing.

6.12.2 Mashq 2: Massivning yig'indisi (o'rta)

`yigindi(butun* arr, butun n)` — massiv elementlarining yig'indisini qaytaring. Ko'rsatkich arifmetikasini ishlating.

6.12.3 Mashq 3: Eng katta element (o'rta)

`maks(butun* arr, butun n)` — massivdagi eng kattasini toping.

6.12.4 Mashq 4: String uzunligi (o'rtacha)

`uzunlik(o'zgarmas_belgi* s)` — Cstring uzunligini hisoblang ('\\0' gacha).

6.12.5 Mashq 5: Massivni teskari aylantirish (qiyinroq)

`teskari(butun* arr, butun n)` — massiv elementlarini joyida (in-place) teskari aylantiring.

6.12.6 Mashq 6: Linked list (qiyin)

Bog'langan ro'yxatga element qo'shish va olib tashlash funksiyalarini yozing.

6.13 Xulosa

- **Xotira** — manzilli baytlar to'plami.
- **Havola (&)** — o'zgaruvchining boshqa nomi. Sodda va xavfsiz.
- **Ko'rsatkich (*)** — o'zgaruvchining manzilini saqlaydi.
- **& (manzil olish)** — o'zgaruvchining manzilini qaytaradi.
- ***** (**dereference**) — ko'rsatkichdagi qiymatga kiradi.
- **nullptr** — bo'sh ko'rsatkich.
- **yangi/o'chirish** — dinamik xotira (heap).
- **Aqlli ko'rsatkichlar** — avtomatik tozalanadigan zamonaviy yondashuv.
- **Memory leak** — yangi qilib o'chirish unutish.
- **Linked list** — ko'rsatkichlar bilan tuzilgan dinamik struktura.

Keyingi bobda biz **massivlar va to'plamlar** ni o'rganamiz — ma'lumotlarni guruhlash va saqlashning eng samarali usullarini ko'rib chiqamiz.

07 · Massivlar va to'plamlar

7.1 Nima uchun to'plamlar kerak?

Tasavvur qiling, siz maktab darajachisiz va sizga 30 ta talaba bahosini saqlash kerak. 30 ta o'zgaruvchini alohida e'lon qilasizmi?

```
butun baho_1 = 85
butun baho_2 = 92
butun baho_3 = 78
// ... 27 ta yana
butun baho_30 = 88
```

Bu juda noqulay. Eng kichik vazifani bajarish — masalan, **o'rtacha bahoni hisoblash** — 30 qator kod talab qiladi. Yangi talaba qo'shish — yangi o'zgaruvchi yozish, hamma joyda yangilash.

Yaxshi yechim — to'plam. Bitta o'zgaruvchida 30 ta (yoki 30 ming) qiymatni saqlash. `uz++` da to'plamlarning ko'p turlari bor, har biri o'zining maqsadi uchun. Bu bobda eng asosiylarini ko'rib chiqamiz:

- **Massiv** (`array`) — qat'iy o'lchamli to'plam
- **Vektor** (`vektor`) — dinamik o'lchamli (o'sib boradigan) massiv
- **Tartiblangan xarita** (`map`) — kalit-qiymat juftlari
- **To'plam** (`set`) — takrorlanmas elementlar
- **Juftlik va uchlik** (`pair, tuple`) — qisqacha guruhlash
- **Ixtiyoriy** (`optional`) — bo'lishi yoki bo'lmasligi mumkin qiymat

7.2 Massivlar (Statik)

Eng oddiy to'plam. **Qat'iy uzunlikdagi** ketma-ket joylashgan elementlar.

7.2.1 E'lon qilish

```
butun baholar[5] = {85, 92, 78, 90, 88}
```

Tahlil: - butun — element turi - baholar — to'plam nomi - [5] — uzunligi (5 ta element) - { ... } — boshlang'ich qiymatlar

7.2.2 Elementga kirish — indeks orqali

```
yozish << baholar[0]           // 85 (birinchi)
yozish << baholar[2]           // 78 (uchinchi)
yozish << baholar[4]           // 88 (oxirgi)
```

Diqqat! Indeks 0 dan boshlanadi, $n-1$ da tugaydi (n — uzunlik).

```
baholar:    [85] [92] [78] [90] [88]
indeks:      0   1   2   3   4
```

7.2.3 Element o'zgartirish

```
baholar[2] = 80           // uchinchi bahoni o'zgartirdik
yozish << baholar[2]       // 80
```

7.2.4 Bo'ylab yurish

```
uchun (butun i = 0; i < 5; i++) {
    yozish << baholar[i] << " "
}
```

Yoki **range-based for**:

```
uchun (butun baho : baholar) {
    yozish << baho << " "
}
```

Ikkinchi variant **soddaroq va xavfsizroq** — indeks haqida o'ylash kerak emas.

7.2.5 Chegaradan tashqari kirish — XATO


```
butun baholar[5]
baholar[10] = 100           // ← XATO: out of bounds!
```

Kompilyator ba'zan buni ushlamaydi, lekin dastur ishga tushganda kutilmagan natija beradi yoki ishdan chiqadi. C++ da bu chegarani **siz** tekshirishingiz kerak.

7.2.6 Massiv hajmi

```
butun baholar[5]
butun n = sizeof(baholar) / sizeof(baholar[0])    // 5
```

Bu — `total_size / element_size`. Lekin bu mexanizm noqulay. Zamonaviy yo'l: `std::array`:

```
#include <array>
std::array<butun, 5> baholar = {85, 92, 78, 90, 88}
yozish << baholar.size()      // 5 - sodda!
```

7.3 Vektor (vektor) — dinamik massiv

vektor (C++ da `std::vector`) — bu o'sib bora oladigan massiv. Hajm oldindan ma'lum bo'lmasa, vektordan foydalaning.

7.3.1 E'lon qilish

```
#include <vector>

vektor<butun> sonlar           // bo'sh
vektor<butun> baholar = {85, 92, 78, 90, 88} // qiymat bilan
vektor<butun> nollar(10, 0)    // 10 ta nol
```

7.3.2 Element qo'shish — `.push_back()`

```
vektor<butun> sonlar
sonlar.push_back(10)
sonlar.push_back(20)
sonlar.push_back(30)
// sonlar = {10, 20, 30}
```

`push_back` — vektorning **oxiriga** element qo'shadi.

7.3.3 Element o'chirish — `.pop_back()`

```
sonlar.pop_back()           // oxirgini olib tashlaydi
```

7.3.4 Element soni — `.size()`

```
yozish << sonlar.size()     // 2 (yuqorisidan keyin)
```

7.3.5 Bo'sh ekanligini tekshirish

```
agar (sonlar.empty()) {  
    yozish << "Bo'sh"  
}
```

7.3.6 Elementga kirish

```
sonlar[0]           // tezroq, lekin chegaradan tashqari xavfli  
sonlar.at(0)        // sekinroq, lekin chegaradan tashqari exception
```

`.at()` — agar chegaradan tashqari bo'lsa, **xatolik ko'taradi** (crash o'rniga). Xavfsizroq.

7.3.7 Bo'ylab yurish

```
// Indeks bilan  
uchun (butun i = 0; i < sonlar.size(); i++) {  
    yozish << sonlar[i] << " "  
}  
  
// Range-based for (afzal)  
uchun (butun son : sonlar) {  
    yozish << son << " "  
}  
  
// Havola bilan (o'zgartirish uchun)  
uchun (butun& son : sonlar) {  
    son *= 2           // har birini ikkilantir  
}
```

7.3.8 Vektor tozalash

```
sonlar.clear() // hamma elementlarni olib tashlaydi
```

7.3.9 Misol: Ortacha baho

```
vektor<butun> baholar = {85, 92, 78, 90, 88, 95}

butun yigindi = 0
uchun (butun b : baholar) {
    yigindi += b
}
ikkilangan ortacha = ikkilangan(yigindi) / baholar.size()
yozish << "O'rtacha: " << ortacha << qator_oxiri
```

7.4 `std::map` — tartiblangan xarita

`map` — bu **kalit-qiymat** juftlari to'plami. Tartibga keltirilgan.

7.4.1 Misol — talaba bahosi xaritasi

```
#include <map>

std::map<matn, butun> baholar
baholar["Aziza"] = 92
baholar["Bobur"] = 85
baholar["Karim"] = 78

yozish << baholar["Aziza"] // 92
```

Bu yerda: - Kalit: `matn` (ism) - Qiymat: `butun` (baho)

7.4.2 Element bor yoki yo'qligini tekshirish

```
agar (baholar.count("Aziza") > 0) {
    yozish << "Aziza topildi"
}

// Yoki - find orqali
agar (baholar.find("Aziza") != baholar.end()) {
    yozish << "Topildi"
}
```

Diqqat! `baholar["Dilshoda"]` — agar “Dilshoda” yo‘q bo‘lsa, **avtomatik yaratadi** (qiymati 0 bilan)! Buni faqat o‘qish uchun ishlatmang.

7.4.3 Bo‘ylab yurish

```
uchun (o'zgarmas avto& juft : baholar) {
    yozish << juft.first << ": " << juft.second << qator_oxiri
}
```

Natija (tartiblangan):

```
Aziza: 92
Bobur: 85
Karim: 78
```

Map elementlari **kalit bo‘yicha** avtomatik tartibga keltiriladi.

7.4.4 Element o‘chirish

```
baholar.erase("Bobur")
```

7.5 `std::set` — takrorlanmas to‘plam

`set` — bu **takrorsiz** elementlar to‘plami. Tartiblangan.

```
#include <set>

std::set<butun> raqamlar
raqamlar.insert(5)
raqamlar.insert(3)
raqamlar.insert(5)           // takror - qo'shi olmaydi
raqamlar.insert(7)

yozish << raqamlar.size()     // 3 (5, 3, 7)
```

7.5.1 Foydali amallar

```
agar (raqamlar.count(5) > 0) {
    yozish << "5 bor"
}

raqamlar.erase(3)           // 3 ni o'chirish
```

7.5.2 Misol: noyob so'zlarni sanash

```
vektor<matn> matnlar = {"olma", "anor", "olma", "anor", "uzum"}
std::set<matn> noyob(matnlar.begin(), matnlar.end())

yozish << "Noyob so'zlar soni: " << noyob.size()    // 3
```

7.6 Boshqa ketma-ketlik to'plamlari

7.6.1 `std::deque` (juft tomonlama navbat)

Vektor kabi, lekin **ikkala tomondan** ham qo'shish/olib tashlash tez:

```
#include <deque>
std::deque<butun> dq
dq.push_back(1)
dq.push_front(0)           // boshiga qo'shish
// dq = {0, 1}
```

7.6.2 `std::list` (bog'langan ro'yxat)

Element qo'shish/olib tashlash o'rtada ham tez. Lekin indeks bilan kirish sekin.

7.6.3 Qaysi birini qachon ishlatish?

Holat	To'plam
Hajm aniq, kichik	array
Hajm o'zgaruvchan, oxiriga qo'shish	vektor
Ikkala tomondan qo'shish	deque
O'rtada ko'p qo'shish/olib tashlash	list
Kalit-qiymat	map
Takrorsiz qiymatlar	set

7.7 std::pair — juftlik

Ikki qiymatni birga saqlash uchun:

```
std::pair<matn, butun> talaba = {"Aziza", 92}
yozish << talaba.first      // "Aziza"
yozish << talaba.second     // 92
```

7.7.1 Foydali misol

```
vektor<std::pair<matn, butun>> baholar = {
    {"Aziza", 92},
    {"Bobur", 85},
    {"Karim", 78}
}

uchun (o'zgarmas avto& p : baholar) {
    yozish << p.first << ": " << p.second << qator_oxiri
}
```

pair map dan farqli — tartiblanmagan va kalit qaytaruvi mumkin (takror).

7.8 std::tuple — uchlik va undan ko'p

Uchta yoki undan ko'p qiymat birga:

```
std::tuple<matn, butun, ikkilangan> talaba = {"Aziza", 22, 4.7}
yozish << std::get<0>(talaba)      // "Aziza"
yozish << std::get<1>(talaba)      // 22
yozish << std::get<2>(talaba)      // 4.7
```

Yoki structured binding:

```
o'zgarmas avto [ism, yosh, baho] = talaba
yozish << ism << " " << yosh << " " << baho
```

7.9 std::optional — bo'lishi mumkin

Ba'zan qiymat bo'lmasligi mumkin. Masalan, qidirish funksiyasi:

```

std::optional<butun> qidirish(vektor<butun> sonlar, butun maqsad) {
    uchun (butun i = 0; i < sonlar.size(); i++) {
        agar (sonlar[i] == maqsad) {
            qaytarish i // topildi
        }
    }
    qaytarish std::nullopt // topilmadi
}

butun asosiy() {
    vektor<butun> v = {10, 20, 30}
    avto natija = qidirish(v, 20)

    agar (natija) {
        yozish << "Topildi, indeks: " << *natija
    }
    aks_holda {
        yozish << "Topilmadi"
    }
}

```

`std::optional` — modern C++ ning eng foydali xususiyatlaridan biri. “qiymat bor yoki yo’q” holatini tabiiy ko’rsatish uchun.

7.10 Amaliy misol: Talabalar jurnali

```

#include <vector>
#include <map>

tuzilma Talaba {
    matn ism
    butun yoshi
    vektor<butun> baholar
}

butun asosiy() {
    Talaba t1 = {"Aziza", 22, {85, 92, 78}}
    Talaba t2 = {"Bobur", 21, {90, 88, 95}}
    Talaba t3 = {"Karim", 23, {65, 70, 75}}

    vektor<Talaba> guruh = {t1, t2, t3}

    uchun (o'zgarmas Talaba& t : guruh) {
        butun yigindi = 0
        uchun (butun b : t.baholar) yigindi += b
        ikkilangan ortacha = ikkilangan(yigindi) / t.baholar.size()
    }
}

```

```
        yozish << t.ism << " (" << t.yoshi << " yosh): "  
            << "o'rtacha = " << ortacha << qator_oxiri  
    }  
  
    qaytarish 0  
}
```

Natija:

```
Aziza (22 yosh): o'rtacha = 85  
Bobur (21 yosh): o'rtacha = 91  
Karim (23 yosh): o'rtacha = 70
```

Bu — real dasturning kichik fragmenti. Kollej, ma'lumotlar bazasi, talaba boshqaruv tizimi.

7.11 Eng ko'p uchraydigan xatolar

7.11.1 1. Chegaradan tashqari kirish

```
vektor<butun> v = {1, 2, 3}  
v[5] = 10 // ← XATO yoki crash
```

7.11.2 2. Bo'sh vektordan element olish

```
vektor<butun> v  
yozish << v[0] // ← XATO
```

Tekshiring: agar `(!v.empty())`.

7.11.3 3. Map elementiga kirib unga avtomatik element qo'shish

```
std::map<matn, butun> m  
agar (m["yangi"] == 0) { // ← "yangi" ni yaratdi (0 qiymat bilan)!  
    // ...  
}  
// To'g'risi: m.count("yangi") yoki m.find()
```

7.11.4 4. Iteratorni o'chirib davom etish


```
vektor<butun> v = {1, 2, 3, 4, 5}
uchun (avto it = v.begin(); it != v.end(); it++) {
    agar (*it == 3) {
        v.erase(it)           // ← XATO: it endi tutashmagan
    }
}
```

To'g'risi:

```
uchun (avto it = v.begin(); it != v.end(); ) {
    agar (*it == 3) {
        it = v.erase(it)      // erase yangi iterator qaytaradi
    }
    aks_holda {
        it++
    }
}
```

7.11.5 5. Vektor nusxasini funksiyaga uzatish

```
butun yigindi(vektor<butun> v) {           // nusxa - sekin!
    // ...
}

butun yigindi(o'zgarmas vektor<butun>& v) { // havola - tez
    // ...
}
```

7.12 Bob bo'yicha mashqlar

7.12.1 Mashq 1: Massivning yig'indisi va o'rtachasi (oson)

Foydalanuvchidan 10 ta son so'rang. Yig'indisi va o'rtachasini chiqaring.

7.12.2 Mashq 2: Eng katta va kichik (oson)

Vektor ichida eng katta va eng kichik elementlarni toping.

7.12.3 Mashq 3: Sonlarni teskari tartibda chiqarish (o'rta)

Foydalanuvchidan vektor olin va uni teskari tartibda chiqaring.

7.12.4 Mashq 4: Takrorlarni topish (o'rta)

Vektor berilgan — undagi takror keladigan sonlarni topib chiqaring. Maslahat: `map` ishlatib, har bir sonni necha marta ko'rganini sanang.

7.12.5 Mashq 5: So'zlar chastotasi (o'rtacha)

Matn berilgan. Har bir so'z necha marta uchrayotganini sanang va chiqaring.

7.12.6 Mashq 6: Tartiblash (qiyinroq)

Vektor sonlarini ostidan kattagacha tartiblang. `std::sort` ishlatmasdan — bubble sort yoki insertion sort algoritmini o'zingiz yozing.

7.12.7 Mashq 7: To'plamlar amallar (qiyin)

Ikkita `set` berilgan. Ularning birlashishi (union), kesishishi (intersection) va farqi (difference) ni hisoblang.

7.13 Xulosa

- **Massiv** — qat'iy uzunlikdagi to'plam.
- **Vektor** — dinamik o'sadigan massiv. Eng ko'p ishlatiladigan.
- **Map** — kalit-qiymat juftlari, tartiblangan.
- **Set** — takrorsiz to'plam.
- **Pair, tuple** — bir necha qiymatni guruhlash.
- **Optional** — qiymat bo'lishi yoki bo'lmasligi mumkin.
- **Bo'ylab yurish** — `uchun` orqali, indeks yoki range-based.
- **Eng ko'p xato** — chegaradan tashqari kirish.

Keyingi bobda biz **sinflar (klasslar)** va **obyektga yo'naltirilgan dasturlash (OOP)** ni o'rganamiz. Bu — zamonaviy dasturlashning eng muhim asoslaridan biri.

08 · Sinflar — Obyektga Yo'naltirilgan Dasturlash

8.1 OOP nima va nima uchun kerak?

Hozirgacha siz dasturlarni **funksiya-tomonidan** o'rganib keldingiz — funksiyalar va o'zgaruvchilarni alohida-alohida tashkil etib. Lekin katta dasturlarda bu yondashuv yetarsiz. **Obyektga yo'naltirilgan dasturlash** (Object-Oriented Programming, OOP) — kodingizni **obyektlar** atrofida tashkil qilish usuli.

8.1.1 Real hayotdan misol

Tasavvur qiling, siz mashina haqida gapirayapsiz. Mashinaning **xususiyatlari** bor (rang, model, yili, dvigatel hajmi) va **harakatlari** (yurish, to'xtash, signal berish, eshikni ochish). *Mashina* — bu shu xususiyat va harakatlarni birgalikda saqlovchi **obyekt**.

Mashinaning **chizmasi** (sxemasi) — bu **sinf**. Sxemadan minglab mashinalar yaratish mumkin, har biri o'zining shaxsiy xususiyatlari bilan, lekin barchasi bir xil sxemaga binoan.

```
[Mashina sinfi]           ← chizma
    ↓ yaratamiz
[Mashina 1] [Mashina 2] [Mashina 3] ← obyektlar
    rangi=qora   rangi=oq   rangi=qizil
    yili=2020    yili=2022   yili=2024
```

8.1.2 OOP afzalliklari

1. **Inkapsulyatsiya** — ma'lumot va kod birga, ichki tafsilotlar yashirin.
2. **Meros** — bir sinfdan boshqasini “merosga olish” — kodni qayta ishlatish.
3. **Polimorfizm** — bitta nom, ko'p versiya — kod moslashuvchan.
4. **Abstraktsiya** — qanday ishlashini bilmasdan ishlatish (mashina ichini bilmaysiz, lekin haydaysiz).

8.2 Sinf e'lon qilish

```
sinf Talaba {  
  ochiq:  
    matn ism  
    butun yoshi  
    ikkilangan ortacha_baho  
  
  bosh malumotni_chiqarish() {  
    yozish << "Ism: " << ism << qator_oxiri  
    yozish << "Yoshi: " << yoshi << qator_oxiri  
    yozish << "O'rtacha: " << ortacha_baho << qator_oxiri  
  }  
}
```

Tahlil: - `sinf Talaba` — `sinf e'loni` - `ochiq:` — bu qismni hammaga ko'rinadi qiladi (default yopiq) - `matn ism`, `butun yoshi` — bu sinfning **maydonlari** (fields) - `malumotni_chiqarish()` — bu sinfning **metodi** (member function)

8.2.1 Obyekt yaratish va ishlatish

```
butun asosiy() {  
  Talaba t1  
  t1.ism = "Aziza"  
  t1.yoshi = 22  
  t1.ortacha_baho = 4.7  
  
  t1.malumotni_chiqarish()  
  
  Talaba t2  
  t2.ism = "Bobur"  
  t2.yoshi = 21  
  t2.ortacha_baho = 4.2  
  
  t2.malumotni_chiqarish()  
  
  qaytarish 0  
}
```

Natija:

```
Ism: Aziza  
Yoshi: 22  
O'rtacha: 4.7  
Ism: Bobur  
Yoshi: 21  
O'rtacha: 4.2
```

`t1` va `t2` — ikkita alohida `Talaba` obyekti. Har biri o'zining ma'lumotlari bilan.

8.2.2 Nuqta operatori

. (nuqta) operatori — obyektning maydoni yoki metodiga kirish uchun:

```
t1.ism                // maydon
t1.malumotni_chiqarish() // metod
```

8.3 Inkapsulyatsiya — yopiq va ochiq

Klassda ikki xil ko'rinish darajasi:

- **ochiq:** (public) — tashqaridan ko'rinadi
- **yopiq:** (private) — faqat sinf ichidan ko'rinadi

8.3.1 Misol

```
sinf BankHisobi {
yopiq:
    butun balans = 0

ochiq:
    bosh pul_qoyish(butun miqdor) {
        agar (miqdor > 0) {
            balans += miqdor
        }
    }

    bosh pul_olish(butun miqdor) {
        agar (miqdor > 0 && miqdor <= balans) {
            balans -= miqdor
        }
        aks_holda {
            yozish << "Yetarli mablag' yo'q!" << qator_oxiri
        }
    }

    butun joriy_balans() {
        qaytarish balans
    }
}
```

Endi:

```
BankHisobi hisob
hisob.pul_qoyish(1000)
hisob.pul_olish(500)
yozish << hisob.joriy_balans()    // 500

hisob.balans = 1000000           // ← XATO: yopiq
```

Foydalanuvchi balans ga to'g'ridan-to'g'ri kira olmaydi. Faqat biz bergan metodlar orqali. Bu — inkapsulyatsiya.

8.3.2 Nima uchun bu muhim?

Tasavvur qiling, bank dasturida balans ochiq bo'lsa, har qanday kod uni o'zgartira oladi:

```
hisob.balans = -1000           // manfiy balans?
hisob.balans = 999999999       // oddiy foydalanuvchida million?
```

yopiq bilan — biz qoidalarini nazorat qilamiz. Faqat pul_qoyish va pul_olish metodlari orqali balans o'zgaradi.

***Real hayotdagi misol:** ATM bankomat. Siz pulni to'g'ridan-to'g'ri hisobingizdan olib chiqolmay-siz — siz tugmachalarni bosasiz, bankomat qoidalarini tekshiradi (parol, miqdor, balans), keyin pul beradi. Bu — inkapsulyatsiya.*

8.4 Konstruktor va destruktor

8.4.1 Konstruktor

Konstruktor — obyekt yaratilganida avtomatik chaqiriladigan maxsus metod. Maydonlarni boshlang'ich qiymat bilan to'ldirish uchun.

```
sinf Talaba {
  ochiq:
    matn ism
    butun yoshi

    // Konstruktor - sinf nomi bilan bir xil, qaytarish turi yo'q
    Talaba(matn yangi_ism, butun yangi_yoshi) {
      ism = yangi_ism
      yoshi = yangi_yoshi
    }
}

butun asosiy() {
  Talaba tl("Aziza", 22)           // konstruktor chaqirildi
}
```

```
Talaba t2("Bobur", 21)

yozish << t1.ism << " " << t1.yoshi
}
```

Konstruktor — sinf nomi bilan bir xil. Qaytarish turi yoʻq.

8.4.2 Standart konstruktor

Agar konstruktorni yozmasangiz, kompilyator avtomatik yaratadi (parametrsiz). Lekin agar siz biror konstruktor yozsangiz, standartini ham siz yozishingiz kerak:

```
sinf Talaba {
ochiq:
    Talaba() { // bo'sh, default
        ism = ""
        yoshi = 0
    }

    Talaba(matn yangi_ism, butun yangi_yoshi) { // parametrli
        ism = yangi_ism
        yoshi = yangi_yoshi
    }
}

Talaba t1 // standart konstruktor
Talaba t2("Aziza", 22) // parametrli
```

8.4.3 Initialization list

Maydonlarni qiymat bilan ishga tushirish samaraliroq usul:

```
sinf Talaba {
ochiq:
    matn ism
    butun yoshi

    Talaba(matn yangi_ism, butun yangi_yoshi)
        : ism(yangi_ism), yoshi(yangi_yoshi)
    { }
}
```

: ism(yangi_ism), yoshi(yangi_yoshi) — bu **initialization list**. U maydonlarni qiymat bilan eʼlon paytida boshlash imkonini beradi.

8.4.4 Destruktor

Destruktor — obyekt yoʻq qilinganida avtomatik chaqiriladi. Tozalash uchun:

```
sinf Fayl {
yopiq:
    matn nom

ochiq:
    Fayl(matn n) : nom(n) {
        yozish << "Faylni ochildi: " << nom << qator_oxiri
    }

    ~Fayl() { // destruktör - ~ bilan
        yozish << "Faylni yopildi: " << nom << qator_oxiri
    }
}

butun asosiy() {
    Fayl f("ma'lumot.txt")
    // ... ish bilan ...
} // bu yerda destruktör chaqiriladi
```

Natija:

```
Faylni ochildi: ma'lumot.txt
Faylni yopildi: ma'lumot.txt
```

Destruktör xotira, fayl, tarmoq aloqasi kabi resurslarni tozalash uchun ishlatiladi.

8.5 Meros (Inheritance)

Meros — bir sinfdan boshqasini “merosga olish”. Yangi sinf — ota-sinfning hamma xususiyatlarini oladi va o'zini qo'shadi.

8.5.1 Misol — hayvonlar

```
sinf Hayvon {
ochiq:
    matn ism
    butun yoshi

    bosh nafas_olish() {
        yozish << ism << " nafas oladi" << qator_oxiri
    }
}

sinf It : ochiq Hayvon { // : ochiq - meros olish
ochiq:
```



```

    bosh hurish() {
        yozish << ism << " huriydi: Vov-vov!" << qator_oxiri
    }
}

butun asosiy() {
    It mening_itim
    mening_itim.ism = "Rex"
    mening_itim.nafas_olish()           // Hayvondan meros
    mening_itim.hurish()                // It ning o'zi
}

```

Natija:

```

Rex nafas oladi
Rex huriydi: Vov-vov!

```

It Hayvon ning hamma ochiq xususiyatlarini oldi.

8.5.2 Konstruktor va meros

```

sinf Hayvon {
    ochiq:
        matn ism
        Hayvon(matn n) : ism(n) {}
}

sinf It : ochiq Hayvon {
    ochiq:
        It(matn n) : Hayvon(n) {           // ota konstruktorini chaqirish
            // ...
        }
}

```

8.5.3 Meros nima uchun foydali?

Bir nechta yaqin xil obyektlar bo'lsa — umumiy qismini ota-sinfga chiqaring:

```

sinf Transport {
    ochiq:
        butun tezlik
        bosh yurish() { ... }
}

sinf Mashina : ochiq Transport { ... }
sinf Motosikl : ochiq Transport { ... }
sinf Velosiped : ochiq Transport { ... }

```

Hammasi tezlik va yurish ni `Transport` dan oladi. Faqat o'ziga xos qismini yozadi.

8.6 Virtual metodlar va polimorfizm

8.6.1 Polimorfizm muammosi

```
sinf Hayvon {
ochiq:
    bosh ovoz() {
        yozish << "Qandaydir ovoz" << qator_oxiri
    }
}

sinf It : ochiq Hayvon {
ochiq:
    bosh ovoz() { // override
        yozish << "Vov-vov!" << qator_oxiri
    }
}

butun asosiy() {
    It mening_itim
    Hayvon* h = &mening_itim
    h->ovoz() // "Qandaydir ovoz" - XATO!
}
```

Biz `It` ning ovozi xohlardik, lekin `Hayvon` ovozi chiqdi. Buni **virtual metod** hal qiladi:

```
sinf Hayvon {
ochiq:
    virtual bosh ovoz() { // virtual qiling!
        yozish << "Qandaydir ovoz" << qator_oxiri
    }
}

sinf It : ochiq Hayvon {
ochiq:
    bosh ovoz() ustidan_yozish { // override deb belgilang
        yozish << "Vov-vov!" << qator_oxiri
    }
}

butun asosiy() {
    It mening_itim
    Hayvon* h = &mening_itim
    h->ovoz() // "Vov-vov!" - TO'G'RI
}
```

8.6.2 Bu nima uchun kuchli?

```
vektor<Hayvon*> hayvonlar
hayvonlar.push_back(yangi It())
hayvonlar.push_back(yangi Mushuk())
hayvonlar.push_back(yangi Ot())

uchun (Hayvon* h : hayvonlar) {
    h->ovoz()                // har biri o'zinikini aytadi!
}
```

Natija:

Vov-vov!
Miyov-miyov!
Pir-pir!

Bitta sikl — uchta turli ovoz. Bu — **polimorfizm**.

8.7 Abstrakt sinf

Ba'zan ota-sinfdan obyekt yaratish ma'nosiz bo'ladi. Masalan, `Hayvon` — bu abstrakt tushuncha. Aslida `It`, `Mushuk` bor, “umumiy hayvon” yo'q.

Pure virtual funksiya orqali sinfni abstrakt qilamiz:

```
sinf Hayvon {
ochiq:
    virtual bosh ovoz() = 0        // = 0 - pure virtual
}

butun asosiy() {
    Hayvon h                      // ← XATO: abstrakt sinfdan obyekt yaratib
    ↪ bo'lmaydi
    It mening_itim                // TO'G'RI
}
```

Pure virtual funktsiyani **albatta** voris sinfda implement qilish kerak.

8.8 Operatorlarni qayta yuklash (Operator Overloading)

Sinflar uchun operatorlarni o'z xohlagancha aniqlay olasiz:

```
sinf Vektor2D {
ochiq:
    ikkilangan x, y

    Vektor2D(ikkilangan a, ikkilangan b) : x(a), y(b) {}

    // + operatorini qayta yuklaymiz
    Vektor2D operator+(o'zgarmas Vektor2D& boshqa) {
        qaytarish Vektor2D(x + boshqa.x, y + boshqa.y)
    }
}

butun asosiy() {
    Vektor2D a(1.0, 2.0)
    Vektor2D b(3.0, 4.0)
    Vektor2D c = a + b                // qayta yuklash ishlaydi!
    yozish << c.x << " " << c.y    // 4.0 5.0
}
```

Bu — matematik tushunchalarni dasturlashda tabiiy ifodalash imkoni.

8.9 static a'zolar

Sinf bo'yicha umumiy a'zo (har bir obyektga emas, sinfnig o'zida):

```
sinf Talaba {
ochiq:
    static butun jami_talaba        // umumiy
    matn ism

    Talaba(matn n) : ism(n) {
        jami_talaba++
    }
}

butun Talaba::jami_talaba = 0        // initsializatsiya

butun asosiy() {
    Talaba t1("Aziza")
    Talaba t2("Bobur")
    Talaba t3("Karim")

    yozish << Talaba::jami_talaba    // 3
}
```

jami_talaba — bu obyektga emas, sinfga tegishli.

8.10 dost (friend) deklaratsiyasi

dost — boshqa funksiya yoki sinfga yopiq a'zolarga kirish ruxsati beradi:

```

sinf Hisob {
yopiq:
    butun balans

    dost bosh balansni_chiqarish(Hisob& h)
}

bosh balansni_chiqarish(Hisob& h) {
    yozish << h.balans                // yopiq ga kira oladi!
}

```

Ehtiyot bo'lib ishlating — inkapsulyatsiyani buzadi.

8.11 Amaliy misol: Bank hisobi tizimi

```

sinf Hisob {
yopiq:
    matn egasi
    butun raqami
    ikkilangan balans

ochiq:
    Hisob(matn ism, butun n) : egasi(ism), raqami(n), balans(0) {}

    bosh qoyish(ikkilangan miqdor) {
        agar (miqdor > 0) {
            balans += miqdor
            yozish << miqdor << " so'm qo'yildi. "
                << "Yangi balans: " << balans << qator_oxiri
        }
    }

    mantiq olish(ikkilangan miqdor) {
        agar (miqdor > 0 && miqdor <= balans) {
            balans -= miqdor
            yozish << miqdor << " so'm olindi. "
                << "Yangi balans: " << balans << qator_oxiri
            qaytarish rost
        }
        yozish << "Yetarli mablag' yo'q!" << qator_oxiri
        qaytarish yolg'on
    }
}

```

```
ikkilangan joriy_balans() o'zgarmas {
    qaytarish balans
}

bosh malumot() o'zgarmas {
    yozish << "Hisob #" << raqami << " (egasi: " << egasi << ")"
    << ", balans: " << balans << qator_oxiri
}

butun asosiy() {
    Hisob h("Aziza Karimova", 12345)
    h.qoyish(1000000)
    h.olish(250000)
    h.olish(2000000)           // mablag' yetmaydi
    h.malumot()

    qaytarish 0
}
```

Natija:

```
1000000 so'm qo'yildi. Yangi balans: 1000000
250000 so'm olindi. Yangi balans: 750000
Yetarli mablag' yo'q!
Hisob #12345 (egasi: Aziza Karimova), balans: 750000
```

8.12 Eng ko'p uchraydigan xatolar

8.12.1 1. Yopiq a'zolarga to'g'ridan-to'g'ri kirishga harakat

```
sinf X { yopiq: butun a }
X x
x.a = 5           // ← XATO
```

8.12.2 2. Konstruktorda qaytarish qilish

```
sinf X {
    ochiq:
    X() {
        qaytarish 5           // ← XATO: konstruktor qaytarmaydi
    }
}
```

8.12.3 3. Virtual destruktorni unutish

```

sinf Ota { ochiq: ~Ota() {} } // virtual emas
sinf Voris : ochiq Ota { ... }

Ota* p = yangi Voris()
o'chirish p // ← XATO: Voris destruktori chaqirilmaydi

```

To'g'risi: `virtual ~Ota() {}`.

8.12.4 4. Slicing

```

Voris v
Ota o = v // Ota qismi nusxalandi, Voris qismi yo'qoldi!

```

Polimorfizm uchun ko'rsatkich yoki havola ishlating:

```

Ota& o = v // havola - slicing yo'q

```

8.12.5 5. Memory leak — destruktorda o'chirish qilmaslik

```

sinf X {
  ochiq:
    butun* p
    X() { p = yangi butun(0) }
    // destruktorda p ni o'chirish kerak!
    ~X() { o'chirish p }
}

```

8.13 Bob bo'yicha mashqlar

8.13.1 Mashq 1: Aylana sinfi (oson)

Aylana sinfi yarating. Maydonlari: `radius`. Metodlari: `yuza()`, `uzunlik()`.

8.13.2 Mashq 2: Kalkulyator sinfi (o'rta)

Kalkulyator sinfini yozing. Metodlari: `qoshish`, `ayirish`, `koPaytirish`, `bolish`. Saqlanadigan maydon — joriy natija.

8.13.3 Mashq 3: Talaba va guruh (o'rta)

Talaba sinfi (ism, baholar vektori, o'rtacha). Guruh sinfi — `vektor<Talaba>`. Guruh metodlari: eng yaxshi talabani topish, o'rtacha bahoni hisoblash.

8.13.4 Mashq 4: Shakl sinflari ierarxiyasi (o'rtacha)

Abstrakt `Shakl` (`pure virtual` `yuza()`). Voris sinflar: `Aylana`, `Toqrtburchak`, `Uchburchak`. Polimorfizm bilan ishlatiladi.

8.13.5 Mashq 5: Vektor2D bilan operatorlar (qiyinroq)

`Vektor2D` ga `+`, `-`, `*` (skalyar bilan ko'paytirish) operatorlarini qayta yuklang.

8.13.6 Mashq 6: Bog'langan ro'yxat — sinf sifatida (qiyin)

`BoglanganRoxat` sinfini yozing. Metodlari: `qoshish`, `ochirish`, `chiqarish`, `uzunlik`.

8.14 Xulosa

- **Sinf** — bu chizma. **Obyekt** — bu chizma asosida yaratilgan narsa.
- **ochiq va yopiq** — ko'rinish nazorati (inkapsulyatsiya).
- **Konstruktor** — obyektни yaratish va boshlash.
- **Destruktor** — obyektни yo'qotish va tozalash.
- **Meros** — bir sinfdan boshqasini "merosga olish".
- **Virtual va polimorfizm** — bir nom, ko'p versiya.
- **Abstrakt sinf** — obyekt yaratib bo'lmaydigan, faqat meros uchun bo'lgan sinf.
- **Operator overloading** — `+`, `-`, va boshqa operatorlarni o'z sinfingiz uchun aniqlash.
- **static a'zolar** — sinfning o'zi uchun, obyekt uchun emas.

Keyingi bobda biz **shablonlar** ni o'rganamiz — `vektor<butun>` va `vektor<matn>` qanday bir kod bilan ishlay olishi qiziq emasmi? Bularning hammasi shablonlar yordamida.

09 · Shablonlar — Umumlashgan dasturlash

9.1 Muammo: bir xil kod, har xil tur

Tasavvur qiling, sizga eng kattasini topadigan funksiya kerak:

```
butun maks(butun a, butun b) {
    qaytarish (a > b) ? a : b
}
```

Yaxshi! Lekin endi sizga ikkilangan lar uchun ham kerak bo'ldi. Yana yozasiz:

```
ikkilangan maks(ikkilangan a, ikkilangan b) {
    qaytarish (a > b) ? a : b
}
```

Endi `matn` uchun ham — yana takrorlaysiz. `kasr` uchun ham — yana. **Bir xil kod, faqat tur farqli.** Bu — yomon. Agar logikani o'zgartirish kerak bo'lsa — barchasini o'zgartirish kerak.

Yechim: shablonlar. Bir marta yozing, kompilyator har bir tur uchun avtomatik versiyani yaratadi.

```
shablon<turdash T>
T maks(T a, T b) {
    qaytarish (a > b) ? a : b
}

butun x = maks(5, 3)           // butun versiya
ikkilangan y = maks(2.5, 3.7) // ikkilangan versiya
matn z = maks("alma", "anor") // matn versiya
```

Bitta funksiya — har xil turlarda ishlaydi!

9.1.1 Bu nima — sehrmi?

Yo'q, sehr emas. **Kompilyator** har bir chaqirish uchun **maxsus versiyani avtomatik yaratadi**. Yuqoridagi kod kompilyatsiyaga shunday o'tadi:

```
butun maks(butun a, butun b)          { qaytarish (a > b) ? a : b }
ikkilangan maks(ikkilangan a, ikkilangan b) { qaytarish (a > b) ? a : b }
matn maks(matn a, matn b)             { qaytarish (a > b) ? a : b }
```

Bu — **kompilyatsiya vaqtidagi kod generatsiyasi**. Demak, shablonlar ish jarayonida sekinlashtirmaydi — har bir versiya alohida tezkor.

9.2 Funksiya shablonlari

9.2.1 Sintaksis

```
shablon<turdash T>
T funksiya_nomi(T parametr) {
    // ... T turida ishlash
}
```

T — bu **shablon parametri**. Bu istalgan tur o'rnida turishi mumkin: butun, matn, Talaba, va h.k.

Nima uchun “T”? Bu shunchaki traditsiya. “Type” so'zining birinchi harfi. Siz U, V, Element, Tur deb nomlashingiz mumkin.

9.2.2 Bir nechta shablon parametri

```
shablon<turdash T, turdash U>
bosh chiqarish(T a, U b) {
    yozish << a << " " << b << qator_oxiri
}

chiqarish(5, "salom")           // T=butun, U=matn
chiqarish(3.14, rost)           // T=ikkilangan, U=mantiq
```

9.2.3 Aniq chaqirish

Ba'zan kompilyator turini aniqlay olmaydi — bu holda aniq aytishingiz mumkin:

```
shablon<turdash T>
T defolt() {
    qaytarish T()           // T ning default qiymati
}

butun a = defolt<butun>()   // 0
```

```

ikkilangan b = defolt<ikkilangan>() // 0.0
matn c = defolt<matn>() // ""

```

9.2.4 Misol: vektorning yig'indisi

```

shablon<turdash T>
T vektor_yigindi(o'zgarmas vektor<T>& v) {
    T natija = T() // 0 yoki ""
    uchun (o'zgarmas T& element : v) {
        natija += element
    }
    qaytarish natija
}

vektor<butun> sonlar = {1, 2, 3, 4, 5}
yozish << vektor_yigindi(sonlar) // 15

vektor<matn> sozlar = {"a", "b", "c"}
yozish << vektor_yigindi(sozlar) // "abc"

```

Bitta funksiya, ikki xil ishlatish.

9.3 Sinf shablonlari

Sinflarni ham shablon qilish mumkin. Bu juda kuchli.

9.3.1 Misol — qutilar

```

shablon<turdash T>
sinf Quti {
yopiq:
    T element

ochiq:
    Quti(T qiymat) : element(qiymat) {}

    T olish() o'zgarmas {
        qaytarish element
    }

    bosh qoyish(T yangi_qiymat) {
        element = yangi_qiymat
    }
}

```

```

}

butun asosiy() {
    Quti<butun> quti1(42)
    yozish << quti1.olish()           // 42

    Quti<matn> quti2("Salom")
    yozish << quti2.olish()           // Salom

    Quti<ikkilangan> quti3(3.14)
    yozish << quti3.olish()           // 3.14
}

```

9.3.2 Standart kutubxonadagi shablonlar

Aslida `vektor<T>`, `std::map<K, V>`, `std::pair<A, B>` — barchasi shablonlar:

```

vektor<butun>           // vektor<T> ning butun versiyasi
vektor<matn>            // vektor<T> ning matn versiyasi
std::map<matn, butun>   // std::map<K, V> ning matn-butun versiyasi

```

9.4 Standart shablon kutubxonasi (STL)

C++ ning **Standart Shablon Kutubxonasi** (STL) — bu tayyor shablonlarning katta to'plami:

9.4.1 Konteynerlar

Konteyner	Tavsif
<code>vektor<T></code>	Dinamik massiv
<code>std::array<T, N></code>	Statik massiv
<code>std::list<T></code>	Bog'langan ro'yxat
<code>std::deque<T></code>	Ikki tomonlama navbat
<code>std::map<K, V></code>	Tartiblangan xarita
<code>std::set<T></code>	Tartiblangan to'plam
<code>std::unordered_map<K, V></code>	Hash xarita (tezroq)
<code>std::unordered_set<T></code>	Hash to'plam

9.4.2 Algoritmilar

```

#include <algorithm>

```

```
vektor<butun> v = {3, 1, 4, 1, 5, 9, 2, 6}

std::sort(v.begin(), v.end())           // tartiblash
std::reverse(v.begin(), v.end())        // teskari
butun yigindi = std::accumulate(v.begin(), v.end(), 0)
butun max_el = *std::max_element(v.begin(), v.end())
butun joy = std::count(v.begin(), v.end(), 1)
```

Hammasi shablon orqali, har xil tur bilan ishlay oladi.

9.5 Concepts (C++20)

Zamonaviy C++ da shablon parametrlarini cheklash mumkin. Masalan, “faqat sonli turlar” deb cheklash:

```
shablon<turdash T>
talab std::integral<T>           // T butun bo'lishi kerak
T kvadrat(T x) {
    qaytarish x * x
}

kvadrat(5)                       // OK
kvadrat(3.14)                    // ← XATO: ikkilangan butun emas
```

Concepts — qadimgi shablonlarning xatolik xabarlarini soddalashtiradi.

9.6 Shablonlar va turlash

Shablonni baʼzan sekin yoki keraksiz ishlatish mumkin. Misol:

```
shablon<turdash T>
T qoshish(T a, T b) { qaytarish a + b }
```

Bu yaxshi. Lekin agar siz `matn + butun` ni qoʻshmoqchi boʻlsangiz:

```
qoshish("Yosh: ", 25)           // ← XATO! Matn va butun bir xil tur emas
```

Yechim — ikki shablon parametri:

```
shablon<turdash T, turdash U>
avto qoshish(T a, U b) {
    qaytarish a + b
}
```

Lekin bu kontekstga qarab — turli xil amallar bo'lishi mumkin.

9.7 Amaliy misol: Generic stack

Stack — bu LIFO (Last In, First Out) ma'lumotlar tuzilmasi. Mantar qutisi kabi: oxirgi qo'yilgan birinchi olinadi.

```
shablon<turdash T>
sinf Stack {
yopiq:
    vektor<T> ma'lumotlar

ochiq:
    bosh push(T qiymat) {
        ma'lumotlar.push_back(qiymat)
    }

    T pop() {
        agar (ma'lumotlar.empty()) {
            qaytarish T()
        }
        T natija = ma'lumotlar.back()
        ma'lumotlar.pop_back()
        qaytarish natija
    }

    T top() o'zgarmas {
        qaytarish ma'lumotlar.back()
    }

    butun olchami() o'zgarmas {
        qaytarish ma'lumotlar.size()
    }

    mantiq bosh() o'zgarmas {
        qaytarish ma'lumotlar.empty()
    }
}

butun asosiy() {
    Stack<butun> s
    s.push(1)
    s.push(2)
```

```

s.push(3)
yozish << s.pop() << qator_oxiri // 3
yozish << s.pop() << qator_oxiri // 2
yozish << s.pop() << qator_oxiri // 1

Stack<matn> matnlar
matnlar.push("a")
matnlar.push("b")
yozish << matnlar.pop() // "b"
}

```

Bitta `Stack` sinfi — har qanday tur bilan ishlay oladi.

9.8 Eng ko'p uchraydigan xatolar

9.8.1 1. Shablonni `header` da ko'rsatmaslik

Funksiya shablonlari kompilyatsiya paytida yangilanadi. Shu sababli, ular **header**da to'liq yozilishi kerak — `.cpp` faylida emas.

9.8.2 2. Turlarni adashtirish

```

shablon<turdash T> T maks(T a, T b)

maks(5, 3.14) // ← XATO: 5 - butun, 3.14 - ikkilangan

```

To'g'risi: `maks<ikkilangan>(5, 3.14)` yoki aniq tur belgilash.

9.8.3 3. Cheksiz instantsiyalash

Murakkab shablonlar kompilyatsiya vaqtini juda ko'paytiradi. Sodda saqlang.

9.9 Bob bo'yicha mashqlar

9.9.1 Mashq 1: min funksiyasi (oson)

`shablon<turdash T> T min(T a, T b)` — kichikrog'ini qaytaradi.

9.9.2 Mashq 2: Vektorda qidirish (o'rta)

`shablon<turdash T> butun topish(vektor<T>& v, T maqsad)` — birinchi indeksini qaytar-ing, yoki -1 (topilmasa).

9.9.3 Mashq 3: Generic juftlik (o'rta)

`shablon<turdash A, turdash B> sinf Juftlik` — first va second maydonlari bilan.

9.9.4 Mashq 4: Navbat (o'rtacha)

`shablon<turdash T> sinf Navbat` (FIFO — birinchi kelgan birinchi ketadi).

9.9.5 Mashq 5: Binary daraxt (qiyin)

`shablon<turdash T> sinf BinaryDaraxt` — qo'shish, qidirish, inorder yurish funksiyalari bi-lan.

9.10 Xulosa

- **Shablon** — bitta kod, ko'p tur uchun.
- **Funksiya shablonlari** — `shablon<turdash T>`.
- **Sinf shablonlari** — sinfning butunlay shablon versiyasi.
- **STL** — tayyor shablonlar kutubxonasi (`vektor`, `map`, `set`, `algoritmlar`).
- **Concepts** — shablon parametrini cheklash (C++20).
- **Afzalligi** — kod qayta ishlatiladi, tezlik yo'qolmaydi.
- **Kamchiligi** — kompilyatsiya vaqti uzayadi, xato xabarlarlari murakkab.

Keyingi bobda biz **xatolik bilan ishlash** ni o'rganamiz — dasturlaringizni mustahkam qilishning eng muhim yo'li.

10 · Xatolik bilan ishlash

10.1 Xatolik — bu hayotning bir qismi

Real dasturlar **doim** xatoliklarga duch keladi. Fayl topilmaydi, internet uziladi, foydalanuvchi 0 ga bo'lishni so'raydi, disk to'lib ketadi, parol noto'g'ri kiritiladi. Yaxshi dastur — bu xatolikni **bo'lmasa kerak**, deb da'vo qiladigan dastur emas, balki xatolikni **oqilona qabul qilib**, foydalanuvchiga aniq xabar beradigan dastur.

Tasavvur qiling, bankomatda pul olmoqchisiz, lekin balans yetmaydi. Bankomat sizga **xato xabari** beradi: "Mablag' yetarli emas". Aksincha bo'lsa nima bo'lardi? Bankomat indamasdan o'chib qolardi yoki noto'g'ri miqdor berardi. **Xatoliklarni** to'g'ri ushlash — yaxshi dasturning asosi.

uz++ (va C++) da xatoliklarni boshqarishning bir nechta usuli bor:

1. **Qaytarish qiymati** — funksiya muvaffaqiyat yoki xatolikni qaytaradi.
2. **Istisno** (exception) — `urinish/ushlash` bloki bilan ushlash.
3. **Natija<T>** / `std::expected` — zamonaviy yondashuv (C++23).
4. **Tanlov<T>** / `std::optional` — qiymat bo'lishi/bo'lmasligi.
5. **tasdiqlash** (assert) — dasturchi xatolarini ushlash.

10.2 Usul 1: Qaytarish qiymati

Eng oddiy va eski usul. Funksiya muvaffaqiyat bo'lsa qiymatni, xato bo'lsa maxsus belgi qaytaradi.

```
butun bolish(butun a, butun b) {
    agar (b == 0) {
        qaytarish -1                // xato belgisi
    }
    qaytarish a / b
}

butun n = bolish(10, 0)
agar (n == -1) {
```

```
yozish << "Xato: 0 ga bo'lib bo'lmaydi" << qator_oxiri  
}
```

Muammosi: -1 — bu odatiy qiymat bo'lishi mumkin. `bolish(-5, 5)` ham -1 qaytaradi! Qanday xato va qanday natijani farqlash qiyin.

10.3 Usul 2: Istisnolar (Exceptions)

Modern C++ ning asosiy yondashuvi: funksiya xato hosil bo'lganda **istisno tashlaydi** (throw), chaqiruvchi uni **ushlaydi** (catch).

10.3.1 urinish / ushlash / irgitish bloki

```
urinish {  
    // xato bo'lishi mumkin kod  
}  
ushlash (xato_turi xato_nomi) {  
    // xato bilan ishlash  
}
```

10.3.2 Misol

```
butun bolish(butun a, butun b) {  
    agar (b == 0) {  
        irgitish std::runtime_error("0 ga bo'lib bo'lmaydi")  
    }  
    qaytarish a / b  
}  
  
butun asosiy() {  
    urinish {  
        butun n = bolish(10, 0)  
        yozish << n << qator_oxiri  
    }  
    ushlash (o'zgarmas std::runtime_error& e) {  
        yozish << "Xato yuz berdi: " << e.what() << qator_oxiri  
    }  
  
    qaytarish 0  
}
```

Natija:

Xato yuz berdi: 0 ga bo'lib bo'lmaydi

10.3.3 Qanday ishlaydi?

1. urinish bloki ichidagi kod bajariladi.
2. Agar kod ichida irgitish bo'lsa — istisno *ko'tarib chiqariladi*.
3. Mos ushlash bloki topiladi — uning kodi bajariladi.
4. Agar mos ushlash topilmasa — dastur ishdan chiqadi (terminate).

10.3.4 Bir nechta ushlash blok

```

urinish {
    // ...
}
ushlash (o'zgarmas std::out_of_range& e) {
    yozish << "Chegaradan tashqari: " << e.what()
}
ushlash (o'zgarmas std::runtime_error& e) {
    yozish << "Runtime xato: " << e.what()
}
ushlash (...) {                                // hamma boshqa xatolar
    yozish << "Noma'lum xato"
}

```

10.3.5 Standart istisnolar

Istisno	Tavsif
std::runtime_error	Umumiy runtime xato
std::logic_error	Mantiqiy xato
std::out_of_range	Chegaradan tashqari
std::invalid_argument	Noto'g'ri argument
std::bad_alloc	Xotira ajratilmadi

10.3.6 O'zingizning istisno turingiz

```

sinf MeningXatom : ochiq std::runtime_error {
    ochiq:
        MeningXatom(matn xabar) : std::runtime_error(xabar) {}
}

bosh xato_yarat() {
    irgitish MeningXatom("Maxsus xato")
}

```

10.3.7 Istisnolarni qachon ishlatish kerak?

Yaxshi holatlar: - Funksiya ish bajara olmadi (fayl ochilmadi, tarmoq yo'q) - Mantiqiy xato (noto'g'ri parametr) - Xotira tugadi

Yomon holatlar: - Oddiy oqim boshqaruvi uchun (sikldan chiqish kabi) - Tez-tez yuz beradigan vaziyatlar uchun (sekin!)

***Diqqat** — **istisno sekin amal**. Istisno tashlash va ushlash — bu sekin operatsiya. Agar siz har soniyada 1000 marta istisno tashlasangiz — dastur juda sekinlashadi. Faqat **istisnoiy vaziyatlar** uchun.*

10.4 Usul 3: Natija<T> / std::expected (C++23)

Bu — eng zamonaviy va xavfsiz yondashuv. Funksiya **qiymat** yoki **xato** qaytaradi:

```
Natija<butun, matn> bolish(butun a, butun b) {
    agar (b == 0) {
        qaytarish std::unexpected("0 ga bo'lib bo'lmaydi")
    }
    qaytarish a / b
}

butun asosiy() {
    avto natija = bolish(10, 0)

    agar (natija) { // muvaffaqiyatli?
        yozish << "Natija: " << *natija
    }
    aks_holda {
        yozish << "Xato: " << natija.error()
    }
}
```

Natija<T, E> — bu **T (qiymat)** yoki **E (xato)** qaytaradigan tur.

Afzalliklari: - Tez (istisno yo'q) - Aniq (siz xatoni unutib bo'lmaysiz — qaytarish qiymati bilan ish) - Tur bo'yicha xavfsiz

10.5 Usul 4: Tanlov<T> / std::optional

Agar qiymat **bo'lishi** yoki **bo'lmasligi** mumkin bo'lsa, lekin xato sababi muhim emas:

```

Tanlov<butun> qidirish(vektor<butun>& v, butun maqsad) {
    uchun (butun i = 0; i < v.size(); i++) {
        agar (v[i] == maqsad) {
            qaytarish i
        }
    }
    qaytarish std::nullopt // topilmadi
}

butun asosiy() {
    vektor<butun> v = {1, 2, 3, 4, 5}
    avto natija = qidirish(v, 3)

    agar (natija) {
        yozish << "Topildi, indeks: " << *natija
    }
    aks_holda {
        yozish << "Topilmadi"
    }
}

```

Tanlov — Natija ning soddalashtirilgan versiyasi.

10.6 Usul 5: tasdiqlash (assert)

tasdiqlash (C++ da assert) — bu **dasturchi xatolarini** ushlash uchun. Foydalanuvchi xatolari uchun emas!

```

#include <cassert>

butun yashir_olish(o'zgarmas vektor<butun>& v, butun i) {
    tasdiqlash(i >= 0 && i < v.size()) // sharti tekshirish
    qaytarish v[i]
}

```

Agar tasdiqlash ichidagi shart **yolg'on** bo'lsa — dastur darhol to'xtaydi va xato chiqaradi. Bu — siz hech qachon kutmagan vaziyatni ushlash uchun.

10.6.1 Release rejimida

tasdiqlash faqat **debug** rejimida ishlaydi. Release rejimida (optimizatsiya bilan) avtomatik o'chiriladi. Shu sababli:

- Foydalanuvchi xatolari uchun ishlatmang
- Faqat dasturchi shartlarini tekshirish uchun
- Asosiy mantiq uchun emas

10.7 Xatolik strategiyasini tanlash

Holat	Tavsiya
Foydalanuvchi xatosi (noto'g'ri parol)	Qaytarish qiymati yoki <code>Tanlov</code>
Fayl yo'q, tarmoq xato	<code>Natija<T, E></code> yoki istisno
Tubdan kutilmagan holat	<code>tasdiqlash</code>
Funksiya ba'zan qiymat qaytarmaydi	<code>Tanlov<T></code>
Kutilgan, lekin past darajadagi xato	<code>Natija<T, E></code>
Yangi C++ kodi	<code>Natija</code> , <code>Tanlov</code> afzal
Eski kod bilan integratsiya	Istisnolar

10.8 Amaliy misol: Konfiguratsiya faylini o'qish

```
#include <fstream>
#include <string>

Natija<matn, matn> faylni_oqish(matn yo'l) {
    std::ifstream f(yo'l)
    agar (!f.is_open()) {
        qaytarish std::unexpected("Fayl ochilmadi: " + yo'l)
    }

    matn mazmun((std::istreambuf_iterator<char>(f)),
                std::istreambuf_iterator<char>())
    qaytarish mazmun
}

butun asosiy() {
    avto natija = faylni_oqish("config.txt")

    agar (natija) {
        yozish << "Fayl mazmuni:" << qator_oxiri << *natija
    }
    aks_holda {
        yozish << "Xato: " << natija.error() << qator_oxiri
        qaytarish 1
    }

    qaytarish 0
}
```

Faqat fayl bor bo'lsa: mazmunini chiqaradi. **Yo'q bo'lsa:** xato xabarini chiqaradi va 1 kod bilan tugaydi.

10.9 Eng ko'p uchraydigan xatolar

10.9.1 1. Istisno ushlab tashlashning unutilishi

```
// urinish blokisiz, istisno dasturning yuqori darajalariga qadar
// ko'tariladi va terminate qiladi
butun n = bolish(10, 0) // crash!
```

10.9.2 2. Hamma istisnolarni catch (...) bilan yutib qo'yish

```
ushlash (...) {
    // hech narsa qilmaslik - xatolarni yashiradi
}
```

Hech bo'lmaganda log qiling yoki qayta tashlang.

10.9.3 3. Istisno xabarisiz

```
irgitish std::runtime_error("") // foydasiz
irgitish std::runtime_error("Fayl ochildi: " + yo'l) // foydali
```

10.9.4 4. Destruktorda istisno tashlash

```
struct X {
    ~X() {
        irgitish std::runtime_error("...") // ← XATO! crash
    }
}
```

Destruktorda hech qachon istisno tashlamang.

10.9.5 5. Resource leak

```
fayl_ochish()
urinish {
    // xato yuz berdi
}
fayl_yopish() // bajarilmaydi!
```

To'g'risi — RAI (Resource Acquisition Is Initialization): destruktordagi fayl yopadi.

10.10 Bob bo'yicha mashqlar

10.10.1 Mashq 1: Xavfsiz bo'lish (oson)

Natija<ikkilangan, matn> qaytaruvchi xavfsiz_bolish funksiyasi.

10.10.2 Mashq 2: Vektorda qidirish (oson)

Tanlov<butun> topish(vektor<butun>&, butun) — topilmasa nullopt.

10.10.3 Mashq 3: Matnni songa aylantirish (o'rta)

Natija<butun, matn> matn_son(matn s) — noto'g'ri formatda xato.

10.10.4 Mashq 4: Kalkulyator istisno bilan (o'rta)

Sodda kalkulyator. 0 ga bo'lish — istisno tashlasin.

10.10.5 Mashq 5: Fayl operatsiyalari (qiyinroq)

Fayl ochish, o'qish, yozish — har bir bosqichda xatolarni boshqarish.

10.11 Xulosa

- **Xatolik** — real dasturlarning oddiy hodisasi.
- **Qaytarish qiymati** — eng oddiy, lekin chalkash usul.
- **Istisnolar** (urinish/ushlash/irgitish) — kuchli, lekin sekin va murakkab.
- **Natija<T, E>** (std::expected, C++23) — zamonaviy, tez, aniq.
- **Tanlov<T>** (std::optional) — qiymat yoki yo'q.
- **tasdiqlash** — faqat dasturchi xatolarini ushlab uchun.

Keyingi bobda biz **matn va satrlar** bilan ishlashni o'rganamiz — matn dasturning eng ko'p ishlatiladigan turi. Aniq matn manipulyatsiyasi sizga juda foydali bo'ladi.

11 · Matn va satrlar bilan ishlash

11.1 Matn — eng ko'p ishlatiladigan tur

Dunyodagi har qanday dasturning katta qismi **matn bilan ishlaydi**. Foydalanuvchidan ism so'rash, sahifaning HTML kodini tahlil qilish, fayldagi matnni o'qish, ma'lumotlar bazasidan ism olish, JSON yuborish — hammasi matn.

uz++ da `matn` (`C++` da `std::string`) turi bilan ifodalanadi. Bu juda kuchli tur — uzunligi cheksiz, ko'p amallar bilan kelgan, va xotirani avtomatik boshqaradi.

```
matn ism = "Aziza"
matn salom = "Salom, " + ism + "!"
yozish << salom                // "Salom, Aziza!"
```

Bu darsda biz matn bilan ishlashning eng kerakli usullarini ko'rib chiqamiz.

11.2 Asosiy matn amallari

11.2.1 Bo'sh matn

```
matn s = ""
matn t                // ham bo'sh
```

11.2.2 Uzunlikni olish

```
matn s = "Salom"
yozish << s.length()           // 5
yozish << s.size()              // 5 (length bilan bir xil)
```

11.2.3 Belgilarga kirish

```
matn s = "Salom"
yozish << s[0]           // 'S'
yozish << s[4]           // 'm'
yozish << s.at(2)        // 'l' (xavfsizroq)
```

11.2.4 Birlashtirish (concatenation)

```
matn a = "Salom, "
matn b = "dunyo!"
matn c = a + b           // "Salom, dunyo!"

c += "?"                 // "Salom, dunyo!?"
```

11.2.5 Solishtirish

```
matn a = "alma"
matn b = "anor"

agar (a == b) { ... }    // tenglik
agar (a < b) { ... }     // alfavit bo'yicha (a < b rost)
agar (a != b) { ... }    // teng emas
```

11.2.6 Ostigi matn (substring)

```
matn s = "Salom, dunyo!"
matn s1 = s.substr(0, 5)   // "Salom"
matn s2 = s.substr(7)     // "dunyo!"
matn s3 = s.substr(7, 5)  // "dunyo"
```

`substr(boshlanish, uzunlik)` — boshlanish indeksidan uzunlik belgi.

11.2.7 Qidirish

```
matn s = "Salom, dunyo!"
butun joy = s.find("dunyo") // 7
butun yoq = s.find("yer")   // matn::npos (topilmadi)

agar (joy != matn::npos) {
    yozish << "Topildi indeks: " << joy
}
```

11.2.8 Almashtirish

```
matn s = "Salom, dunyo!"
s.replace(7, 5, "olam")
// "Salom, olam!"
```

`replace(boshlanish, uzunlik, yangi_matn).`

11.3 Matnni boshqa turlarga aylantirish

11.3.1 Son → matn

```
butun yosh = 25
matn s = std::to_string(yosh)           // "25"

ikkilangan baho = 3.14
matn t = std::to_string(baho)           // "3.140000"
```

11.3.2 Matn → son

```
matn s = "42"
butun n = std::stoi(s)                  // 42

matn t = "3.14"
ikkilangan d = std::stod(t)             // 3.14
```

`std::stoi` (string to int), `std::stod` (string to double).

Diqqat! Agar matn raqam emas — istisno tashlanadi:

```
butun n = std::stoi("abc")              // ← XATO!
```

Xavfsiz versiya:

```
urinish {
    butun n = std::stoi(s)
}
ushlash (o'zgarmas std::invalid_argument& e) {
    yozish << "Bu raqam emas"
}
```

11.4 Matnni solishtirish

11.4.1 Aniq taqqoslash

```
matn a = "Salom"
matn b = "salom"

a == b // yolg'on (S va s farqli)
```

11.4.2 Katta-kichik harfsiz taqqoslash

```
#include <algorithm>
#include <cctype>

bosh kichiklash(matn& s) {
    std::transform(s.begin(), s.end(), s.begin(),
        [](belgi c) { qaytarish std::tolower(c) });
}

matn a = "Salom", b = "SALOM"
kichiklash(a)
kichiklash(b)
agar (a == b) {
    yozish << "Bir xil"
}
```

11.5 Formatli matn (C++20/23 std::format)

std::format — bu Python ning f-string ga o'xshash kuchli vosita.

```
#include <format>

matn ism = "Aziza"
butun yosh = 22

matn s = std::format("Salom, {}! Yoshingiz {}", ism, yosh)
// "Salom, Aziza! Yoshingiz 22"
```

11.5.1 Format spetsifikatorlari

```
std::format("{:>10}", "abc")           // "      abc" (o'ng tomon)
std::format("{:<10}", "abc")           // "abc      " (chap)
std::format("{:^10}", "abc")           // "      abc      " (markaz)

std::format("{:.2f}", 3.14159)         // "3.14" (2 ta kasr)
std::format("{:6.2f}", 3.14)           // "   3.14"
std::format("{:08.3f}", 3.14)         // "0003.140"

std::format("{:x}", 255)                // "ff" (heks)
std::format("{:o}", 8)                  // "10" (oktal)
std::format("{:b}", 5)                  // "101" (binar)
```

11.5.2 Real misol

```
ikkilangan narx = 15999.50
matn s = std::format("Mahsulot narxi: {:, .2f} so'm", narx)
// "Mahsulot narxi: 15,999.50 so'm"
```

`std::cout` ning `<<` operatori bilan taqqoslang — `std::format` ancha kuchli va o'qish oson.

11.6 Son va matn orasidagi konvertatsiya

11.6.1 `std::stringstream`

Murakkab amallar uchun:

```
#include <sstream>

std::stringstream ss
ss << "Yosh: " << 22 << ", baho: " << 4.7
matn s = ss.str()           // "Yosh: 22, baho: 4.7"
```

11.6.2 Matnlardan qiymat o'qish

```
matn satr = "42 3.14 salom"
std::stringstream ss(satr)

butun n
ikkilangan d
matn s

ss >> n >> d >> s
// n = 42, d = 3.14, s = "salom"
```

11.7 UTF-8 va o'zbek alifbosi

uz++ Unicode (UTF-8) ni qo'llab-quvvatlaydi. Lekin ehtiyot bo'ling:

```
matn s = "O'zbekiston"
yozish << s.length()           // 12 (BAYT soni, harf emas!)
```

`length()` — bu **bayt** soni, harf soni emas. O'zbek harflari "O", "Sh" bir necha bayt egallaydi.

11.7.1 Harflar bilan to'g'ri ishlash

Murakkab tahlillar uchun maxsus UTF-8 kutubxonalari ishlatiladi. Sodda hollarda — `std::wstring` (keng matn):

```
#include <string>
std::wstring s = L"O'zbekiston" // L - wide
```

11.8 `std::string_view` — matn ko'rinishi

`std::string_view` — matnga **ko'rinish** beradi, nusxa olmaydi:

```
bosh chiqarish(std::string_view sv) {
    yozish << sv
}

chiqarish("Salom")           // nusxa olmaydi - tez!
chiqarish(matn("Salom, dunyo!")) // nusxa olmaydi
```

Funksiya parametrlari uchun — afzal usul (nusxa olmaslik).

11.9 Amaliy misol: Matn tahlilchisi

Foydalanuvchi yozgan jumlada so'zlar, harflar, gaplarini sanaydigan dastur:

```
matn jumla = "Bugun ob-havo ajoyib. Quyosh charaqlab turadi. Salom!"

butun harflar = 0
```

```

butun sozlar = 1
butun gaplar = 0

uchun (belgi c : jumla) {
    agar (std::isalpha(c)) {
        harflar++
    }
    agar (c == ' ') {
        sozlar++
    }
    agar (c == '.' || c == '!' || c == '?') {
        gaplar++
    }
}

yozish << "Harflar: " << harflar << qator_oxiri
yozish << "So'zlar: " << sozlar << qator_oxiri
yozish << "Gaplar: " << gaplar << qator_oxiri

```

Natija:

Harflar: 38
 So'zlar: 8
 Gaplar: 3

11.10 Eng ko'p uchraydigan xatolar

11.10.1 1. Chegaradan tashqari

```

matn s = "Salom"
yozish << s[10] // ← XATO yoki kutilmagan natija

```

11.10.2 2. UTF-8 da indeks bilan ishlash

```

matn s = "O'zbekiston"
yozish << s[1] // ← XATO: harfning bir qismi

```

11.10.3 3. std::stoi xato

```

butun n = std::stoi("abc") // ← XATO: istisno

```

11.10.4 4. `nullptr` ni matnga aylantirish

```
o'zgarmas belgi* p = nullptr
matn s = p // ← XATO: crash
```

11.10.5 5. `c_str()` ni saqlash

```
o'zgarmas belgi* p = matn("Salom").c_str()
yozish << p // ← XATO: matn yo'qolgan, p invalid
```

11.11 Bob bo'yicha mashqlar

11.11.1 Mashq 1: Palindrom (o'rta)

So'z palindrommi (orqadan o'qisangiz ham bir xil — “ada”, “kanal”).

11.11.2 Mashq 2: So'z sanash (o'rta)

Jumlada so'zlar sonini sanang.

11.11.3 Mashq 3: Bosh harf bilan boshlash (o'rta)

Har bir so'zning birinchi harfini bosh harfga aylantiring.

11.11.4 Mashq 4: Caesar shifri (qiyinroq)

Matnni N pozitsiyaga siljitib shifrlang.

11.11.5 Mashq 5: HTML belgilarni almashtirish (qiyinroq)

<, >, & — `<`, `>`, `&` ga almashtiring.

11.12 Xulosa

- `matn` (`std::string`) — asosiy matn turi.
- Asosiy amallar — `length`, `substr`, `find`, `replace`, `+`.

- **Konvertatsiya** — `std::to_string`, `std::stoi`, `std::stod`.
- **Formatlash** — `std::format` (C++20+) eng kuchli vosita.
- **UTF-8** — bayt va harf farqi muhim.
- **`string_view`** — nusxa olmaydigan ko'rinish.

Keyingi bobda biz **fayllar bilan ishlash** ni o'rganamiz — ma'lumotlarni diskka saqlash va o'qish.

12 · Fayl tizimi va JSON

12.1 Nima uchun fayllar?

Sizning dasturingiz **xotirada** ishlaydi. Lekin xotira — vaqtinchalik. Dasturni yopganingizda — ma'lumotlar yo'qoladi. Agar foydalanuvchi ma'lumotlarini, sozlamalarni, hujjatlarni saqlash kerak bo'lsa — **fayl** kerak.

Real dasturlardagi fayllar: - **Konfiguratsiya** — `config.ini`, `settings.json` - **Ma'lumotlar** — `users.csv`, `database.db` - **Hujjatlar** — `report.pdf`, `document.docx` - **Loglar** — `app.log` - **Resurslar** — `image.png`, `icon.svg`

Bu bobda `uz++` orqali fayllar bilan ishlashni o'rganamiz.

12.2 Faylga yozish

12.2.1 Asosiy yo'l: `std::ofstream`

```
#include <fstream>

butun asosiy() {
    std::ofstream fayl("salom.txt")

    agar (!fayl.is_open()) {
        yozish << "Faylni ochib bo'lmadi" << qator_oxiri
        qaytarish 1
    }

    fayl << "Salom, dunyo!" << qator_oxiri
    fayl << "Bu ikkinchi qator." << qator_oxiri
    fayl << "Yosh: " << 25 << qator_oxiri

    fayl.close() // ixtiyoriy - destruktor ham yopadi
    qaytarish 0
}
```

```
}
```

`std::ofstream` ("output file stream") — faylga yozish uchun. `<<` operatori bilan — xuddi yozish (`std::cout`) kabi.

12.2.2 Qo'shimcha rejim

```
std::ofstream fayl("log.txt", std::ios::app)    // append (qo'shish)
fayl << "Yangi qator" << qator_oxiri
```

`std::ios::app` — faylning oxiriga qo'shadi (eski mazmunni o'chirmaydi).

12.3 Fayldan o'qish

12.3.1 `std::ifstream`

```
#include <fstream>
#include <string>

butun asosiy() {
    std::ifstream fayl("salom.txt")

    agar (!fayl.is_open()) {
        yozish << "Faylni ochib bo'lmadi" << qator_oxiri
        qaytarish 1
    }

    matn qator
    davr (std::getline(fayl, qator)) {
        yozish << qator << qator_oxiri
    }

    qaytarish 0
}
```

`std::getline(fayl, qator)` — fayldan bitta qator o'qiydi.

12.3.2 Belgi-belgi o'qish

```
belgi c
davr (fayl.get(c)) {
    yozish << c
}
```

12.3.3 Soʻz-soʻz oʻqish

```
matn soz
davr (fayl >> soz) {
    yozish << soz << qator_oxiri
}
```

12.3.4 Butun faylni bir boʻlakda

```
matn mazmun((std::istreambuf_iterator<char>(fayl)),
             std::istreambuf_iterator<char>())
```

Sintaksis murakkab, lekin tez ishlaydi.

12.4 CSV bilan ishlash

CSV (Comma-Separated Values) — eng oddiy maʼlumot saqlash formati:

```
ism,yosh,baho
Aziza,22,4.7
Bobur,21,4.2
Karim,23,3.9
```

12.4.1 Oʻqish

```
std::ifstream fayl("talabalar.csv")
matn qator

// Sarlavhani o'tkazib yuborish
std::getline(fayl, qator)

davr (std::getline(fayl, qator)) {
    std::stringstream ss(qator)
    matn ism, yosh_s, baho_s
```

```
std::getline(ss, ism, ',')
std::getline(ss, yosh_s, ',')
std::getline(ss, baho_s, ',')

butun_yosh = std::stoi(yosh_s)
ikkilangan_baho = std::stod(baho_s)

yozish << ism << " " << yosh << " " << baho << qator_oxiri
}
```

12.4.2 Yozish

```
std::ofstream fayl("talabalar.csv")
fayl << "ism,yosh,baho" << qator_oxiri
fayl << "Aziza,22,4.7" << qator_oxiri
fayl << "Bobur,21,4.2" << qator_oxiri
```

12.5 JSON bilan ishlash

JSON (JavaScript Object Notation) — bugungi kunda **eng ko'p ishlatiladigan** ma'lumot formati. Veb API'lar, konfiguratsiya, saqlash — hammasi JSON.

12.5.1 JSON ko'rinishi

```
{
  "ism": "Aziza",
  "yosh": 22,
  "baholar": [85, 92, 78],
  "manzili": {
    "shahar": "Toshkent",
    "kocha": "Amir Temur"
  }
}
```

12.5.2 nlohmann/json kutubxonasi

uz++ standart kutubxonasida JSON modullari mavjud. Eng mashhur kutubxona `nlohmann/json` (yoki `uz++` ning o'z `json` moduli).

```
#include <nlohmann/json.hpp>
using json = nlohmann::json;
```

```

json talaba = {
    {"ism", "Aziza"},
    {"yosh", 22},
    {"baholar", {85, 92, 78}}
}

// String ga aylantirish
matn s = talaba.dump(4)           // 4 - chekinish
yozish << s

```

12.5.3 JSON ni faylga saqlash

```

std::ofstream f("talaba.json")
f << talaba.dump(4)

```

12.5.4 Fayldan o'qish

```

std::ifstream f("talaba.json")
json data
f >> data

matn ism = data["ism"]
butun yosh = data["yosh"]
auto baholar = data["baholar"]

```

12.5.5 Vektorni JSON ga

```

vektor<matn> mevalar = {"olma", "anor", "uzum"}
json j = mevalar
yozish << j.dump()           // ["olma","anor","uzum"]

```

12.6 Fayl yo'llari (C++17 `std::filesystem`)

C++17 dan boshlab, fayl tizimi bilan ishlash uchun maxsus kutubxona:

```

#include <filesystem>
namespace fs = std::filesystem

fs::path p = "papka/fayl.txt"

```

```
agar (fs::exists(p)) {  
    yozish << "Fayl bor"  
}  
  
agar (fs::is_directory(p)) {  
    yozish << "Bu papka"  
}  
  
uzun_hajm = fs::file_size(p)  
yozish << "Hajmi: " << hajm << " bayt"
```

12.6.1 Papkada barcha fayllarni listing

```
uchun (o'zgarmas_avto& entry : fs::directory_iterator("/home")) {  
    yozish << entry.path() << qator_oxiri  
}
```

12.6.2 Papka yaratish

```
fs::create_directory("yangi_papka")  
fs::create_directories("ota/bola/nevara") // bir necha daraja
```

12.6.3 Fayl o'chirish

```
fs::remove("fayl.txt")  
fs::remove_all("papka") // papka va ichidagilarni
```

12.6.4 Ko'chirish va qayta nomlash

```
fs::rename("eski.txt", "yangi.txt")  
fs::copy("manba.txt", "nusxa.txt")
```

12.7 Amaliy misol: Talabalar ma'lumotlar bazasi (JSON)


```

#include <fstream>
#include <vector>
#include <nlohmann/json.hpp>
using json = nlohmann::json;

tuzilma Talaba {
    matn ism
    butun yoshi
    vektor<butun> baholar
}

// Talabani JSON ga
json talaba_json(o'zgarmas Talaba& t) {
    qaytarish json{
        {"ism", t.ism},
        {"yoshi", t.yoshi},
        {"baholar", t.baholar}
    }
}

// JSON dan talabaga
Talaba json_talaba(o'zgarmas json& j) {
    qaytarish Talaba{
        j["ism"],
        j["yoshi"],
        j["baholar"].get<vektor<butun>>()
    }
}

butun asosiy() {
    vektor<Talaba> guruh = {
        {"Aziza", 22, {85, 92, 78}},
        {"Bobur", 21, {90, 88, 95}},
        {"Karim", 23, {65, 70, 75}}
    }

    // JSON ga yozish
    json data = json::array()
    uchun (o'zgarmas Talaba& t : guruh) {
        data.push_back(talaba_json(t))
    }

    std::ofstream f("guruh.json")
    f << data.dump(4)
    f.close()

    // O'qib qaytarish
    std::ifstream f2("guruh.json")
    json yangi_data
    f2 >> yangi_data

    uchun (o'zgarmas avto& j : yangi_data) {

```

```
Talaba t = json_talaba(j)
yozish << t.ism << " (" << t.yoshi << ")" << qator_oxiri
}

qaytarish 0
}
```

Saqlangan guruh.json:

```
[
  {
    "baholar": [85, 92, 78],
    "ism": "Aziza",
    "yoshi": 22
  },
  {
    "baholar": [90, 88, 95],
    "ism": "Bobur",
    "yoshi": 21
  },
  ...
]
```

12.8 Eng ko'p uchraydigan xatolar

12.8.1 1. Faylni yopishni unutish

```
std::ofstream f("fayl.txt")
f << "...
// f.close() yo'q - odatda OK, destruktordan yopadi
```

Lekin agar siz faylni boshqa joyda o'qimoqchi bo'lsangiz:

```
f.close() // aniq yopish
std::ifstream f2("fayl.txt")
```

12.8.2 2. Fayl yo'qligini tekshirmaslik

```
std::ifstream f("yoq.txt")
matn qator
std::getline(f, qator) // bo'sh qator olinadi
```

Doim tekshiring: `f.is_open()`.

12.8.3 3. UTF-8 muammosi (Windows)

Windows'da matn fayllari ba'zan UTF-16. C++ standart oqimlari UTF-8 bilan ishlaydi. Maxsus konvertatsiya kerak bo'lishi mumkin.

12.8.4 4. Yo'l ajratuvchilar

Windows: `C:\Users\file.txt` yoki `C:/Users/file.txt` Linux: `/home/user/file.txt`

Eng yaxshisi — `std::filesystem::path` ishlatish, u o'zi sozlaydi.

12.8.5 5. Katta fayllarni butunlay xotiraga yuklash

```
matn katta = oqish_butun_fayl("1GB.bin")    // 1 GB xotira!
```

Katta fayllar uchun — qator-qator yoki bo'lak-bo'lak.

12.9 Bob bo'yicha mashqlar

12.9.1 Mashq 1: Faylga yozish (oson)

Foydalanuvchidan jumla so'rang va `output.txt` ga yozing.

12.9.2 Mashq 2: Fayldan o'qish (oson)

`input.txt` faylidagi har bir qatorni chiqaring.

12.9.3 Mashq 3: Qatorlar sonini sanash (oson)

Berilgan faylda nechta qator borligini sanang.

12.9.4 Mashq 4: So'z chastotasi (o'rta)

Matn faylida har bir so'z necha marta uchrayotganini sanab, natijani boshqa faylga JSON formatida saqlang.

12.9.5 Mashq 5: CSV → JSON (o'rta)

CSV faylni o'qib, JSON formatda saqlang.

12.9.6 Mashq 6: Telefon kitobi (qiyinroq)

JSON da saqlanadigan telefon kitobi: qo'shish, qidirish, o'chirish.

12.10 Xulosa

- `std::ofstream` — faylga yozish.
- `std::ifstream` — fayldan o'qish.
- `std::getline` — qator-qator o'qish.
- CSV — oddiy ma'lumotlar formati.
- JSON — zamonaviy, eng keng ishlatiladigan format.
- `std::filesystem` (C++17) — fayl tizimi bilan ishlash.

Keyingi bobda biz **ko'p oqimlilik** (multithreading) ni o'rganamiz — zamonaviy kompyuterlarning bir vaqtning o'zida bir necha vazifa bajarish qobiliyatidan to'liq foydalanish usuli.

13 · Ko'p oqimlilik — Parallel dasturlash

13.1 Nega ko'p oqimlilik kerak?

Zamonaviy kompyuterlarning protsessorlarida **4, 8, 16 yoki undan ko'p yadro** bor. Lekin agar siz oddiy dastur yozsangiz — u **bitta yadroda** ishlaydi. Boshqa yadrolar bo'sh turadi! Bu — resurs sarfi.

Ko'p oqimlilik (multithreading) — bu bir dasturning bir necha **oqimlari** bir vaqtning o'zida ishlashi. Har bir oqim — alohida yadroni ishlatishi mumkin.

13.1.1 Real hayotdagi misol

Tasavvur qiling, restoranda 4 ta oshpaz bor. Oddiy yondashuv: bitta oshpaz buyurtmalarni navbat bilan tayyorlasin, qolganlari kutsin. Aqlli yondashuv: 4 ta oshpaz **parallel** ishlasin — har biri o'z buyurtmasini.

Dasturda ham xuddi shunday:

```
Oqim 1: faylni o'qish      _____>
Oqim 2: hisoblash         _____>
Oqim 3: foydalanuvchi     _____>
Oqim 4: tarmoq            _____>
                           parallel ishlashadi
```

Bu — to'rt baravar tezroq (yoki shunchaga yaqin).

13.1.2 Qachon ko'p oqimlilik foydali?

- **Hisoblashga boy** vazifalar (matritsa ko'paytirish, AI, video kodlash)
- **I/O bilan kutuvchi** vazifalar (fayl, tarmoq) — bittasi kutsa, boshqa oqim ishlasin
- **Interaktiv dasturlar** — UI muzlamasligi uchun

13.1.3 Qachon kerak emas?

- Sodda, tezkor amallar (oqim ochish overhead'i amalga zarar yetkazadi)

- Ketma-ket mantiqiy ishlar (bir-biridan bog'liq)
-

13.2 `std::thread` — birinchi oqim

```
#include <thread>
#include <iostream>

bosh salom() {
    yozish << "Salom oqimdan!" << qator_oxiri
}

butun asosiy() {
    std::thread t(salom)           // yangi oqim yaratish va ishga tushirish
    t.join()                       // u tugashini kutish

    yozish << "Asosiyda" << qator_oxiri
    qaytarish 0
}
```

Tushuntirish: - `std::thread t(salom)` — yangi oqim ochiladi va `salom()` funksiyasi unda boshlanadi. - `t.join()` — asosiy oqim `t` tugashini kutadi.

13.2.1 Parametrlı funksiya bilan

```
bosh chiqarish(matn xabar, butun marta) {
    uchun (butun i = 0; i < marta; i++) {
        yozish << xabar << qator_oxiri
    }
}

std::thread t(chiqarish, "Salom", 3)
t.join()
```

13.2.2 Lambda bilan

```
std::thread t([](butun n) {
    uchun (butun i = 0; i < n; i++) {
        yozish << i << qator_oxiri
    }
}, 5)
t.join()
```

13.2.3 Bir necha oqim

```
butun asosiy() {
    std::thread t1([]{ yozish << "Oqim 1" << qator_oxiri });
    std::thread t2([]{ yozish << "Oqim 2" << qator_oxiri });
    std::thread t3([]{ yozish << "Oqim 3" << qator_oxiri });

    t1.join();
    t2.join();
    t3.join();
}
```

Natija — **tartibsiz** chiqishi mumkin:

```
Oqim 2
Oqim 1
Oqim 3
```

Yoki:

```
Oqim 1Oqim 2

Oqim 3
```

Chunki oqimlar parallel ishlashadi va `yozi sh` ga bir vaqtda yozishlari mumkin. Bu — **birinchi qiyinchilik**.

13.3 Sinxronizatsiya — `std::mutex`

Yuqoridagi muammoning yechimi — **mutex** (mutual exclusion). “Hech kim bir vaqtda kira olmaydi” qoidasi.

```
#include <mutex>

std::mutex qulf

bosh xavfsiz_yozish(matn xabar) {
    qulf.lock()
    yozish << xabar << qator_oxiri
    qulf.unlock()
}
```

Endi:

```
std::thread t1([]{ xavfsiz_yozish("Oqim 1") });
std::thread t2([]{ xavfsiz_yozish("Oqim 2") });
t1.join();
t2.join();
```

Natija — har doim toza:

Oqim 1
Oqim 2

(yoki teskari, lekin bo'lib chiqmaydi).

13.3.1 lock_guard — xavfsizroq usul

lock() va unlock() ni qo'lda chaqirish — xavfli. Agar oraliqda istisno yuz bersa, unlock() chaqirilmaydi.

std::lock_guard — RAII (avtomatik) qulflash:

```
bosh xavfsiz_yozish(matn xabar) {
    std::lock_guard<std::mutex> lg(qulf)
    yozish << xabar << qator_oxiri
    // lg destruktori avtomatik unlock qiladi
}
```

13.3.2 Resurs poyga (Race condition)

Quyidagi kodda muammo bor:

```
butun hisoblagich = 0

bosh oshirish() {
    uchun (butun i = 0; i < 100000; i++) {
        hisoblagich++
    }
}

butun asosiy() {
    std::thread t1(oshirish)
    std::thread t2(oshirish)
    t1.join()
    t2.join()
    yozish << hisoblagich // 200000 deb kutiladi, lekin...
}
```

Natija **noaniq** — 100000 dan 200000 gacha biror son. Nima uchun?

hisoblagich++ aslida 3 ta amal: 1. hisoblagich ni o'qish 2. 1 qo'shish 3. Qayta saqlash

Ikki oqim bir vaqtda 1 ni o'qib, 1 qo'shib, 2 ni saqlasa — ikkalasi ham 2 ga oshiradi, lekin aslida 3 bo'lishi kerak edi.

Yechim:

```
std::mutex m
butun hisoblagich = 0

bosh oshirish() {
    uchun (butun i = 0; i < 100000; i++) {
        std::lock_guard<std::mutex> lg(m)
        hisoblagich++
    }
}
```

Endi har bir `hisoblagich++` atomar — boshqa oqim aralasholmaydi.

13.4 `std::atomic` — engil sinxronizatsiya

Mutex sekin. Ba'zan `std::atomic` tezroq:

```
#include <atomic>

std::atomic<butun> hisoblagich = 0

bosh oshirish() {
    uchun (butun i = 0; i < 100000; i++) {
        hisoblagich++ // atomar amal
    }
}
```

`atomic` oddiy turlar uchun (butun, ko'rsatkich, mantiq) — protsessor darajasidagi sinxronizatsiya. Mutex'dan tez.

13.5 `std::async` — asinxron bajarish

Oqim ochib, uning natijasini olish:

```
#include <future>

butun katta_hisob() {
    butun yigindi = 0
    uchun (butun i = 0; i < 1000000; i++) {
```

```
        yigindi += i
    }
    qaytarish yigindi
}

butun asosiy() {
    std::future<butun> f = std::async(std::launch::async, katta_hisob)

    // Boshqa ish qilish ...
    yozish << "Hisobni kutyapmiz" << qator_oxiri

    butun natija = f.get() // natijani olamiz (kutamiz agar tayyor
↪ bo'lmasa)
    yozish << "Natija: " << natija
}
```

`std::async` — yangi oqimda funksiyani ishga tushiradi va `future` qaytaradi. `future.get()` orqali natijani olamiz.

13.6 `std::promise` Va `std::future`

Oqimlar o'rtasida ma'lumot uzatish:

```
std::promise<butun> p
std::future<butun> f = p.get_future()

std::thread t([&p]() {
    butun natija = 42
    p.set_value(natija)
})

butun n = f.get() // 42
t.join()
```

`promise` — ma'lumot yuborgich. `future` — ma'lumot qabul qiluvchi.

13.7 Oqimlar havzasi (Thread pool)

Har gal yangi oqim ochish — qimmat. **Thread pool** — oldindan yaratilgan oqimlar to'plami. Vazifalar navbatga qo'yiladi.

`uz++` da `uzpp::OqimHavzasi` mavjud (yoki o'zingiz yaratishingiz mumkin).

```

#include <vector>
#include <thread>

sinf OqimHavzasi {
yopiq:
    vektor<std::thread> oqimlar

ochiq:
    OqimHavzasi(butun n) {
        uchun (butun i = 0; i < n; i++) {
            oqimlar.emplace_back([]() {
                // har bir oqim navbatdan vazifa olib bajaradi
            })
        }
    }
    // ...
}

```

Bu murakkab mavzu — bu yerda faqat tushuncha berdik.

13.8 Sinxronizatsiya vositalari

Vosita	Maqsadi
std::mutex	Yagona kirish (mutual exclusion)
std::lock_guard	RAII mutex (avtomatik unlock)
std::unique_lock	Yanada moslashuvchan mutex
std::atomic	Atomar o'zgaruvchilar
std::condition_variable	Oqimni kutish/signalga uyg'otish
std::semaphore (C++20)	Resurs hisoblagich
std::barrier (C++20)	Bir necha oqimni sinxronlash

13.9 Amaliy misol: Parallel massiv qayta ishlash

```

#include <vector>
#include <thread>
#include <numeric>

butun yigindi_qism(o'zgarmas vektor<butun>& v, butun bosh, butun oxir) {
    qaytarish std::accumulate(v.begin() + bosh, v.begin() + oxir, 0)
}

```

```

butun asosiy() {
    vektor<butun> v(1000000, 1)           // million ta 1

    // Bitta oqimda
    butun y1 = std::accumulate(v.begin(), v.end(), 0)
    yozish << "Bitta oqim: " << y1 << qator_oxiri

    // To'rtta oqimda
    butun n = v.size()
    butun qism = n / 4
    std::future<butun> f1 = std::async(yigindi_qism, std::ref(v), 0, qism)
    std::future<butun> f2 = std::async(yigindi_qism, std::ref(v), qism, qism*2)
    std::future<butun> f3 = std::async(yigindi_qism, std::ref(v), qism*2, qism*3)
    std::future<butun> f4 = std::async(yigindi_qism, std::ref(v), qism*3, n)

    butun y2 = f1.get() + f2.get() + f3.get() + f4.get()
    yozish << "To'rt oqim: " << y2 << qator_oxiri
}

```

Katta ma'lumotlar uchun — to'rtta oqim ~4 marta tezroq.

13.10 Eng ko'p uchraydigan xatolar

13.10.1 1. join() qilishni unutish

```

std::thread t(salom)
// t.join() yo'q - dastur tugaganda crash

```

13.10.2 2. Race condition

Mutex'siz birgalikda o'zgartirilgan o'zgaruvchilar.

13.10.3 3. Deadlock

Ikki oqim bir-birini kutadi va hech qachon ozod bo'lmaydi:

```

std::mutex m1, m2

void f1() {
    m1.lock()
    m2.lock()
    // ...
}

```

```
void f2() {
    m2.lock()           // teskari tartib!
    m1.lock()
    // ...
}
```

f1 m1 ni oldi, m2 ni kutadi. f2 m2 ni oldi, m1 ni kutadi. **Hech qachon tugamaydigan kutish.**

Yechim: Mutex'larni doim **bir xil tartibda** olish.

13.10.4 4. Detached oqim resurslari

```
{
    std::vector<int> v = {1, 2, 3}
    std::thread t([&v]() {
        // ...v ni ishlatish...
    })
    t.detach()
}
```

// v yo'qoldi, lekin oqim hali ishlaydi!

13.10.5 5. Thread overhead

```
uchun (butun i = 0; i < 1000000; i++) {
    std::thread t([]() { /* kichik ish */ })
    t.join()
}
```

Million ta oqim ochish — bu juda sekin va resurs sarfi. Thread pool ishlateng.

13.11 Bob bo'yicha mashqlar

13.11.1 Mashq 1: Birinchi oqim (oson)

Yangi oqimda "Salom, oqim!" deb chiqaring.

13.11.2 Mashq 2: Bir nechta oqim (oson)

5 ta oqim yarating, har biri o'z raqamini chiqarsin.

13.11.3 Mashq 3: Hisoblagich bilan mutex (o'rta)

Ikki oqim hisoblagichni 100000 marta oshirsin — natija aniq bo'lsin.

13.11.4 Mashq 4: Parallel yig'indi (o'rtacha)

Katta vektorni 4 oqimda yig'indini hisoblang.

13.11.5 Mashq 5: Producer-Consumer (qiyin)

`Producer` oqim navbatga element qo'yadi. `Consumer` oqim oladi. `Mutex` va `condition_variable` ishlating.

13.12 Xulosa

- **Ko'p oqimlilik** — bir vaqtda bir necha ishni bajarish.
- `std::thread` — yangi oqim ochish.
- `std::mutex` — yagona kirish nazorati.
- `std::lock_guard` — RAII mutex.
- `std::atomic` — engil sinxronizatsiya.
- `std::async/std::future` — asinxron ishlatish va natija olish.
- **Race condition** va **deadlock** — eng ko'p tarqalgan muammolar.

Keyingi bobda biz **standart kutubxona** ni umumiy ko'rib chiqamiz — `uz++` bilan birga keladigan barcha foydali modullar haqida.

14 · Standart kutubxona

14.1 uz++ standart kutubxonasi haqida

uz++ — bu C++23 ustida qurilgan. Demak, sizning ixtiyoringizda C++ ning **butun** standart kutubxonasi bor. Bundan tashqari, uz++ o'zining o'zbek tilidagi modullarini taqdim etadi.

Bu bobda biz uz++ standart kutubxonasini va shu bilan birga foydali C++ modullarini umumiy ko'rib chiqamiz. Bu — ma'lumot bobi. Har bir moduldan amalda foydalanganingiz sayin chuqurroq tushunasiz.

14.1.1 uz++ standart modullari

Hozirda o'zbeklashtirilgan modullar:

Modul	Tavsif
matn	Matn (string) bilan ishlash
xatoliklar	Xatolik turlari va istisnolar
vaqt	Vaqt va sana bilan ishlash
matematika	Matematik funksiyalar
sinov	Test framework
jurnal	Logging (jurnallash)

14.2 matematika moduli

Matematik funksiyalar va konstantalar.

```
#include <matematika>

// Konstantalar
yozish << matematika::PI           // 3.14159...
yozish << matematika::E           // 2.71828...
```

```
// Asosiy funksiyalar
yozish << matematika::ildiz(16)           // 4 (kvadrat ildiz)
yozish << matematika::quvvat(2, 10)        // 1024 (2^10)
yozish << matematika::log(100)            // 4.605... (tabiiy logarifm)
yozish << matematika::log10(1000)         // 3 (o'nlik logarifm)

// Trigonometriya
yozish << matematika::sinus(matematika::PI / 2) // 1.0
yozish << matematika::kosinus(0)           // 1.0
yozish << matematika::tangens(matematika::PI / 4) // 1.0

// Yaxlitlash
yozish << matematika::yaxlitla(3.7)        // 4
yozish << matematika::pastga(3.7)         // 3
yozish << matematika::yuqoriga(3.2)       // 4

// Min/max/abs
yozish << matematika::min(3, 7)           // 3
yozish << matematika::maks(3, 7)         // 7
yozish << matematika::absolyut(-5)       // 5
```

14.3 vaqt moduli

Vaqt va sana bilan ishlash.

```
#include <vaqt>

// Joriy vaqt
avto hozir = vaqt::hozir()
yozish << "Hozir: " << hozir.formatla("%Y-%m-%d %H:%M:%S")

// Vaqt elementlari
yozish << "Yil: " << hozir.yil()
yozish << "Oy: " << hozir.oy()
yozish << "Kun: " << hozir.kun()
yozish << "Soat: " << hozir.soat()

// Sana yaratish
avto sana = vaqt::Sana(2026, 5, 21)

// Farq
avto kelajak = sana.qoshish_kun(30)
yozish << kelajak.formatla("%Y-%m-%d")

// Vaqt o'lchash
avto bosh = vaqt::hozir()
// ... biron ish ...
```



```
avto oxir = vaqt::hozir()
yozish << "O'tdi: " << (oxir - bosh).millisekund() << " ms"
```

14.4 *sinov* moduli — Test framework

uz++ da test yozish uchun sodda framework.

```
#include <sinov>

@sinov("yigindi to'g'ri ishlashi kerak") {
    sinov::tekshirish(yigindi(2, 3) == 5)
    sinov::tekshirish(yigindi(0, 0) == 0)
    sinov::tekshirish(yigindi(-1, 1) == 0)
}

@sinov("matn uzunligini olish") {
    sinov::tenglik(matn("salom").length(), 5)
    sinov::tenglik(matn("").length(), 0)
}
```

Testlarni ishga tushirish:

```
uzpp sinov
```

Natija:

```
[OK] yigindi to'g'ri ishlashi kerak
[OK] matn uzunligini olish
2/2 muvaffaqiyatli
```

14.5 *jurnal* moduli — Logging

Dasturning ishini kuzatish uchun:

```
#include <jurnal>

butun asosiy() {
    jurnal::malumot("Dastur boshlandi")
    jurnal::ogohlantirish("Bu - ogohlantirish")
    jurnal::xato("Xato yuz berdi")
}
```

```
butun yosh = 25
jurnal::malumot("Foydalanuvchi yoshi: ", yosh)
}
```

Natija:

```
[2026-05-21 14:30:00] [MA'LUMOT] Dastur boshlandi
[2026-05-21 14:30:00] [OGOHLANTIRISH] Bu - ogohlantirish
[2026-05-21 14:30:00] [XATO] Xato yuz berdi
[2026-05-21 14:30:00] [MA'LUMOT] Foydalanuvchi yoshi: 25
```

Faylga yozish ham mumkin:

```
jurnal::fayl("app.log")
jurnal::malumot("Bu fayl ga yoziladi")
```

14.6 kriptο moduli — Kriptografiya

Hash, shifrlash, parol himoyasi:

```
#include <kripto>

// MD5 / SHA-256 hash
matn s = "Salom, dunyo!"
yozish << kripto::md5(s)
yozish << kripto::sha256(s)

// Parolni xavfsiz saqlash
matn parol = "qiyin_parol_123"
matn hash = kripto::parol_hash(parol)

// Tekshirish
 mantiq to'gri = kripto::parol_tekshirish("qiyin_parol_123", hash)
```

14.7 tarmoq moduli — Tarmoq aloqalari

HTTP so'rovlari, socket'lar:

```
#include <tarmoq>

// HTTP GET so'rov
```

```

avto javob = tarmoq::http_oluvchi("https://api.example.com/data")

agar (javob.holat == 200) {
    yozish << javob.matn
}

// POST so'rov
tarmoq::http_yuborish("https://api.example.com/post", json_data)

```

14.8 suniy_intellekt moduli — AI integratsiyasi

LLM va AI bilan ishlash:

```

#include <suniy_intellekt>

avto suxbat = suniy_intellekt::Suhbat()
suxbat.tizim("Sen foydali yordamchisan")

matn savol = "C++ da const ko'rsatkich qanday yaratiladi?"
matn javob = suxbat.sorash(savol)
yozish << javob

```

14.9 C++ standart kutubxonasidan foydalanish

Eng ko'p ishlatiladigan modullar:

14.9.1 <algorithm> — Algoritmlar

```

#include <algorithm>
vektor<butun> v = {3, 1, 4, 1, 5, 9, 2, 6}

std::sort(v.begin(), v.end())
std::reverse(v.begin(), v.end())
butun joy = std::count(v.begin(), v.end(), 1)
avto max_el = std::max_element(v.begin(), v.end())

// Filtering
vektor<butun> juft_lar
std::copy_if(v.begin(), v.end(), std::back_inserter(juft_lar),
    [](butun x) { qaytarish x % 2 == 0 })

```

```
// Map (transform)
vektor<butun> kvadratlar
std::transform(v.begin(), v.end(), std::back_inserter(kvadratlar),
    [](butun x) { qaytarish x * x })
```

14.9.2 <random> — Tasodifiy sonlar

```
#include <random>

std::random_device rd
std::mt19937 gen(rd())
std::uniform_int_distribution<> dist(1, 100)

butun tasodif = dist(gen)           // 1 dan 100 gacha
```

14.9.3 <chrono> — Vaqt o'lchovi

```
#include <chrono>

avto bosh = std::chrono::high_resolution_clock::now()
// ... ish ...
avto oxir = std::chrono::high_resolution_clock::now()

avto millisekund = std::chrono::duration_cast<std::chrono::milliseconds>(oxir - bosh)
yozish << millisekund.count() << " ms"
```

14.9.4 <regex> — Regular expressions

```
#include <regex>

matn s = "Telefon: +998901234567, email: aziza@example.com"
std::regex telefon(R"(\+\d{12})")
std::smatch m

agar (std::regex_search(s, m, telefon)) {
    yozish << "Telefon topildi: " << m[0]
}
```

14.9.5 <thread>, <mutex>, <future> — Ko'p oqimlilik

13-bobda ko'rdik.

14.9.6 <filesystem> — Fayl tizimi

12-bobda ko'rdik.

14.10 Amaliy misol: Vaqt hisoblagichi bilan testlash

```
#include <vaqt>
#include <jurnal>
#include <vector>
#include <algorithm>

butun asosiy() {
    jurnal::malumot("Test boshlandi")

    avto bosh = vaqt::hozir()

    // Vazifa: 1 million tasodifiy son yaratib, tartiblash
    vektor<butun> v(1000000)
    std::generate(v.begin(), v.end(), []() { qaytarish std::rand() })

    std::sort(v.begin(), v.end())

    avto oxir = vaqt::hozir()
    avto ms = (oxir - bosh).millisekund()

    jurnal::malumot("Tartiblash vaqti: ", ms, " ms")

    qaytarish 0
}
```

Bu dasturning chiqishi:

```
[2026-05-21 14:30:00] [MA'LUMOT] Test boshlandi
[2026-05-21 14:30:01] [MA'LUMOT] Tartiblash vaqti: 234 ms
```

14.11 Bob bo'yicha mashqlar

14.11.1 Mashq 1: Tasodifiy son (oson)

<random> orqali 1 dan 100 gacha tasodifiy 10 ta son chiqaring.

14.11.2 Mashq 2: Vaqt o'lchash (oson)

`<chrono>` orqali Fibonachchi 30-elementni hisoblash vaqtini o'lchang.

14.11.3 Mashq 3: Regex (o'rta)

Berilgan matnda barcha email manzillarni toping.

14.11.4 Mashq 4: Log fayl tahlili (o'rtacha)

Log fayldan barcha xato satrlarini boshqa faylga yozing.

14.11.5 Mashq 5: Test framework (qiyinroq)

O'zingizning matematik funksiyalaringiz uchun testlar yozing.

14.12 Xulosa

- `uz++` standart kutubxonasi (`matn`, `vaqt`, `matematika`, `sinov`, `jurnal`, `kripto`, `tarmoq`, `suniy_intellekt`) — o'zbek tilida.
- `C++` standart kutubxonasi (`<algorithm>`, `<random>`, `<regex>`, `<chrono>`, `<thread>`, `<filesystem>`) — to'liq mavjud.
- Har bir modul aniq vazifa uchun mo'ljallangan.
- Yangi modullar doim qo'shilmoqda.

Keyingi bobda biz **GUI dasturlash** ni o'rganamiz — grafik interfeysli dasturlar yaratish.

15 · GUI dasturlash (CopperSpice)

15.1 GUI nima va nima uchun kerak?

Hozirgacha bizning dasturlarimiz **konsol** (terminal) orqali ishlardi. Foydalanuvchi matn yozadi, dastur matn chiqaradi. Bu — yaxshi, lekin zamonaviy foydalanuvchi **grafik interfeysni** (GUI — Graphical User Interface) kutadi: tugmalar, oynalar, menyu, slaydlar.

GUI dasturlar: - Veb-brauzer (Chrome, Firefox) - Hujjat muharrirlari (Word, Notepad++) - Suratlar tahrir qiluvchilar (Photoshop, GIMP) - O'yinlar - Operatsion tizim oyna boshqaruvchisi uz++ da GUI yaratish uchun biz **CopperSpice** kutubxonasidan foydalanamiz. Bu Qt'ning ochiq versiyasi.

***Diqqat:** Bu bob faqat tushuncha berish uchun. CopperSpice'ni o'rnatish va sozlash alohida hujjatlarda batafsil bayon etilgan.*

15.2 Birinchi GUI dastur

```
// salom_oyna.uzpp
#include <QApplication>
#include <QLabel>
#include <QPushButton>
#include <QVBoxLayout>
#include <QWidget>

butun asosiy(butun argc, belgi* argv[]) {
    QApplication dastur(argc, argv)

    QWidget oyna
    oyna.setWindowTitle("Birinchi GUI dasturim")
    oyna.resize(400, 300)

    QLabel* matn = new QLabel("Salom, dunyo!")
```

```

QPushButton* tugma = new QPushButton("Bosing!")

QVBoxLayout* maket = new QVBoxLayout(&oyna)
maket->addWidget(matn)
maket->addWidget(tugma)

oyna.show()
qaytarish dastur.exec()
}

```

Ishga tushirish:

```
uzpp qurish salom_oyna.uzpp --bare -I C:/CopperSpice/include -l QtCore -l QtGui
```

--bare — uz++ runtime'siz qurish. -I va -l — CopperSpice sarlavhalari va kutubxonalari.

15.2.1 Bu dasturda nima bor?

1. **QApplication** — har bir GUI dasturning asosi. Hodisalarni boshqaradi.
2. **QWidget** — asosiy oyna.
3. **QLabel** — matn ko'rsatuvchi vidjet.
4. **QPushButton** — tugma.
5. **QVBoxLayout** — vertikal joylashuv (yuqoridan pastga).
6. **oyna.show()** — oynani ko'rsatish.
7. **dastur.exec()** — hodisalar tsiklini ishga tushirish.

15.3 Maketlar (Layouts)

Vidjetlarni oynaga joylashish — *layout* orqali:

15.3.1 Vertikal va gorizontal

```

QVBoxLayout          // vertikal
QHBoxLayout          // gorizontal

```

15.3.2 Grid (panjara)

```

QGridLayout* maket = new QGridLayout()
maket->addWidget(vidjet1, 0, 0)    // qator 0, ustun 0
maket->addWidget(vidjet2, 0, 1)    // qator 0, ustun 1
maket->addWidget(vidjet3, 1, 0)    // qator 1, ustun 0

```


15.3.3 Forma

```
QFormLayout* maket = new QFormLayout()
maket->addRow("Ism:", new QLineEdit())
maket->addRow("Yosh:", new QSpinBox())
```

15.4 Signallar va slotlar

CopperSpice'ning eng kuchli xususiyati — **signal-slot** tizimi. Tugma bosilganda biror funksiyani chaqirish uchun:

```
QPushButton* tugma = new QPushButton("Bosing!")

// Lambda bilan
QObject::connect(tugma, &QPushButton::clicked, []() {
    yozish << "Tugma bosildi!" << qator_oxiri
})

// Yoki funksiya bilan
bosh tugma_bosildi() {
    yozish << "Tugma bosildi!" << qator_oxiri
}
QObject::connect(tugma, &QPushButton::clicked, tugma_bosildi)
```

Signal — vidjet “men hodisani his qildim” deydi. Slot — funksiya “men buni qabul qilaman” deydi.

15.4.1 Misol: Hisoblagich

```
butun hisoblagich = 0
QLabel* yorliq = new QLabel("0")
QPushButton* tugma = new QPushButton("Oshir")

QObject::connect(tugma, &QPushButton::clicked, [&]() {
    hisoblagich++
    yorliq->setText(std::to_string(hisoblagich).c_str())
})
```

Har safar tugma bosilganda — hisoblagich oshadi va yorliqda yangilanadi.

15.5 Muhim vidjetlar

Vidjet	Maqsadi
QLabel	Matn yoki rasm ko'rsatish
QPushButton	Tugma
QLineEdit	Bir qatorlik matn maydoni
QTextEdit	Ko'p qatorli matn
QCheckBox	Belgilash kutisi
QRadioButton	Radio tanlov
QComboBox	Pastga tushadigan ro'yxat
QSpinBox	Son tanlash
QSlider	Slayder
QProgressBar	Progress indikatori
QListWidget	Ro'yxat
QTableWidget	Jadval
QTreeWidget	Daraxt

15.5.1 Misol: matn maydonidan o'qish

```
QLineEdit* matn_maydoni = new QLineEdit()
QPushButton* tugma = new QPushButton("Tasdiqlash")

QObject::connect(tugma, &QPushButton::clicked, [matn_maydoni]() {
    QString matn = matn_maydoni->text()
    yozish << "Foydalanuvchi yozdi: " << matn.toString()
})
```

15.6 Menyu va status qatori

```
QMainWindow oyna

// Menyu yaratish
QMenu* fayl_menyu = oyna.menuBar()->addMenu("Fayl")
fayl_menyu->addAction("Yangi")
fayl_menyu->addAction("Ochish")
fayl_menyu->addSeparator()
fayl_menyu->addAction("Chiqish")

// Status qatori
oyna.statusBar()->showMessage("Tayyor")
```

15.7 To'liq misol: Mini matn muharriri

```

#include <QApplication>
#include <QMainWindow>
#include <QTextEdit>
#include <QMenu>
#include <QMenuBar>
#include <QFileDialog>
#include <QFile>
#include <QTextStream>

butun asosiy(butun argc, belgi* argv[]) {
    QApplication dastur(argc, argv)

    QMainWindow oyna
    oyna.setWindowTitle("Mini muharrir")
    oyna.resize(800, 600)

    QTextEdit* muharrir = new QTextEdit()
    oyna.setCentralWidget(muharrir)

    // Menyu
    QMenu* fayl_menyu = oyna.menuBar()->addMenu("Fayl")

    QAction* yangi = fayl_menyu->addAction("Yangi")
    QObject::connect(yangi, &QAction::triggered, [muharrir]() {
        muharrir->clear()
    })

    QAction* ochish = fayl_menyu->addAction("Ochish")
    QObject::connect(ochish, &QAction::triggered, [&]() {
        QString yol = QFileDialog::getOpenFileName(&oyna, "Faylni ochish")
        agar (!yol.isEmpty()) {
            QFile f(yol)
            agar (f.open(QIODevice::ReadOnly | QIODevice::Text)) {
                QTextStream oqim(&f)
                muharrir->setPlainText(oqim.readAll())
            }
        }
    })

    QAction* saqlash = fayl_menyu->addAction("Saqlash")
    QObject::connect(saqlash, &QAction::triggered, [&]() {
        QString yol = QFileDialog::getSaveFileName(&oyna, "Saqlash")
        agar (!yol.isEmpty()) {
            QFile f(yol)
            agar (f.open(QIODevice::WriteOnly | QIODevice::Text)) {
                QTextStream oqim(&f)
                oqim << muharrir->toPlainText()
            }
        }
    })
}

```

```
fayl_menyu->addSeparator()

QAction* chiqish = fayl_menyu->addAction("Chiqish")
QObject::connect(chiqish, &QAction::triggered, &dastur, &QApplication::quit)

oyna.show()
qaytarish dastur.exec()
}
```

Bu — to'liq ishlaydigan matn muharriri! Fayl ochish, saqlash, yangi.

15.8 --bare rejimi

CopperSpice/Qt bilan ishlash uchun uz++ ning maxsus rejimi:

```
uzpp qurish dastur.uzpp --bare -I /path/to/copperspice/include -l QtCore -l QtGui
```

--bare: - Standart uz++ runtime'siz qurish - Tashqi kutubxonalar bilan integratsiya uchun - Sof C++ asosida

15.9 Bob bo'yicha mashqlar

15.9.1 Mashq 1: Salom dunyo oynasi (oson)

Oddiy "Salom, dunyo!" yorlig'i bo'lgan oyna yarating.

15.9.2 Mashq 2: Kalkulyator (o'rta)

GUI kalkulyator: ikkita son, amal tugmalari, natija.

15.9.3 Mashq 3: Soat (o'rta)

QTimer bilan har soniya yangilanadigan soat ko'rsating.

15.9.4 Mashq 4: To'do ro'yxati (o'rtacha)

QListWidget bilan ishlar ro'yxati. Qo'shish, o'chirish tugmalari.

15.9.5 Mashq 5: Suratlar ko'rsatkichi (qiyinroq)

Papkadan suratlarni o'qib `QLabel` da ko'rsating. "Oldingi" va "Keyingi" tugmalari bilan.

15.10 Xulosa

- **GUI** — grafik interfeys, zamonaviy dasturlarning standarti.
- **CopperSpice** (yoki `Qt`) — `uz++` uchun asosiy GUI kutubxonasi.
- **`QApplication`** — har bir GUI dasturning asosi.
- **Vidjetlar** — `QLabel`, `QPushButton`, `QLineEdit`, va h.k.
- **Maketlar** — vidjetlarni joylashtirish.
- **Signal va slot** — hodisa-funksiya bog'lanishi.
- **`--bare`** — `uz++` ning maxsus GUI rejimi.

Keyingi bobda **ilovalar** bor — kalit so'zlar lug'ati, o'rnatish bo'yicha to'liq yo'riqnoma, foydali resurslar.

16 · Ilovalar

16.1 A · Kalit soʻzlar lugʻati

uz++ ning barcha kalit soʻzlari va ularning C++ ekvivalentlari.

16.1.1 Asosiy turlar

uz++	C++	Maʼnosi
butun	int	Butun son (32 bit)
uzun	long long	Katta butun son (64 bit)
kasr	float	Haqiqiy son (32 bit)
ikkilangan	double	Aniq haqiqiy son (64 bit)
mantiq	bool	Rost/yolgʻon
belgi	char	Bitta belgi
matn	std::string	Matn satri
bosh	void	Boʻsh (qiymatsiz)

16.1.2 Konteynerlar

uz++	C++	Tavsif
vektor	std::vector	Dinamik massiv
roy'xat	std::list	Bogʻlangan roʻyxat
tartiblangan_xarita	std::map	Tartiblangan kalit-qiymat
xarita	std::unordered_map	Hash xarita
to'plam	std::set	Takrorlanmas toʻplam
massiv	std::array	Statik massiv
juftlik	std::pair	Juftlik
uchlik	std::tuple	Uchlik va undan koʻp

16.1.3 Qoʻshimcha turlar

uz++	C++	Tavsif
ixtiyoriy	std::optional	Bo'lishi mumkin yoki yo'q
Natija	std::expected	Qiymat yoki xato
Tanlov	std::optional	Tanlov
yagona_korsatkich	std::unique_ptr	Yagona ko'rsatkich
umumiy	std::shared_ptr	Bo'lishilgan ko'rsatkich
funksiya	std::function	Funksiya obyekti
kelajak	std::future	Kelajakdagi qiymat
vada	std::promise	Va'da (promise)
qulf	std::mutex	Mutex
atomik	std::atomic	Atomar o'zgaruvchi

16.1.4 Boshqaruv tuzilmalari

uz++	C++	Ma'nosi
agar	if	Agar
aks_holda / yoki	else	Aks holda
uchun	for	Takrorlash sikli
davr	while	Davriy takrorlash
bajar/davr	do/while	Bajar/davr
tanlash	switch	Tanlash
holat	case	Holat
boshqa	default	Standart
to'xtatish	break	To'xtatish
davom_ettirish	continue	Davom ettirish
qaytarish	return	Qaytarish

16.1.5 Xatolik bilan ishlash

uz++	C++	Ma'nosi
urinish	try	Urinib ko'rish
ushlash	catch	Ushlash
irgitish	throw	Irgitish
tasdiqlash	assert	Tasdiqlash (debug)

16.1.6 Sinf va tuzilmalar

uz++	C++	Ma'nosi
sinf	class	Sinf
tuzilma	struct	Tuzilma
ochiq	public	Ochiq
yopiq	private	Yopiq
himoyalangan	protected	Himoyalangan

uz++	C++	Maʼnosi
mavhum	virtual	Virtual
ustidan_yozish	override	Qayta yozish
dost	friend	Doʻst
shablon	template	Shablon
tur	typename	Tur nomi
turdash	typename	Turdosh

16.1.7 Boshqa kalit soʻzlar

uz++	C++	Tavsif
oʻzgarmas	const	Oʻzgarmas
oʻzgaruvchan	auto	Avtomatik tur
qator_ichi	inline	Ichki funksiya
yangi	new	Dinamik xotira
oʻchirish	delete	Xotirani qaytarish
rost	true	Mantiqiy rost
yolgʻon	false	Mantiqiy yolgʻon
nullptr	nullptr	Boʻsh koʻrsatkich

16.2 B · Oʻrnatish boʻyicha batafsil qoʻllanma

16.2.1 Windows

1. **Yuklab olish:** [GitHub Releases](#) sahifasidan eng soʻnggi `uzpp-setup.exe` faylini yuklab oling.
2. **Oʻrnatish:** Yuklangan faylni ishga tushiring. Oʻrnatuvchi quyidagilarni avtomatik oʻrnatadi:
 - `uzpp.exe` — transpilyator
 - MinGW-w64 `g++` — C++ kompilyatori
 - VS Code kengaytmasi (`uzpp.uzpp`)

3. **Tekshirish:**

```
uzpp --yordam
```

Agar yordam matni chiqsa, oʻrnatish muvaffaqiyatli.

16.2.2 Linux

```
curl -fsSL https://raw.githubusercontent.com/timetolivechk-spec/uzpp/main/install.sh
↵ | bash
```

`install.sh` skripti: - g++ o'rnatilganligini tekshiradi (yo'q bo'lsa — apt/yum/pacman orqali o'rnatadi) - uzpp ni `/usr/local/bin/` ga joylashtiradi - VS Code mavjud bo'lsa, kengaytmani ham o'rnatadi

16.2.3 macOS

```
curl -fsSL https://raw.githubusercontent.com/timetolivechk-spec/uzpp/main/install.sh
↵ | bash
```

Yoki Homebrew orqali (kelajakda):

```
brew install uzpp
```

g++ o'rniga clang++ ishlatiladi.

16.3 C · Fayl kengaytmalari

Kengaytma	Tavsif
<code>.uzpp</code>	uz++ manba fayli
<code>.cpp</code> / <code>.cxx</code> / <code>.cc</code>	C++ manba fayli
<code>.hpp</code> / <code>.h</code>	C++ sarlavha fayli
<code>.exe</code>	Bajarib bo'ladigan fayl (Windows)
<code>.so</code>	Dinamik kutubxona (Linux)
<code>.dylib</code>	Dinamik kutubxona (macOS)
<code>.dll</code>	Dinamik kutubxona (Windows)

16.4 D · CLI buyruqlari

16.4.1 Asosiy buyruqlar

Buyruq	Tavsif
<code>uzpp qurish fayl.uzpp</code>	Dasturni kompilyatsiya qilish
<code>uzpp ishga-tushirish fayl.uzpp</code>	Kompilyatsiya + ishga tushirish
<code>uzpp sinov</code>	Testlarni bajarish
<code>uzpp formatlash</code>	Kodni formatlash
<code>uzpp lsp</code>	LSP server ishga tushirish
<code>uzpp hujjat</code>	Hujjat generatsiya qilish

Buyruq	Tavsif
uzpp --yordam	Yordam
uzpp --versiya	Versiya

16.4.2 Bayroqlar

Bayroq	Tavsif
--debug	Debug rejimida kompilyatsiya
--bare	uzpp_runtime.hpp siz qurish (Qt/CopperSpice uchun)
-I yo'l	Include katalogi
-l yo'l	Kutubxona fayli
--target=os	Maqsadli platforma
--show-cpp	C++ kodini ko'rsatish
--optimize=N	Optimizatsiya darajasi (0-3)

16.5 E · C++23 mosligi

uz++ C++23 standartini to'liq qo'llab-quvvatlaydi:

C++23 xususiyati	uz++ da mavjud
std::expected (Natija<T>)	☒
std::format	☒
std::ranges	☒
std::generator (coroutines)	☒
auto(x) decay copy	☒
if consteval	☒
Deducing this	☒
static operator()	☒
std::print / std::println	☒
std::mdspan	☒

16.6 F · Foydali resurslar

16.6.1 Rasmiy

- **Repozitoriy:** github.com/timetolivechk-spec/uzpp
- **Hujjatlar:** docs/ papkasi
- **Misollar:** misollar/ papkasi (15 ta misol)
- **Xatoliklar:** [GitHub Issues](#)
- **VS Code Marketplace:** [uzpp.uzpp](#)

16.6.2 Tashqi

- **CopperSpice:** copperspice.com
- **C++23 standarti:** eel.is/c++draft
- **cppreference:** en.cppreference.com

16.7 G · Eng ko'p uchraydigan xatoliklar

Xatolik	Sababi	Yechim
Xato: Fayl topilmadi	Noto'g'ri fayl yo'li	Absolyut yo'ldan foydalaning yoki CWD ni tekshiring
asosiy() qaytish turi butun bo'lishi kerak	bosh asosiy()	butun asosiy() deb yozing
Kalit so'z o'zgaruvchi nomi sifatida ishlatilishi mumkin emas	O'zgaruvchi nomi kalit so'z bilan ustma-ust tushdi	Boshqa nom tanlang
Yopilmagan belgi literal ld returned 1 exit status	Apostrof noto'g'ri yopilgan Havola xatosi yoki disk to'lgan	' juftligini tekshiring Disk joyini tekshiring, --bare flag'ini sinab ko'ring
uzpp_runtime.hpp: No such file	--bare flag'i kerak	--bare qo'shing yoki -I stdlib
CsSignal::connect xatosi	CopperSpice sarlavhalari topilmadi	-I C:/CopperSpice/include/... qo'shing
0 ga bo'lib bo'lmadi out_of_range	Bo'luvchi 0 Vektor/massiv chegaradan tashqari	agar (b != 0) tekshiring Indeks va hajmni tekshiring
bad_alloc	Xotira tugadi	Memory leak yoki katta hajm

16.8 H · Kitob bo'yicha xulosa

Tabriklayman! Siz uz++ kitobini o'qib tamomladingiz. Endi sizning oldingizda dasturlash dunyosining keng maydoni ochildi.

16.8.1 Siz nimani o'rgandingiz?

- **Dasturlash asoslari** — o'zgaruvchilar, turlar, operatorlar
- **Mantiq va boshqaruv** — agar, sikllar, funksiyalar
- **Xotira boshqaruvi** — ko'rsatkichlar, havolalar, aqlli ko'rsatkichlar

- **Ma'lumotlar tuzilmalari** — vektor, xarita, to'plam, ro'yxat
- **OOP** — sinflar, meros, polimorfizm
- **Umumlashgan dasturlash** — shablonlar, STL
- **Xatoliklar boshqaruvi** — istisnolar, Natija, Tanlov
- **Matn va fayllar** — kuchli matn ishlash, JSON, fayl tizimi
- **Ko'p oqimlilik** — parallel dasturlash, sinxronizatsiya
- **GUI** — CopperSpice bilan grafik dastur yaratish

16.8.2 Endi nima qilish?

1. **Loyihalar qiling.** Kitobni o'qish — bu birinchi qadam. Mahorat amaliyot orqali keladi.
2. **Boshqalarning kodini o'qing.** [misollar/](#) papkasidagi 15 ta misolni ko'rib chiqing.
3. **O'z kutubxonangizni yarating.** Foydali bo'lgan funksiyalarni to'plang.
4. **Hamjamiyatga qo'shiling.** Savol bering, javob bering, ko'maklashing.
5. **Open source ga hissa qo'shing.** `uz++` ning o'zini ham yaxshilashga yordam berishingiz mumkin!

16.8.3 So'nggi maslahat

Yaxshi dasturchining sirlari:

- **Har kuni kod yozing.** Hatto 30 daqiqa.
- **Xato qilishdan qo'rqmang.** Har bir xato — bu o'rganish.
- **Tezroq emas, aniqroq.** "Tez ishlaydigan xato kod" — bu yomon kod.
- **Boshqalardan o'rganing.** Hech kim hamma narsani bilmaydi.
- **Boshqalarga o'rgating.** O'rgatish — eng yaxshi o'rganish.

`uz++` — bu siz uchun. O'z tilingizda, o'z istalgan jarayonda, o'z loyihalaringizni yarating!

Kitob yakuni. Ushbu qo'llanma sizga `uz++` dasturlash tilini o'rganishda yordam berdi degan umiddamiz. Savol, taklif yoki xatolik bo'lsa — [GitHub Issues](#) ga yozing.

`uz++` — o'zbek tilida zamonaviy dasturlash!

Kitob versiyasi: v2.2.1 | 2026-yil may | © Shaxriyor Malikov | MIT Litsenziyasi